



Integer multiplication in time $O(n \log n)$

David Harvey, Joris van Der Hoeven

► To cite this version:

David Harvey, Joris van Der Hoeven. Integer multiplication in time $O(n \log n)$. *Annals of Mathematics*, 2021, 10.4007/annals.2021.193.2.4 . hal-02070778v2

HAL Id: hal-02070778

<https://hal.science/hal-02070778v2>

Submitted on 28 Nov 2020

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Integer multiplication in time $O(n \log n)$

DAVID HARVEY AND JORIS VAN DER HOEVEN

ABSTRACT. We present an algorithm that computes the product of two n -bit integers in $O(n \log n)$ bit operations, thus confirming a conjecture of Schönhage and Strassen from 1971. Our complexity analysis takes place in the multitape Turing machine model, with integers encoded in the usual binary representation. Central to the new algorithm is a novel “Gaussian resampling” technique that enables us to reduce the integer multiplication problem to a collection of multidimensional discrete Fourier transforms over the complex numbers, whose dimensions are all powers of two. These transforms may then be evaluated rapidly by means of Nussbaumer’s fast polynomial transforms.

1. INTRODUCTION

Let $M(n)$ denote the time required to multiply two n -bit integers. We work in the multitape Turing model, in which the time complexity of an algorithm refers to the number of steps performed by a deterministic Turing machine with a fixed, finite number of linear tapes [35]. The main results of this paper also hold in the Boolean circuit model [41, Sec. 9.3], with essentially the same proofs.

For functions $f(n_1, \dots, n_k)$ and $g(n_1, \dots, n_k)$, we write $f(n) = O(g(n))$ to indicate that there exists a constant $C > 0$ such that $f(n) \leq Cg(n)$ for all tuples $n = (n_1, \dots, n_k)$ in the domain of f . Similarly, we write $f(n) = \Omega(g(n))$ to mean that $f(n) \geq Cg(n)$ for all n in the domain of f , and $f(n) = \Theta(g(n))$ to indicate that both $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$ hold. From Section 2 onwards we will always explicitly restrict the domain of f to ensure that $g(n) > 0$ throughout this domain. However, in this Introduction we will slightly abuse this notation: when writing for instance $f(n) = O(n \log n \log \log n)$, we tacitly assume that the domain of f has been restricted to $[n_0, \infty)$ for some sufficiently large threshold n_0 .

Schönhage and Strassen conjectured in 1971 that the true complexity of integer multiplication is given by $M(n) = \Theta(n \log n)$ [40], and in the same paper established their famous upper bound $M(n) = O(n \log n \log \log n)$. In 2007 their result was sharpened by Fürer to $M(n) = O(n \log n K^{\log^* n})$ [12, 13] for some unspecified constant $K > 1$, where $\log^* n$ denotes the iterated logarithm, i.e., $\log^* x := \min\{k \geq 0 : \log^{\circ k} x \leq 1\}$. Prior to the present work, the record stood at $M(n) = O(n \log n 4^{\log^* n})$ [22].

The main result of this paper is a verification of the upper bound in Schönhage and Strassen’s conjecture, thus closing the remaining $4^{\log^* n}$ gap:

Theorem 1.1. *There is an integer multiplication algorithm achieving*

$$M(n) = O(n \log n).$$

Harvey was supported by the Australian Research Council (grant FT160100219).

If the Schönhage–Strassen conjecture is correct, then Theorem 1.1 is asymptotically optimal. Unfortunately, no super-linear lower bound for $M(n)$ is known. Perhaps the best available evidence in favour of the conjecture is the $\Omega(n \log n)$ lower bound [6, 36] that has been proved for the “on-line” variant of the problem, in which the k -th bit of the product must be written before the $(k + 1)$ -th bits of the multiplicands are read. Again, the true complexity of on-line multiplication is not known: currently, the best known upper bound is $O(n \log n \exp(C\sqrt{\log \log n}))$ for $C = \sqrt{2 \log 2} + o(1)$ [29].

Theorem 1.1 has many immediate consequences, as many computational problems may be reduced to integer multiplication. For example, the theorem implies that quotients and k -th roots of real numbers may be computed to a precision of n significant bits in time $O(n \log n)$, that transcendental functions and constants such as e^x and π may be computed to precision n in time $O(n \log^2 n)$, and that the greatest common divisor of two n -bit integers may be found in time $O(n \log^2 n)$ [5].

Another interesting application is to the problem of computing DFTs (discrete Fourier transforms) over $\mathbb{C}$. Given a transform length $m \geq 2$ and a target accuracy of $p = \Omega(\log m)$ bits, it was pointed out in [20, 25] that one may use Bluestein’s trick [2] followed by Kronecker substitution [14, Corollary 8.27] to reduce a given DFT of length m to an integer multiplication problem of size $O(mp)$. Theorem 1.1 then implies that the DFT may be evaluated in time $O(mp \log(mp))$. This compares favourably with the traditional FFT (fast Fourier transform) approach, which requires $O(m \log m)$ operations in $\mathbb{C}$, and thus time $O(m \log m M(p)) = O(mp \log m \log p)$ in the Turing model.

This faster method for computing DFTs over $\mathbb{C}$ leads to various further applications. One such application is the conversion of an n -digit integer from one base to another, for example from binary to decimal, in time $O(n \log^2 n / \log \log n)$ [30]. Alternatively, if one wishes to multiply two n -digit integers in a fixed base $\beta \neq 2$, then it is possible to adapt the new algorithm to obtain a direct $O(n \log n)$ -time multiplication algorithm that works in base β throughout. This is asymptotically faster than reducing the problem to binary via the above-mentioned base conversion algorithms.

All of the algorithms presented in this paper can be made completely explicit, and all implied big- O constants are in principle effectively computable. On the other hand, we make no attempt to minimise these constants or to otherwise exhibit a practical multiplication algorithm. Our aim is to establish the theoretical $O(n \log n)$ bound as directly as possible.

We will actually describe two new multiplication algorithms. The first one depends on an unproved hypothesis concerning the least prime in an arithmetic progression. This hypothesis is weaker than standard conjectures in this area, but stronger than the best unconditional results currently available. We give only a brief sketch of this algorithm (see Section 1.2.1); a detailed treatment is given in the companion paper [24], which also presents an analogue of this algorithm for multiplication in $\mathbb{F}_q[x]$. The bulk of the present paper (Sections 2–5) concentrates on working out the details of the second algorithm, which is technically more involved, but has the virtue of reaching the $O(n \log n)$ bound unconditionally.

In the remainder of Section 1, we review the literature on integer multiplication (Section 1.1), and give an overview of the new algorithms (Section 1.2).

1.1. Survey of integer multiplication algorithms. The first improvement on the classical $M(n) = O(n^2)$ bound was found by Karatsuba in 1962. Significant progress was made during the 1960s by Toom, Cook, Schönhage and Knuth; see [25, Sec. 1.1] for further historical details and references for this period. FFTs were brought into the picture by Schönhage and Strassen [40] soon after the publication of the FFT by Cooley and Tukey [7]; see [28] for more on the history of the FFT. The multiplication algorithms published since [40] may be roughly classified into four families:

(1) *Schönhage–Strassen’s first algorithm* [40] is, in retrospect, the most straightforward FFT-based integer multiplication algorithm imaginable. By splitting the n -bit multiplicands into chunks of size $\Theta(\log n)$, they reduce to the problem of multiplying polynomials in $\mathbb{Z}[x]$ of degree $\Theta(n/\log n)$ and coefficient size $\Theta(\log n)$. The product in $\mathbb{Z}[x]$ is handled by means of FFTs over $\mathbb{C}$, i.e., evaluating the polynomials at suitable complex roots of unity, multiplying their values pointwise in $\mathbb{C}$, and then interpolating to obtain the product polynomial. Elements of $\mathbb{C}$ are represented approximately, with a precision of $\Theta(\log n)$ bits. Arithmetic operations in $\mathbb{C}$ (such as multiplication) are reduced to arithmetic in $\mathbb{Z}$ by scaling by a suitable power of two. This leads to the recursive estimate

$$M(n) = O(n M(n')) + O(n \log n), \quad n' = O(\log n),$$

whose explicit solution is

$$M(n) = O(K^{\log^* n} n \log n \log \log n \dots \log^{o((\log^* n)-1)} n)$$

for some constant $K > 0$. The algorithm achieves an exponential size reduction at each recursion level, from n to $O(\log n)$, and the number of levels is $\log^* n + O(1)$.

Pollard suggested a similar algorithm at around the same time [37], working over a finite field rather than $\mathbb{C}$. He did not analyse the bit complexity, but with some care one can prove essentially the same complexity bound as for the complex case (some technical difficulties arise due to the cost of finding suitable primes; these may be resolved by techniques similar to those discussed in [25, Sec. 8.2]).

(2) *Schönhage–Strassen’s second algorithm* is the more famous and arguably the more ingenious of the two algorithms presented in [40]. It is probably the most widely used large-integer multiplication algorithm in the world today, due to the highly optimised implementation included in the free GNU Multiple Precision Arithmetic Library (GMP) [17, 15], which underlies the large-integer capabilities of all of the major contemporary computer algebra systems.

The basic recursive problem is taken to be multiplication in $\mathbb{Z}/(2^n+1)\mathbb{Z}$, where n is a power of two. Let $n' := 2^{\lceil (\log_2 2n)/2 \rceil} = \Theta(n^{1/2})$ and $T := 2n/n' = \Theta(n^{1/2})$, so that $(n')^2 \in \{2n, 4n\}$ and $T \mid n'$; then by splitting the inputs into chunks of size $n'/2$, the problem is reduced to multiplication in $R[x]/(x^T+1)$ where $R := \mathbb{Z}/(2^{n'}+1)\mathbb{Z}$. The powers of 2 in R are sometimes called “synthetic” roots of unity, as they have been synthesised algebraically, or “fast” roots of unity, as one can multiply an element of R by an arbitrary power of 2 in linear time, i.e., in time $O(n')$. Consequently, for $\omega := 2^{n'/T}$, one may evaluate a polynomial at $\omega, \omega^3, \dots, \omega^{2T-1}$ (the roots of x^T+1) via the FFT in time $O((n' \log n')n') = O(n \log n)$. The original multiplication problem is thus reduced to T pointwise multiplications in R , which are handled recursively. Writing $M_1(n)$ for the cost of a product in $\mathbb{Z}/(2^n+1)\mathbb{Z}$,

one obtains the recurrence

$$(1.1) \quad M_1(n) < \frac{2n}{n'} M_1(n') + O(n \log n), \quad n' = O(n^{1/2}).$$

Unlike the first Schönhage–Strassen algorithm, this algorithm performs only a *geometric* size reduction, from n to $O(n^{1/2})$, at each recursion level, and the number of recursion levels is $\log_2 \log n + O(1) = O(\log \log n)$.

The constant 2 in (1.1), which arises from zero-padding in the initial splitting stage, plays a crucial role in the complexity analysis: it ensures that at each recursion level, the total cost of the “fast” FFTs remains $O(n \log n)$, with the same implied constant at each level. The overall cost is thus $M_1(n) = O(n \log n \log \log n)$.

(3) *Fürer’s algorithm* [12, 13] combines the best features of the two Schönhage–Strassen algorithms: the exponential size reduction from the first algorithm, and the fast roots of unity from the second one. The overall strategy is similar to the first algorithm, but instead of working over $\mathbb{C}$, one uses a bivariate splitting to reduce to a polynomial multiplication problem over $R := \mathbb{C}[y]/(y^r + 1)$, where $r = \Theta(\log n)$ is a power of two. This ring contains a synthetic root of unity y of order $2r$, but also inherits higher-order roots of unity from $\mathbb{C}$. Elements of $\mathbb{C}$ are represented approximately, with a precision of $O(\log n)$ bits; thus an element of R occupies $O((\log n)^2)$ bits.

Fürer’s key insight is to apply the Cooley–Tukey FFT decomposition in radix $2r$ instead of radix two. He decomposes each “long” transform of length $\Theta(n/(\log n)^2)$ into many “short” transforms of length $2r$, with one round of expensive “twiddle factor” multiplications interposed between each layer of short transforms. The short transforms take advantage of the synthetic roots of unity, and the twiddle factor multiplications are handled recursively (via Kronecker substitution). This leads to the recurrence

$$M(n) = O\left(\frac{n \log n}{n' \log n'} M(n')\right) + O(n \log n), \quad n' = O((\log n)^2),$$

and then to the explicit bound $M(n) = O(n \log n K^{\log^* n})$ for some constant $K > 1$. Fürer did not give a specific value for K , but it is argued in [25, Sec. 7] that careful optimisation of his algorithm leads to the value $K = 16$.

Several authors have given variants of Fürer’s algorithm that also achieve $M(n) = O(n \log n K^{\log^* n})$, using essentially the same idea but working over different rings. De, Kurur, Saha and Saptharishi [10] replace $\mathbb{C}$ by a p -adic ring $\mathbb{Q}_p$; this has the benefit of avoiding numerical analysis over $\mathbb{C}$, but the value of K becomes somewhat larger. Covanov and Thomé give another variant that achieves $K = 4$, conditional on a conjecture on the distribution of generalised Fermat primes [8].

(4) *The Harvey–van der Hoeven–Lecerf algorithm* [25] follows Fürer in decomposing a “long” transform into many “short” transforms of exponentially smaller length. However, instead of working over a ring containing fast roots of unity, one works directly over $\mathbb{C}$ (as in the first Schönhage–Strassen algorithm), and converts the short transforms back to multiplication problems via Bluestein’s trick [2]. These short products are then handled recursively.

The first version given in [25] achieved $M(n) = O(n \log n K^{\log^* n})$ with $K = 8$. The value of K was improved gradually over a sequence of papers [18, 19, 21], reaching $K = 4$ in [22]. All of these algorithms perform exponential size reduction, and the number of recursion levels is $\log^* n + O(1)$.

An interesting feature of these algorithms — related to the fact that they dispense with the need for fast roots of unity — is that they can be adapted to prove bounds of the form $O(n \log n K^{\log^* n})$ for the cost of multiplying polynomials in $\mathbb{F}_q[x]$ of degree n (for fixed q). This was first established with the constant $K = 8$ in [26], and improved to $K = 4$ in [23]. As mentioned previously, the first of the two new algorithms presented in this paper may be adapted to obtain an $O(n \log n)$ bound for the $\mathbb{F}_q[x]$ case [24], but unfortunately this result is still conditional and so does not yet supersede the unconditional $O(n \log n 4^{\log^* n})$ bound given in [23].

1.2. Overview of new algorithms. Our new algorithms are motivated by the observation that certain *multivariate* polynomial rings admit particularly efficient multiplication algorithms. Let r be a power of two, and for $d \geq 2$ consider the ring

$$(1.2) \quad R[x_1, \dots, x_{d-1}] / (x_1^{t_1} - 1, \dots, x_{d-1}^{t_{d-1}} - 1), \quad R := \mathbb{C}[y] / (y^r + 1),$$

where $t_i \mid 2r$ for all i . One may multiply in this ring by first using FFTs to evaluate each x_i at the synthetic t_i -th roots of unity (the powers of y^{2r/t_i}), then multiplying pointwise in R , and finally performing inverse FFTs. Such transforms were studied extensively by Nussbaumer in the late 1970s (see for example [32]), and are sometimes known as *fast polynomial transforms*. They consist entirely of additions and subtractions in $\mathbb{C}$, and require no multiplications in $\mathbb{C}$ whatsoever.

In Sections 1.2.1 and 1.2.2 below, we outline two different ways of fashioning an integer multiplication algorithm from the polynomial multiplication algorithm just described. The key issue is to show how to transport an integer multiplication problem, which is intrinsically one-dimensional, to a ring of the type (1.2).

In both cases, we begin with the following setup. Suppose that we wish to multiply two n -bit integers. We choose a dimension parameter $d \geq 2$ and distinct primes $s_1, \dots, s_d \approx (n/\log n)^{1/d}$, subject to certain conditions that will be explained in Sections 1.2.1 and 1.2.2. Just as in the first Schönhage–Strassen algorithm, we split the inputs into around $n/\log n$ chunks of roughly $\log n$ bits, thereby reducing the problem to multiplication in $\mathbb{Z}[x]/(x^{s_1 \cdots s_d} - 1)$. Now, following a technique described by Agarwal and Cooley [1] (which is closely related to the Good–Thomas FFT algorithm [16, 42]), we observe that the Chinese remainder theorem induces an isomorphism $\mathbb{Z}[x]/(x^{s_1 \cdots s_d} - 1) \cong \mathbb{Z}[x_1, \dots, x_d]/(x_1^{s_1} - 1, \dots, x_d^{s_d} - 1)$, so the problem amounts to computing a product in the latter ring. For this, it suffices to show how to efficiently compute a multidimensional complex DFT of size $s_1 \times \cdots \times s_d$, i.e., with respect to the complex s_i -th roots of unity, to an accuracy of $O(\log n)$ bits.

1.2.1. A conditional algorithm — Rader’s trick. Suppose that we are able to choose the primes $s_1, \dots, s_d$ so that $s_i \equiv 1 \pmod{r}$, where $r \geq 2$ is a power of two, and where the s_i are not much larger than r . We may then deploy a multidimensional generalisation of *Rader’s algorithm* [38] to reduce the given DFT of size $s_1 \times \cdots \times s_d$ to a multiplication problem in the ring $\mathbb{C}[x_1, \dots, x_d]/(x_1^{s_1-1} - 1, \dots, x_d^{s_d-1} - 1)$ (together with some lower-dimensional multiplication problems of negligible cost). Crucially, the convolution lengths have been reduced from s_i to $s_i - 1$. Writing $s_i - 1 = q_i r$, where the q_i are “small”, we may further reduce this product to a collection of complex DFTs of size $q_1 \times \cdots \times q_d$, plus a collection of multiplication problems in $\mathbb{C}[x_1, \dots, x_d]/(x_1^r - 1, \dots, x_d^r - 1)$. After replacing x_d with $e^{\pi i/r} y$, we see that the latter products are exactly of the type (1.2). As discussed previously, we may use synthetic FFTs to reduce such a product to a collection of pointwise

products in $R = \mathbb{C}[y]/(y^r + 1)$. These in turn are converted to integer multiplication problems via Kronecker substitution, and then handled recursively.

The main sticking point in the above algorithm is the cost of the auxiliary DFTs of size $q_1 \times \cdots \times q_d$. There are various options available for evaluating these DFTs, but to ensure that this step does not dominate the complexity, the key issue is to keep the size of the q_i under control. What we are able to prove is the following. For positive, relatively prime integers r and a , define

$$P(a, r) := \min\{s > 0 : s \text{ prime and } s = a \bmod r\},$$

and put $P(r) := \max_a P(a, r)$. *Linnik's theorem* states that there is an absolute constant $L > 1$ such that $P(r) = O(r^L)$. (In our application, we are interested in bounding $P(1, r)$ when r is a power of two.) The best published value for L is currently $L = 5.18$ [43], and under the Generalised Riemann Hypothesis one may take $L = 2 + \varepsilon$ for any $\varepsilon > 0$ [27]. In the companion paper [24], we present an integer multiplication algorithm following the plan just described, but working over a finite field instead of $\mathbb{C}$. We prove that if Linnik's theorem holds for some $L < 1 + \frac{1}{303}$, and if we take d near 10^6 , then the cost of the auxiliary DFTs can be controlled and one does in fact obtain an overall $M(n) = O(n \log n)$ bound. We expect that the same argument works over $\mathbb{C}$, with a possibly different threshold for L , but we have not worked out the details.

On the other hand, it is widely expected that the bound $P(r) = O(r^L)$ should hold for *any* $L > 1$. For this reason, we strongly suspect that the algorithm sketched above does run in time $O(n \log n)$, despite us being unable to supply a proof. For further discussion, and examples of even stronger bounds for $P(r)$ that are expected to hold, see [24].

Remark 1.2. The idea of evaluating a multidimensional transform via a combination of Rader's algorithm and polynomial transforms was previously suggested in a different context by Nussbaumer and Quandalle [33, p. 141].

1.2.2. An unconditional algorithm — Gaussian resampling. The rest of the paper is devoted to the second method. Here we choose the primes $s_1, \dots, s_d$ in such a way that each s_i is slightly smaller than a power of two t_i , and so that $t_1 \cdots t_d = O(s_1 \cdots s_d)$. Finding such primes is easily accomplished using Eratosthenes' sieve and the prime number theorem with a suitable error term (see Lemma 5.1).

Assume as before that we wish to compute a complex multidimensional DFT of size $s_1 \times \cdots \times s_d$, to an accuracy of $O(\log n)$ bits. Our key innovation is to show that *this problem may be reduced directly to the problem of computing a complex multidimensional DFT of size $t_1 \times \cdots \times t_d$.*

The idea of the reduction is as follows. Suppose that we are given as input an $s_1 \times \cdots \times s_d$ array of complex numbers $u = (u_{j_1, \dots, j_d})_{0 \leq j_i < s_i}$. We may regard this array as lying inside the d -dimensional unit torus $(\mathbb{R}/\mathbb{Z})^d$: we imagine the coefficient $u_{j_1, \dots, j_d}$ to be plotted at coordinates $(j_1/s_1, \dots, j_d/s_d)$ in the torus (see Figure 1). We construct from u an intermediate $t_1 \times \cdots \times t_d$ array $v = (v_{k_1, \dots, k_d})_{0 \leq k_i < t_i}$. Again, we think of $v_{k_1, \dots, k_d}$ as being plotted at coordinates $(k_1/t_1, \dots, k_d/t_d)$ in the torus. The coefficients of v are defined to be certain linear combinations of the coefficients of u . The weights are essentially d -dimensional Gaussians, so each coefficient of v depends mainly on the “nearby” coefficients of u within the torus.

This construction has two crucial properties. First, the rapid decay of the Gaussians allows us to compute (i.e., approximate) the coefficients of v very quickly from

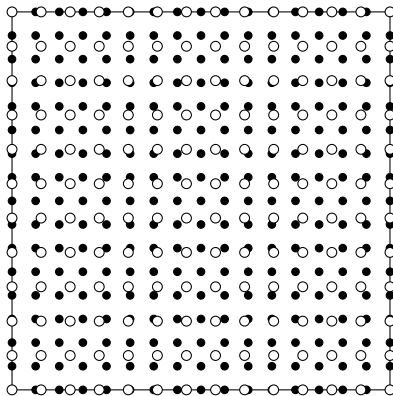


FIGURE 1. Torus $(\mathbb{R}/\mathbb{Z})^2$ with 13×11 source array (white circles) superimposed over 16×16 target array (black circles)

those of u ; indeed, the cost of this step is asymptotically negligible compared to the cost of the DFTs themselves. Second, using the fact that the Fourier transform of a Gaussian is a Gaussian, we will show that $\hat{u}$ and $\hat{v}$ (the DFTs of u and v) are related by a fairly simple system of linear equations. In fact, the matrix of this system is of the same type as the matrix relating u and v . The system is somewhat overdetermined, because $t_1 \cdots t_d > s_1 \cdots s_d$. Provided that the ratios t_i/s_i are not too close to 1, we will show that this system may be solved in an efficient and numerically stable manner, and that we may therefore recover $\hat{u}$ from $\hat{v}$. This procedure forms the core of our “Gaussian resampling” method, and is developed in detail in Section 4. It is closely related to the Dutt–Rokhlin algorithm for non-equispaced FFTs [11]; see Section 4.4.3 for a discussion of the similarities and differences.

We have therefore reduced to the problem of computing $\hat{v}$ from v , and we are free to do this by any convenient method. Note that this is a DFT of size $t_1 \times \cdots \times t_d$ rather than $s_1 \times \cdots \times s_d$. In Section 3 we will show how to use a multivariate generalisation of Bluestein’s algorithm [2] to reduce this DFT to a multiplication problem in a ring of the form (1.2). As already pointed out, such a product may be handled efficiently via synthetic FFTs; the details of this step are also discussed in Section 3.

Analysis of this algorithm leads to a recurrence inequality of the form

$$(1.3) \quad M(n) < \frac{Kn}{n'} M(n') + O(n \log n), \quad n' = n^{\frac{1}{d} + o(1)},$$

where K is an absolute constant, and in particular does not depend on d . (In Section 5 we establish (1.3) with the explicit constant $K = 1728$, and in Section 5.4 we list some optimisations that improve it to $K = 8$.) The first term arises from pointwise multiplications in a ring of the type $R = \mathbb{C}[y]/(y^r + 1)$, and the second term from the fast FFTs and other auxiliary operations, including computing v from u and recovering $\hat{u}$ from $\hat{v}$.

We stress here the similarity with the corresponding bound (1.1) for the second Schönhage–Strassen algorithm; the difference is that we are now free to choose d . In Section 5, we will simply take $d := 1729$ (any constant larger than K would do), and then it is easy to see that (1.3) implies that $M(n) = O(n \log n)$. (A similar

analysis holds for the conditional algorithm sketched in Section 1.2.1, for different values of K and d .)

It is striking that for fixed d , the new algorithm performs only a geometric size reduction at each recursion level, just like the second Schönhage–Strassen algorithm, and unlike the first Schönhage–Strassen algorithm or any of the post-Fürer algorithms. In the new algorithm, the total cost of the FFTs actually *decreases* by the constant factor $d/K > 1$ at each subsequent recursion level, unlike in the second Schönhage–Strassen algorithm, where it remains constant at each level, or any of the other algorithms mentioned, where it increases by a constant factor at each level.

Actually, it is possible to allow d to grow with n , so as to achieve size reduction faster than geometric. With some care, this leads to a better constant in the main $O(n \log n)$ bound, by shifting more work from the pointwise multiplications into the fast FFTs. We will not carry out the details of this analysis.

Finally, we mention that our reduction from a DFT of size $s_1 \times \cdots \times s_d$ to one of size $t_1 \times \cdots \times t_d$ is highly non-algebraic, and depends heavily on the archimedean property of $\mathbb{R}$. Consequently, we do not know how to give an analogue of this algorithm for multiplication in $\mathbb{F}_q[x]$.

Acknowledgments. The authors would like to thank the anonymous referees, whose thoughtful comments helped to improve the presentation of these results.

2. DFTs, CONVOLUTIONS AND FIXED-POINT ARITHMETIC

In the Turing model we cannot compute with elements of $\mathbb{C}$ exactly. In this section we introduce a technical framework for systematic discussion of DFTs and convolutions in the setting of fixed-point arithmetic. This framework is loosely based on the presentation in [25, Sec. 3], and will be used throughout the rest of the paper. The impatient reader may skip most of the section and use it as a reference in case of doubt. To this effect, Table 1 contains a summary of the notation introduced in this section.

2.1. Integer arithmetic. Integers are assumed to be stored in the standard binary representation. We briefly recall several well-known results concerning integer arithmetic; see [5, Ch. 1] for further details and literature references.

Let $n \geq 1$, and assume that we are given as input $x, y \in \mathbb{Z}$ such that $|x|, |y| \leq 2^n$. We may compute $x + y$ and $x - y$ in time $O(n)$. For multiplication, we will often use the crude estimate $M(n) = O(n^{1+\delta})$, where for the rest of the paper δ denotes a small, fixed positive quantity; for definiteness, we assume that $\delta < \frac{1}{8}$. If $y > 0$, then we may compute the quotients $\lfloor x/y \rfloor$ and $\lceil x/y \rceil$ in time $O(n^{1+\delta})$. More generally, for a fixed positive rational number a/b , and assuming $x, y > 0$, we may compute $\lfloor (x/y)^{a/b} \rfloor$ and $\lceil (x/y)^{a/b} \rceil$ in time $O(n^{1+\delta})$.

2.2. Fixed-point coordinate vectors. Fix a precision parameter $p \geq 100$. Let $\mathbb{C}_o := \{u \in \mathbb{C} : |u| \leq 1\}$ denote the complex unit disc, and set

$$\tilde{\mathbb{C}}_o := (2^{-p}\mathbb{Z}[i]) \cap \mathbb{C}_o = \{2^{-p}(x + iy) : x, y \in \mathbb{Z} \text{ and } x^2 + y^2 \leq 2^{2p}\}.$$

In the Turing model, we represent an element $z = 2^{-p}(x + iy) \in \tilde{\mathbb{C}}_o$ by the pair of integers (x, y) . It occupies $O(p)$ bits of storage, as $|x|, |y| \leq 2^p$. The precision p is always known from context and does not need to be stored alongside z .

$\delta \in (0, \frac{1}{8})$	a constant such that $M(n) = O(n^{1+\delta})$	(§2.1)
$p \geq 100$	working precision in bits	(§2.2)
$\ \cdot\ $	supremum norm on V (a finite-dimensional vector space over $\mathbb{C}$) with respect to a specified basis B_V	(§2.2)
$V_\circ$	closed unit ball in V under the supremum norm	(§2.2)
$\tilde{V}_\circ$	set of vectors in $V_\circ$ whose coordinates are fixed-point complex numbers with p bits of accuracy	(§2.2)
$\rho: V_\circ \rightarrow \tilde{V}_\circ$	round-towards-zero function	(§2.2)
$\tilde{v} \in \tilde{V}_\circ$	a fixed-point approximation to a vector $v \in V_\circ$	(§2.2)
$\varepsilon(\tilde{v})$	(scaled) error incurred in approximating v by $\tilde{v}$	(§2.2)
$\mathcal{F}_n: \mathbb{C}^n \rightarrow \mathbb{C}^n$	complex DFT of length n	(§2.4)
$\mathcal{F}_{n_1, \dots, n_d}$	multidimensional complex DFT of size $n_1 \times \dots \times n_d$	(§2.4)
$\mathcal{R}$	the ring $\mathbb{C}[y]/(y^r + 1)$, for a power of two $r \geq 2$	(§2.3)
$\mathcal{G}_n: \mathcal{R}^n \rightarrow \mathcal{R}^n$	synthetic DFT of length n , where $n \mid 2r$	(§2.4)
$\mathcal{G}_{n_1, \dots, n_d}$	multidimensional synthetic DFT of size $n_1 \times \dots \times n_d$	(§2.4)
$u \cdot v$	pointwise product of vectors/tensors	(§2.4)
$u * v$	convolution product of vectors/tensors	(§2.4)
$\mathcal{A}$	a linear (or bilinear) map between finite-dimensional complex vector spaces	(§2.6)
$\ \mathcal{A}\ $	operator norm of $\mathcal{A}$	(§2.6)
$\tilde{\mathcal{A}}$	a “numerical approximation” for $\mathcal{A}$ (assuming $\ \mathcal{A}\ \leq 1$), i.e., a computable function intended to approximate $\mathcal{A}$ on fixed-point inputs	(§2.6)
$\varepsilon(\tilde{\mathcal{A}})$	worst-case error incurred in approximating $\mathcal{A}$ by $\tilde{\mathcal{A}}$	(§2.6)
$C(\tilde{\mathcal{A}})$	worst-case cost of evaluating $\tilde{\mathcal{A}}$ on a single vector	(§2.6)

TABLE 1. Glossary of notation introduced in Section 2

We define a round-towards-zero function $\rho: \mathbb{C} \rightarrow \mathbb{C}$ as follows. First, define $\rho_0: \mathbb{R} \rightarrow \mathbb{Z}$ by $\rho_0(x) := \lfloor x \rfloor$ for $x \geq 0$, and $\rho_0(x) := \lceil x \rceil$ for $x < 0$. Then define $\rho_0: \mathbb{C} \rightarrow \mathbb{Z}[i]$ by setting $\rho_0(x + iy) := \rho_0(x) + i\rho_0(y)$ for $x, y \in \mathbb{R}$. Observe that $|\rho_0(u)| \leq |u|$ and $|\rho_0(u) - u| < \sqrt{2}$ for any $u \in \mathbb{C}$. Finally, set

$$\rho(u) := 2^{-p} \rho_0(2^p u), \quad u \in \mathbb{C}.$$

Thus $|\rho(u)| \leq |u|$ and $|\rho(u) - u| < \sqrt{2} \cdot 2^{-p}$ for any $u \in \mathbb{C}$. Clearly $\rho(\mathbb{C}_\circ) \subset \tilde{\mathbb{C}}_\circ$.

Now let V be a nonzero, finite-dimensional vector space over $\mathbb{C}$. In this paper, every such V is understood to come equipped with a privileged choice of ordered basis $B_V = \{b_0, \dots, b_{m-1}\}$, where $m = \dim_{\mathbb{C}} V \geq 1$. For the special case $V = \mathbb{C}^m$, we always take the standard basis; in particular, for $V = \mathbb{C}$ the basis is simply $\{1\}$.

We define a norm $\|\cdot\|: V \rightarrow [0, \infty)$ in terms of the basis B_V by setting

$$\|\lambda_0 b_0 + \dots + \lambda_{m-1} b_{m-1}\| := \max_j |\lambda_j|, \quad \lambda_j \in \mathbb{C}.$$

This norm satisfies $\|u + v\| \leq \|u\| + \|v\|$ and $\|\lambda u\| = |\lambda| \|u\|$ for any $u, v \in V$, $\lambda \in \mathbb{C}$. The unit ball in V is defined to be

$$V_\circ := \{u \in V : \|u\| \leq 1\} = \{\lambda_0 b_0 + \dots + \lambda_{m-1} b_{m-1} : \lambda_j \in \mathbb{C}_\circ\},$$

and we also define

$$\tilde{V}_\circ := \{\lambda_0 b_0 + \cdots + \lambda_{m-1} b_{m-1} : \lambda_j \in \tilde{\mathbb{C}}_\circ\}.$$

We extend ρ to a function $\rho: V \rightarrow V$ by acting componentwise, i.e., we put

$$\rho(\lambda_0 b_0 + \cdots + \lambda_{m-1} b_{m-1}) := \sum_j \rho(\lambda_j) b_j, \quad \lambda_j \in \mathbb{C}.$$

Then $\|\rho(u)\| \leq \|u\|$ and $\|\rho(u) - u\| < \sqrt{2} \cdot 2^{-p}$ for any $u \in V$. Clearly $\rho(V_\circ) \subset \tilde{V}_\circ$.

In the special case $V = \mathbb{C}$ we have simply $\|u\| = |u|$ for any $u \in \mathbb{C}$, and the notations $\mathbb{C}_\circ$, $\tilde{\mathbb{C}}_\circ$ and $\rho: \mathbb{C} \rightarrow \mathbb{C}$ all agree with their previous definitions.

In the Turing model, an element $u \in \tilde{V}_\circ$ is represented by its coordinate vector with respect to B_V , i.e., as a list of m elements of $\tilde{\mathbb{C}}_\circ$, so u occupies $O(mp)$ bits of storage.

For $u \in V_\circ$, we systematically use the notation $\tilde{u} \in \tilde{V}_\circ$ to indicate a fixed-point approximation for u that has been computed by some algorithm. We write

$$\varepsilon(\tilde{u}) := 2^p \|\tilde{u} - u\|$$

for the associated error, measured as a multiple of 2^{-p} (the “unit in the last place”). For example, we have the following result for addition and subtraction in V .

Lemma 2.1 (Addition/subtraction). *Recall that $m = \dim_{\mathbb{C}} V \geq 1$. Given as input $u, v \in \tilde{V}_\circ$, in time $O(mp)$ we may compute an approximation $\tilde{w} \in \tilde{V}_\circ$ for $w := \frac{1}{2}(u \pm v) \in V_\circ$ such that $\varepsilon(\tilde{w}) < 1$.*

Proof. Consider first the case $m = 1$, i.e., assume that $V = \mathbb{C}$. Let $u = 2^{-p}a$ and $v = 2^{-p}b$ where $a, b \in \mathbb{Z}[i]$ and $|a|, |b| \leq 2^p$. Since the denominators of the real and imaginary parts of $2^p w = \frac{1}{2}(a \pm b)$ are at most 2, we have $|\rho_0(2^p w) - 2^p w| \leq ((\frac{1}{2})^2 + (\frac{1}{2})^2)^{1/2} = \frac{1}{\sqrt{2}}$. Define $\tilde{w} := \rho(w) = 2^{-p} \rho_0(2^p w)$. We may clearly compute $\tilde{w}$ in time $O(p)$, and $\varepsilon(\tilde{w}) = 2^p \|\rho(w) - w\| \leq \frac{1}{\sqrt{2}} < 1$. The general case ($m \geq 1$) follows by applying the same argument in each coordinate. $\square$

Occasionally we will encounter a situation in which we have computed an approximation $\tilde{u} \in \tilde{V}_\circ$ for some $u \in V$, and we wish to compute an approximation for cu , where $c \geq 1$ is a fixed integer scaling factor for which it is known that $cu \in V_\circ$. A typical example is the final scaling step in an inverse FFT. Unfortunately, the obvious approximation $c\tilde{u}$ might lie just outside $V_\circ$. To simplify subsequent estimates, it will be technically convenient to adjust $c\tilde{u}$ slightly to obtain a vector that is guaranteed to lie in $V_\circ$. This adjustment may be carried out as follows.

Lemma 2.2 (Scaling). *Recall that $m = \dim_{\mathbb{C}} V \geq 1$. Let $u \in V$ and let c be an integer such that $1 \leq c \leq 2^p$. Assume that $\|u\| \leq c^{-1}$, and let $v := cu \in V_\circ$. Given as input c and an approximation $\tilde{u} \in \tilde{V}_\circ$, in time $O(mp^{1+\delta})$ we may compute an approximation $\tilde{v} \in \tilde{V}_\circ$ such that $\varepsilon(\tilde{v}) < 2c \cdot \varepsilon(\tilde{u}) + 3$.*

Proof. Again it suffices to handle the case $m = 1$, $V = \mathbb{C}$.

We first compute $2^{-p}(x + iy) := c\tilde{u}$ in time $O(p^{1+\delta})$. Note that $c\tilde{u}$ might not lie in $\tilde{\mathbb{C}}_\circ$, but x and y are certainly integers with $O(p)$ bits.

Next we compute $a := x^2 + y^2$ in time $O(p^{1+\delta})$, so that $a^{1/2} = 2^p |c\tilde{u}|$. If $a \leq 2^{2p}$ then already $c\tilde{u} \in \tilde{\mathbb{C}}_\circ$, so we may simply take $\tilde{v} := c\tilde{u}$, and then $\varepsilon(\tilde{v}) = 2^p |\tilde{v} - v| = 2^p |c\tilde{u} - cu| = c \cdot \varepsilon(\tilde{u}) < 2c \cdot \varepsilon(\tilde{u}) + 3$.

Suppose instead that $a > 2^{2p}$ (i.e., $c\tilde{u} \notin \tilde{\mathbb{C}}_\circ$). We then compute $b := \lceil a^{1/2} \rceil > 2^p$, again in time $O(p^{1+\delta})$. Let $z := 2^p c\tilde{u}/b = (x + iy)/b$ and $\tilde{v} := \rho(z)$. Note that $\tilde{v} = 2^{-p}(x' + iy')$ where $x' = \rho_0(2^p x/b)$ and $y' = \rho_0(2^p y/b)$, so we may compute $\tilde{v}$ in time $O(p^{1+\delta})$. We have $|\tilde{v}| \leq |z| = 2^p |c\tilde{u}|/b \leq 2^p |c\tilde{u}|/a^{1/2} = 1$, so indeed $\tilde{v} \in \tilde{\mathbb{C}}_\circ$. Moreover,

$$|\tilde{v} - v| \leq |\tilde{v} - z| + |z - c\tilde{u}| + |c\tilde{u} - v| = |\rho(z) - z| + |z| \left|1 - \frac{b}{2^p}\right| + c|\tilde{u} - u|,$$

so $\varepsilon(\tilde{v}) < \sqrt{2} + |2^p - b| + c \cdot \varepsilon(\tilde{u})$. We also have $2^p < b < a^{1/2} + 1 = 2^p |c\tilde{u}| + 1$, so

$$0 < b - 2^p < 2^p |c\tilde{u}| - 2^p + 1 \leq 2^p |cu| - 2^p + 2^p |c\tilde{u} - cu| + 1 \leq c \cdot \varepsilon(\tilde{u}) + 1.$$

We conclude that $|2^p - b| < c \cdot \varepsilon(\tilde{u}) + 1$, and therefore $\varepsilon(\tilde{v}) < 2c \cdot \varepsilon(\tilde{u}) + (1 + \sqrt{2})$. $\square$

2.3. Coefficient rings. By a *coefficient ring* we mean a finite-dimensional commutative $\mathbb{C}$ -algebra R with identity (together with a privileged basis B_R). We are chiefly interested in the following two examples:

- (1) *Complex case:* $\mathbb{C}$ itself, with the basis $\{1\}$.
- (2) *Synthetic case:* for a power of two $r \geq 2$, the ring $\mathcal{R} := \mathbb{C}[y]/(y^r + 1)$, with the basis $\{1, y, \dots, y^{r-1}\}$, so that $\|\lambda_0 + \lambda_1 y + \dots + \lambda_{r-1} y^{r-1}\| = \max_j |\lambda_j|$.

Let R be a coefficient ring of dimension $r \geq 1$ with basis B_R , and let $n \geq 1$. Then R^n is a vector space of dimension nr over $\mathbb{C}$. We associate to R^n the “nested” basis formed by concatenating n copies of B_R . In particular, we have $\|u\| = \max_j \|u_j\|$ for $u = (u_0, \dots, u_{n-1}) \in R^n$. In place of the awkward expressions $(R^n)_\circ$ and $(\tilde{R}^n)_\circ$, we write more compactly $R_\circ^n$ and $\tilde{R}_\circ^n$. In the Turing model, an element of $\tilde{R}_\circ^n$ occupies $O(nrp)$ bits of storage.

Now let $d \geq 1$ and $n_1, \dots, n_d \geq 1$. We write $\otimes_{i=1}^d R^{n_i}$, or just $\otimes_i R^{n_i}$ when d is understood, for the tensor product $R^{n_1} \otimes_R \dots \otimes_R R^{n_d}$. It is a free R -module of rank $n_1 \dots n_d$, and also a vector space over $\mathbb{C}$ of dimension $n_1 \dots n_d r$. An element $u \in \otimes_i R^{n_i}$ may be regarded as a d -dimensional array of elements of R of size $n_1 \times \dots \times n_d$. For indices $j_1, \dots, j_d$ where $0 \leq j_i < n_i$, we write $u_{j_1, \dots, j_d} \in R$ for the $(j_1, \dots, j_d)$ -th component of u .

We associate to $\otimes_i R^{n_i}$ the nested basis consisting of $n_1 \dots n_d$ copies of B_R arranged in lexicographical order, i.e., listing the coordinates of u in the order $(u_{0, \dots, 0}, u_{0, \dots, 1}, \dots, u_{n_1-1, \dots, n_d-1})$. Observe then that $\|u\| = \max_{j_1, \dots, j_d} \|u_{j_1, \dots, j_d}\|$. Instead of $(\otimes_i R^{n_i})_\circ$ and $(\otimes_i \tilde{R}^{n_i})_\circ$, we write $\otimes_i R_\circ^{n_i}$ and $\otimes_i \tilde{R}_\circ^{n_i}$. In the Turing model, an element of $\otimes_i \tilde{R}_\circ^{n_i}$ occupies $O(n_1 \dots n_d rp)$ bits of storage.

Let $u \in \otimes_i R^{n_i}$. By an *i-slice* of u we mean a one-dimensional sub-array of u , consisting of the entries $u_{j_1, \dots, j_d}$ where $j_1, \dots, j_{i-1}, j_{i+1}, \dots, j_d$ are held fixed and j_i varies over $\{0, \dots, n_i - 1\}$. We will occasionally wish to apply a given algorithm separately to each of the $n_1 \dots n_{i-1} n_{i+1} \dots n_d$ distinct i -slices of some $u \in \otimes_i \tilde{R}_\circ^{n_i}$. To accomplish this in the Turing model, we must first rearrange the data so that each i -slice is stored contiguously. In the lexicographical order specified above, this amounts to performing $n_1 \dots n_{i-1}$ matrix transpositions of size $n_i \times (n_{i+1} \dots n_d)$. This data rearrangement may be performed in time $O(n_1 \dots n_d rp \log n_i)$ (assuming $n_i \geq 2$) using a fast matrix transposition algorithm [4, Appendix].

2.4. DFTs and convolutions. Let R be a coefficient ring and let $n \geq 1$. Throughout the paper we adopt the convention that for a vector $u = (u_0, \dots, u_{n-1}) \in R^n$

and an integer j , the expression u_j always means $u_{j \bmod n}$. For $u, v \in R^n$, we define the pointwise product $u \cdot v \in R^n$ and the convolution product $u * v \in R^n$ by

$$(u \cdot v)_j := u_j v_j, \quad (u * v)_j := \sum_{k=0}^{n-1} u_k v_{j-k}, \quad 0 \leq j < n.$$

Then $(R^n, \cdot)$ and $(R^n, *)$ are both commutative rings, isomorphic respectively to the direct sum of n copies of R , and the polynomial ring $R[x]/(x^n - 1)$.

A *principal n -th root of unity in R* is an element $\omega \in R$ such that $\omega^n = 1$, $\sum_{k=0}^{n-1} (\omega^j)^k = 0$ for every integer $j \neq 0 \pmod{n}$, and $\|\omega u\| = \|u\|$ for all $u \in R$. (The last condition is not part of the standard definition, but it is natural in our setting where R carries a norm, and essential for error estimates.) We define an associated R -linear DFT map $F_\omega: R^n \rightarrow R^n$ by the formula

$$(F_\omega u)_j := \frac{1}{n} \sum_{k=0}^{n-1} \omega^{-jk} u_k, \quad u \in R^n, \quad 0 \leq j < n.$$

It is immediate that ω^{-1} ($= \omega^{n-1}$) is also a principal n -th root of unity in R , and that $\|F_\omega u\| \leq \|u\|$ for all $u \in R^n$.

Lemma 2.3 (Convolution formula). *For any $u, v \in R^n$ we have*

$$\frac{1}{n} u * v = n F_{\omega^{-1}}(F_\omega u \cdot F_\omega v).$$

Proof. For each j , the product $(F_\omega u)_j (F_\omega v)_j$ is equal to

$$\frac{1}{n^2} \sum_{s=0}^{n-1} \sum_{t=0}^{n-1} \omega^{-j(s+t)} u_s v_t = \frac{1}{n^2} \sum_{k=0}^{n-1} \omega^{-jk} \sum_{s+t=k \pmod{n}} u_s v_t = \frac{1}{n} F_\omega(u * v)_j,$$

so $F_\omega(u * v) = n F_\omega u \cdot F_\omega v$. On the other hand, for any $w \in R^n$ we have

$$(F_{\omega^{-1}}(F_\omega w))_j = \frac{1}{n^2} \sum_{s=0}^{n-1} \omega^{sj} \sum_{t=0}^{n-1} \omega^{-st} w_t = \frac{1}{n^2} \sum_{t=0}^{n-1} \left(\sum_{s=0}^{n-1} \omega^{s(j-t)} \right) w_t = \frac{1}{n} w_j,$$

so $F_{\omega^{-1}} F_\omega w = \frac{1}{n} w$. Taking $w := u * v$, we obtain the desired result. $\square$

For the two coefficient rings mentioned earlier, we choose ω as follows:

- (1) *Complex case.* For $R = \mathbb{C}$, let $n \geq 1$ be any positive integer, and put $\omega := e^{2\pi i/n}$. We denote F_ω in this case by $\mathcal{F}_n: \mathbb{C}^n \rightarrow \mathbb{C}^n$. Explicitly,

$$(\mathcal{F}_n u)_j = \frac{1}{n} \sum_{k=0}^{n-1} e^{-2\pi i j k/n} u_k, \quad u \in \mathbb{C}^n, \quad 0 \leq j < n.$$

We also write $\mathcal{F}_n^*: \mathbb{C}^n \rightarrow \mathbb{C}^n$ for $F_{\omega^{-1}}$.

- (2) *Synthetic case.* For $R = \mathcal{R} = \mathbb{C}[y]/(y^r + 1)$ where $r \geq 2$ is a power of two, let n be any positive divisor of $2r$. Then $\omega := y^{2r/n}$ is a principal n -th root of unity in $\mathcal{R}$. We denote F_ω in this case by $\mathcal{G}_n: \mathcal{R}^n \rightarrow \mathcal{R}^n$. Explicitly,

$$(\mathcal{G}_n u)_j = \frac{1}{n} \sum_{k=0}^{n-1} y^{-2r j k/n} u_k, \quad u \in \mathcal{R}^n, \quad 0 \leq j < n.$$

We also write $\mathcal{G}_n^*: \mathcal{R}^n \rightarrow \mathcal{R}^n$ for $F_{\omega^{-1}}$.

All of the concepts introduced above may be generalised to the multidimensional setting as follows. For $u, v \in \otimes_i R^{n_i}$, we define the pointwise product $u \cdot v \in \otimes_i R^{n_i}$ and the convolution product $u * v \in \otimes_i R^{n_i}$ by

$$\begin{aligned} (u \cdot v)_{j_1, \dots, j_d} &:= u_{j_1, \dots, j_d} v_{j_1, \dots, j_d}, \\ (u * v)_{j_1, \dots, j_d} &:= \sum_{k_1=0}^{n_1-1} \cdots \sum_{k_d=0}^{n_d-1} u_{k_1, \dots, k_d} v_{j_1-k_1, \dots, j_d-k_d}. \end{aligned}$$

Then $(\otimes_i R^{n_i}, \cdot)$ is isomorphic to the direct sum of $n_1 \cdots n_d$ copies of R , and $(\otimes_i R^{n_i}, *)$ is isomorphic to $R[x_1, \dots, x_d]/(x_1^{n_1} - 1, \dots, x_d^{n_d} - 1)$.

Let $\omega_1, \dots, \omega_d \in R$ be principal roots of unity of orders $n_1, \dots, n_d$. We define an associated R -linear d -dimensional DFT map by taking the tensor product (over R) of the corresponding one-dimensional DFTs, that is,

$$F_{\omega_1, \dots, \omega_d} := \otimes_i F_{\omega_i} : \otimes_i R^{n_i} \rightarrow \otimes_i R^{n_i}.$$

Explicitly, for $u \in \otimes_i R^{n_i}$ we have

$$(F_{\omega_1, \dots, \omega_d} u)_{j_1, \dots, j_d} = \frac{1}{n_1 \cdots n_d} \sum_{k_1=0}^{n_1-1} \cdots \sum_{k_d=0}^{n_d-1} \omega_1^{-j_1 k_1} \cdots \omega_d^{-j_d k_d} u_{k_1, \dots, k_d}.$$

The multidimensional analogue of Lemma 2.3 is

$$(2.1) \quad \frac{1}{n_1 \cdots n_d} u * v = n_1 \cdots n_d F_{\omega_1^{-1}, \dots, \omega_d^{-1}} (F_{\omega_1, \dots, \omega_d} u \cdot F_{\omega_1, \dots, \omega_d} v),$$

and is proved in exactly the same way.

In particular, in the “complex case” we obtain the d -dimensional transform

$$\mathcal{F}_{n_1, \dots, n_d} := \otimes_i \mathcal{F}_{n_i} : \otimes_i \mathbb{C}^{n_i} \rightarrow \otimes_i \mathbb{C}^{n_i}$$

(take $\omega_i := e^{2\pi i/n_i}$), and in the “synthetic case” the d -dimensional transform

$$\mathcal{G}_{n_1, \dots, n_d} := \otimes_i \mathcal{G}_{n_i} : \otimes_i \mathcal{R}^{n_i} \rightarrow \otimes_i \mathcal{R}^{n_i}$$

(where each n_i is a divisor of $2r$, and $\omega_i := y^{2r/n_i}$). We define similarly $\mathcal{F}_{n_1, \dots, n_d}^* := \otimes_i \mathcal{F}_{n_i}^*$ and $\mathcal{G}_{n_1, \dots, n_d}^* := \otimes_i \mathcal{G}_{n_i}^*$.

Any algorithm for computing $\mathcal{F}_n$ may easily be adapted to obtain an algorithm for computing $\mathcal{F}_n^*$, by adjusting signs appropriately. A similar remark applies to $\mathcal{G}_n$, and to the multidimensional generalisations of these maps. For the rest of the paper, we make use of these observations without further comment.

2.5. Fixed-point multiplication. We now consider the complexity of multiplication in the coefficient rings $R = \mathbb{C}$ and $R = \mathcal{R}$. In both cases we reduce the problem to integer multiplication. For the case $R = \mathcal{R}$ (Lemma 2.5) we will express the complexity in terms of $M(\cdot)$ itself, as this eventually feeds into the main recurrence inequality for $M(\cdot)$ that we prove in Section 5.3. For the case $R = \mathbb{C}$ (Lemma 2.4) we do not need the best possible bound; to simplify the subsequent complexity analysis, we prefer to use the crude estimate $M(p) = O(p^{1+\delta})$.

Lemma 2.4 (Multiplication in $\mathbb{C}$). *Given as input $u, v \in \tilde{\mathbb{C}}_\circ$, in time $O(p^{1+\delta})$ we may compute an approximation $\tilde{w} \in \tilde{\mathbb{C}}_\circ$ for $w := uv \in \mathbb{C}_\circ$ such that $\varepsilon(\tilde{w}) < 2$.*

Proof. We take $\tilde{w} := \rho(w)$, so that $\varepsilon(\tilde{w}) = 2^p \|\rho(w) - w\| < \sqrt{2} < 2$. Writing $u = 2^{-p}a$ and $v = 2^{-p}b$ where $a, b \in \mathbb{Z}[i]$ and $|a|, |b| \leq 2^p$, we have $\tilde{w} = 2^{-p}\rho_0(2^{-p}ab)$. Thus $\tilde{w}$ may be computed in time $O(p^{1+\delta})$ by multiplying out the real and imaginary parts of a and b , and then summing and rounding appropriately. $\square$

For the case $R = \mathcal{R}$, observe first that for any $u, v \in \mathcal{R}$ we have $\|uv\| \leq r \|u\| \|v\|$, as each coefficient of $uv = (u_0 + \dots + u_{r-1}y^{r-1})(v_0 + \dots + v_{r-1}y^{r-1}) \bmod y^r + 1$ is a sum of exactly r terms of the form $\pm u_i v_j$. In particular, if $u, v \in \mathcal{R}_\circ$, then $uv/r \in \mathcal{R}_\circ$.

Lemma 2.5 (Multiplication in $\mathcal{R}$). *Assume that $r \geq 2$ is a power of two and that $r < 2^{p-1}$. Given as input $u, v \in \tilde{\mathcal{R}}_\circ$, in time $4M(3rp) + O(rp)$ we may compute an approximation $\tilde{w} \in \tilde{\mathcal{R}}_\circ$ for $w := uv/r \in \mathcal{R}_\circ$ such that $\varepsilon(\tilde{w}) < 2$.*

Proof. Write $2^p u = U_0(y) + iU_1(y)$ and $2^p v = V_0(y) + iV_1(y)$ where U_j and V_j are polynomials in $\mathbb{Z}[y]$ of degree less than r and whose coefficients lie in the interval $[-2^p, 2^p]$. Then $2^{2p}rw = W_0(y) + iW_1(y)$ where

$$W_0 := (U_0V_0 - U_1V_1) \bmod y^r + 1, \quad W_1 := (U_0V_1 + U_1V_0) \bmod y^r + 1.$$

We use the following algorithm, which is based on the well-known Kronecker substitution technique [14, Corollary 8.27].

(1) *Pack coefficients.* Evaluate $U_j(2^{3p}), V_j(2^{3p}) \in \mathbb{Z}$ for $j = 0, 1$. As the input coefficients have at most p bits, this amounts to concatenating the coefficients with appropriate zero-padding (or one-padding in the case of negative coefficients), plus some carry and sign handling. The cost of this step is $O(rp)$.

(2) *Multiply in $\mathbb{Z}$.* Let $W_{j,k} := U_j V_k \in \mathbb{Z}[y]$ for $j, k \in \{0, 1\}$. Compute the four integer products $W_{j,k}(2^{3p}) = U_j(2^{3p})V_k(2^{3p})$. The cost of this step is $4M(3rp)$.

(3) *Unpack coefficients.* For each pair (j, k) , the coefficients of $W_{j,k} \in \mathbb{Z}[y]$ are bounded in absolute value by $r(2^p)^2 < 2^{3p-1}$, so $W_{j,k}$ may be recovered from the integer $W_{j,k}(2^{3p})$ in time $O(rp)$. (In more detail: the constant term of $W_{j,k}$ lies in the interval $(-2^{3p-1}, 2^{3p-1})$, so it is easily read off the last $3p$ bits of $W_{j,k}(2^{3p})$. After stripping off this term, one proceeds to the linear term, and so on.) We then deduce the polynomials $W_0 = (W_{0,0} - W_{1,1}) \bmod y^r + 1$ and $W_1 = (W_{0,1} + W_{1,0}) \bmod y^r + 1$ in time $O(rp)$.

(4) *Scale and round.* Let $c_\ell := (W_0)_\ell + i(W_1)_\ell \in \mathbb{Z}[i]$ for $\ell \in \{0, \dots, r-1\}$. Then $w = (2^{2p}r)^{-1}(c_0 + \dots + c_{r-1}y^{r-1})$, so, recalling that $\|w\| = \|uv\|/r \leq 1$, we have $|c_\ell| \leq 2^{2p}r$ for each ℓ . In time $O(rp)$ we may compute $\tilde{w} := \rho(w) = 2^{-p} \sum_{\ell=0}^{r-1} \rho_0((2^{2p}r)^{-1}c_\ell)y^\ell$ (each division by $2^{2p}r$ amounts to a bit shift). Since $\|w\| \leq 1$, we have $\tilde{w} \in \tilde{\mathcal{R}}_\circ$, and as usual, $\varepsilon(\tilde{w}) = 2^p \|\rho(w) - w\| < \sqrt{2} < 2$. $\square$

2.6. Linear and bilinear maps. Let $\mathcal{A}: V \rightarrow W$ be a $\mathbb{C}$ -linear map between finite-dimensional vector spaces V and W . We define the operator norm of $\mathcal{A}$ to be

$$\|\mathcal{A}\| := \sup_{v \in V_\circ} \|\mathcal{A}v\|.$$

Example 2.6. For the normalised DFT map F_ω defined in Section 2.4, we have $\|F_\omega\| \leq 1$. The same therefore holds for $\mathcal{F}_n, \mathcal{F}_n^*, \mathcal{G}_n, \mathcal{G}_n^*$, and for the multivariate generalisations of these maps. (In fact, all of these maps have norm exactly 1.)

Assume now that $\|\mathcal{A}\| \leq 1$. By a *numerical approximation* for $\mathcal{A}$ we mean a function $\tilde{\mathcal{A}}: \tilde{V}_o \rightarrow \tilde{W}_o$ that is computed by some algorithm, typically via fixed-point arithmetic. The error of the approximation is defined to be

$$\varepsilon(\tilde{\mathcal{A}}) := \max_{v \in \tilde{V}_o} 2^p \|\tilde{\mathcal{A}}v - \mathcal{A}v\|.$$

We write $C(\tilde{\mathcal{A}})$ for the time required to compute $\tilde{\mathcal{A}}v$ from v (taking the maximum over all possible inputs $v \in \tilde{V}_o$).

Lemma 2.7 (Error propagation). *Let $\mathcal{A}: V \rightarrow W$ be a $\mathbb{C}$ -linear map such that $\|\mathcal{A}\| \leq 1$, and let $v \in V_o$. Let $\tilde{\mathcal{A}}: \tilde{V}_o \rightarrow \tilde{W}_o$ be a numerical approximation for $\mathcal{A}$, and let $\tilde{v} \in \tilde{V}_o$ be an approximation for v . Then $\tilde{w} := \tilde{\mathcal{A}}\tilde{v} \in \tilde{W}_o$ is an approximation for $w := \mathcal{A}v \in W_o$ such that $\varepsilon(\tilde{w}) \leq \varepsilon(\tilde{\mathcal{A}}) + \varepsilon(\tilde{v})$.*

Proof. We have

$$\begin{aligned} \varepsilon(\tilde{w}) &= 2^p \|\tilde{\mathcal{A}}\tilde{v} - \mathcal{A}v\| \leq 2^p \|\tilde{\mathcal{A}}\tilde{v} - \mathcal{A}\tilde{v}\| + 2^p \|\mathcal{A}\tilde{v} - \mathcal{A}v\| \\ &\leq \varepsilon(\tilde{\mathcal{A}}) + 2^p \|\mathcal{A}\| \|\tilde{v} - v\| \leq \varepsilon(\tilde{\mathcal{A}}) + \varepsilon(\tilde{v}). \end{aligned} \quad \square$$

Lemma 2.7 yields the following estimate for compositions of linear maps.

Corollary 2.8 (Composition). *Let $\mathcal{A}: U \rightarrow V$ and $\mathcal{B}: V \rightarrow W$ be $\mathbb{C}$ -linear maps such that $\|\mathcal{A}\|, \|\mathcal{B}\| \leq 1$. Let $\tilde{\mathcal{A}}: \tilde{U}_o \rightarrow \tilde{V}_o$ and $\tilde{\mathcal{B}}: \tilde{V}_o \rightarrow \tilde{W}_o$ be numerical approximations. Then $\tilde{\mathcal{C}} := \tilde{\mathcal{B}}\tilde{\mathcal{A}}: \tilde{U}_o \rightarrow \tilde{W}_o$ is a numerical approximation for $\mathcal{C} := \mathcal{B}\mathcal{A}: U \rightarrow W$ such that $\varepsilon(\tilde{\mathcal{C}}) \leq \varepsilon(\tilde{\mathcal{B}}) + \varepsilon(\tilde{\mathcal{A}})$.*

Proof. For any $u \in \tilde{U}_o$, if we set $v := \mathcal{A}u \in V_o$ and $\tilde{v} := \tilde{\mathcal{A}}u \in \tilde{V}_o$, then

$$2^p \|\tilde{\mathcal{B}}\tilde{\mathcal{A}}u - \mathcal{B}\mathcal{A}u\| = 2^p \|\tilde{\mathcal{B}}\tilde{v} - \mathcal{B}v\| \leq \varepsilon(\tilde{\mathcal{B}}) + \varepsilon(\tilde{v}) \leq \varepsilon(\tilde{\mathcal{B}}) + \varepsilon(\tilde{\mathcal{A}}). \quad \square$$

The above definitions and results may be adapted to the case of a $\mathbb{C}$ -bilinear map $\mathcal{A}: U \times V \rightarrow W$ as follows. We define

$$\|\mathcal{A}\| := \sup_{u \in U_o, v \in V_o} \|\mathcal{A}(u, v)\|.$$

If $\|\mathcal{A}\| \leq 1$, then a *numerical approximation* for $\mathcal{A}$ is a function $\tilde{\mathcal{A}}: \tilde{U}_o \times \tilde{V}_o \rightarrow \tilde{W}_o$ that is computed by some algorithm. The error of the approximation is

$$\varepsilon(\tilde{\mathcal{A}}) := \max_{u \in \tilde{U}_o, v \in \tilde{V}_o} 2^p \|\tilde{\mathcal{A}}(u, v) - \mathcal{A}(u, v)\|,$$

and $C(\tilde{\mathcal{A}})$ denotes the time required to compute $\tilde{\mathcal{A}}(u, v)$ from u and v . Lemma 2.7 has the following analogue in the bilinear case.

Lemma 2.9 (Bilinear error propagation). *Let $\mathcal{A}: U \times V \rightarrow W$ be a $\mathbb{C}$ -bilinear map with $\|\mathcal{A}\| \leq 1$, and let $u \in U_o, v \in V_o$. Let $\tilde{\mathcal{A}}: \tilde{U}_o \times \tilde{V}_o \rightarrow \tilde{W}_o$, $\tilde{u} \in \tilde{U}_o$, $\tilde{v} \in \tilde{V}_o$ be approximations. Then $\tilde{w} := \tilde{\mathcal{A}}(\tilde{u}, \tilde{v}) \in \tilde{W}_o$ is an approximation for $w := \mathcal{A}(u, v) \in W_o$ such that $\varepsilon(\tilde{w}) \leq \varepsilon(\tilde{\mathcal{A}}) + \varepsilon(\tilde{u}) + \varepsilon(\tilde{v})$.*

Proof. We have

$$\begin{aligned} \varepsilon(\tilde{w}) &\leq 2^p (\|\tilde{\mathcal{A}}(\tilde{u}, \tilde{v}) - \mathcal{A}(\tilde{u}, \tilde{v})\| + \|\mathcal{A}(\tilde{u}, \tilde{v}) - \mathcal{A}(u, \tilde{v})\| + \|\mathcal{A}(u, \tilde{v}) - \mathcal{A}(u, v)\|) \\ &= 2^p (\|\tilde{\mathcal{A}}(\tilde{u}, \tilde{v}) - \mathcal{A}(\tilde{u}, \tilde{v})\| + \|\mathcal{A}(\tilde{u} - u, \tilde{v})\| + \|\mathcal{A}(u, \tilde{v} - v)\|) \\ &\leq \varepsilon(\tilde{\mathcal{A}}) + 2^p \|\mathcal{A}\| \|\tilde{u} - u\| \|\tilde{v}\| + 2^p \|\mathcal{A}\| \|u\| \|\tilde{v} - v\| \\ &\leq \varepsilon(\tilde{\mathcal{A}}) + \varepsilon(\tilde{u}) + \varepsilon(\tilde{v}). \end{aligned} \quad \square$$

The following application of Lemma 2.9 will frequently be useful.

Corollary 2.10. *Let $u, v \in \mathbb{C}_\circ$, and let $w := uv \in \mathbb{C}_\circ$. Given as input approximations $\tilde{u}, \tilde{v} \in \tilde{\mathbb{C}}_\circ$, in time $O(p^{1+\delta})$ we may compute an approximation $\tilde{w} \in \tilde{\mathbb{C}}_\circ$ such that $\varepsilon(\tilde{w}) < \varepsilon(\tilde{u}) + \varepsilon(\tilde{v}) + 2$.*

Proof. Define a bilinear map $\mathcal{A}: \mathbb{C} \times \mathbb{C} \rightarrow \mathbb{C}$ by $\mathcal{A}(u, v) := uv$. Then $\|\mathcal{A}\| \leq 1$, and Lemma 2.4 yields an approximation $\tilde{\mathcal{A}}: \tilde{\mathbb{C}}_\circ \times \tilde{\mathbb{C}}_\circ \rightarrow \tilde{\mathbb{C}}_\circ$ such that $\varepsilon(\tilde{\mathcal{A}}) < 2$ and $\mathcal{C}(\tilde{\mathcal{A}}) = O(p^{1+\delta})$. Applying Lemma 2.9 to $\mathcal{A}$ and $\tilde{\mathcal{A}}$ yields the desired result. $\square$

2.7. Tensor products. The following result shows how to construct numerical approximations for tensor products of linear maps over a coefficient ring.

Lemma 2.11 (Tensor products). *Let R be a coefficient ring of dimension $r \geq 1$. Let $d \geq 1$, let $m_1, \dots, m_d, n_1, \dots, n_d \geq 2$, and put $M := \prod_i \max(m_i, n_i)$. For $i \in \{1, \dots, d\}$, let $\mathcal{A}_i: R^{m_i} \rightarrow R^{n_i}$ be an R -linear map with $\|\mathcal{A}_i\| \leq 1$, and let $\tilde{\mathcal{A}}_i: \tilde{R}_\circ^{m_i} \rightarrow \tilde{R}_\circ^{n_i}$ be a numerical approximation. Let $\mathcal{A} := \otimes_i \mathcal{A}_i: \otimes_i R^{m_i} \rightarrow \otimes_i R^{n_i}$ (note that automatically $\|\mathcal{A}\| \leq 1$).*

Then we may construct a numerical approximation $\tilde{\mathcal{A}}: \otimes_i \tilde{R}_\circ^{m_i} \rightarrow \otimes_i \tilde{R}_\circ^{n_i}$ such that $\varepsilon(\tilde{\mathcal{A}}) \leq \sum_i \varepsilon(\tilde{\mathcal{A}}_i)$ and

$$\mathcal{C}(\tilde{\mathcal{A}}) \leq M \sum_i \frac{\mathcal{C}(\tilde{\mathcal{A}}_i)}{\max(m_i, n_i)} + O(Mrp \log M).$$

Proof. For $i \in \{0, 1, \dots, d\}$, let

$$U^i := R^{n_1} \otimes \dots \otimes R^{n_{i-1}} \otimes R^{m_i} \otimes R^{m_{i+1}} \otimes \dots \otimes R^{m_d}.$$

In particular, $U^0 = \otimes_i R^{m_i}$ and $U^d = \otimes_i R^{n_i}$. The map $\mathcal{A}: U^0 \rightarrow U^d$ admits a decomposition $\mathcal{A} = \mathcal{B}_d \cdots \mathcal{B}_1$ where $\mathcal{B}_i: U^{i-1} \rightarrow U^i$ is given by

$$\mathcal{B}_i := \mathcal{I}_{n_1} \otimes \dots \otimes \mathcal{I}_{n_{i-1}} \otimes \mathcal{A}_i \otimes \mathcal{I}_{m_{i+1}} \otimes \dots \otimes \mathcal{I}_{m_d}$$

(here $\mathcal{I}_k$ denotes the identity map on R^k). In other words, $\mathcal{B}_i$ acts by applying $\mathcal{A}_i$ separately on each i -slice. Explicitly, for any $u \in U^{i-1}$ we have $(\mathcal{B}_i u)_{j_1, \dots, j_d} = (\mathcal{A}_i v)_{j_i}$ where $v \in R^{m_i}$ is the vector defined by $v_k := u_{j_1, \dots, j_{i-1}, k, j_{i+1}, \dots, j_d}$. In particular, $\|v_k\| \leq 1$ whenever $\|u\| \leq 1$, whence $\|\mathcal{B}_i\| \leq 1$.

We may define an approximation $\tilde{\mathcal{B}}_i: \tilde{U}_\circ^{i-1} \rightarrow \tilde{U}_\circ^i$ by mimicking the above formula for $\mathcal{B}_i$; i.e., for $u \in \tilde{U}_\circ^{i-1}$ we define $(\tilde{\mathcal{B}}_i u)_{j_1, \dots, j_d} := (\tilde{\mathcal{A}}_i v)_{j_i}$, where $v \in \tilde{R}_\circ^{m_i}$ is given by $v_k := u_{j_1, \dots, j_{i-1}, k, j_{i+1}, \dots, j_d}$. We may evaluate $\tilde{\mathcal{B}}_i$ by first rearranging the data so that each i -slice is stored contiguously (see Section 2.3), then applying $\tilde{\mathcal{A}}_i$ to each i -slice, and finally rearranging the data back into the correct order. We then define $\tilde{\mathcal{A}} := \tilde{\mathcal{B}}_d \cdots \tilde{\mathcal{B}}_1$.

We clearly have $\varepsilon(\tilde{\mathcal{B}}_i) = \varepsilon(\tilde{\mathcal{A}}_i)$ for all i , so by Corollary 2.8 we obtain $\varepsilon(\tilde{\mathcal{A}}) \leq \sum_i \varepsilon(\tilde{\mathcal{B}}_i) = \sum_i \varepsilon(\tilde{\mathcal{A}}_i)$. The cost of the data rearrangement at stage i is

$$\begin{aligned} O(n_1 \cdots n_{i-1} n_i m_{i+1} \cdots m_d r p \log n_i) + O(n_1 \cdots n_{i-1} m_i m_{i+1} \cdots m_d r p \log m_i) \\ = O(Mrp (\log n_i + \log m_i)), \end{aligned}$$

so the total over all i is $O(Mrp \log M)$. The total cost of the invocations of $\tilde{\mathcal{A}}_i$ is

$$\sum_i n_1 \cdots n_{i-1} m_{i+1} \cdots m_d \mathcal{C}(\tilde{\mathcal{A}}_i) \leq \sum_i \frac{M}{\max(m_i, n_i)} \mathcal{C}(\tilde{\mathcal{A}}_i). \quad \square$$

2.8. Exponential functions. The next three results concern the approximation of real and complex exponentials. We use the following facts:

- We may compute an n -bit approximation for π , i.e., an integer u such that $|2^{-n}u - \pi| \leq 2^{-n}$, in time $O(n^{1+\delta})$. Similarly for $\log 2$.
- For z lying in a fixed bounded subset of $\mathbb{C}$, we may compute an n -bit approximation for e^z in time $O(n^{1+\delta})$. More precisely, for any constant $C > 0$, given integers x and y such that $|2^{-n}(x+iy)| \leq C$, we may compute integers u and v such that $|2^{-n}(u+iv) - \exp(2^{-n}(x+iy))| \leq 2^{-n}$ in time $O(n^{1+\delta})$.

In fact these tasks may be performed in time $O(M(n) \log n)$; see [3, Ch. 6–7] or [5, Ch. 4]).

Lemma 2.12 (Complex exponentials). *Let $k \geq 1$ and j be integers such that $0 \leq j < k$, and let $w := e^{2\pi i j/k} \in \mathbb{C}_\circ$. Given j and k as input, we may compute an approximation $\tilde{w} \in \tilde{\mathbb{C}}_\circ$ such that $\varepsilon(\tilde{w}) < 2$ in time $O(\max(p, \log k)^{1+\delta})$.*

Proof. Let $p' := p + 3$. We first compute a p' -bit approximation $\tilde{r}$ for $r := 2\pi j/k \in [0, 2\pi]$, i.e., so that $|\tilde{r} - r| \leq 2^{-p'}$, in time $O(\max(p, \log k)^{1+\delta})$. We then compute a p' -bit approximation $\tilde{u}$ for $u := \exp(-2 \cdot 2^{-p'} + \tilde{r}i) \in \mathbb{C}_\circ$, i.e., so that $|\tilde{u} - u| \leq 2^{-p'}$, in time $O(p^{1+\delta})$ (the $2 \cdot 2^{-p'}$ term ensures that $\tilde{u}$ lies within the unit circle). Let $\eta := (-2 \cdot 2^{-p'} + \tilde{r}i) - ri$; then $|\eta| \leq \sqrt{5} \cdot 2^{-p'}$ and $|u - w| = |\exp(-2 \cdot 2^{-p'} + \tilde{r}i) - \exp(ri)| = |\exp(\eta) - 1| \leq 3 \cdot 2^{-p'}$, so $|\tilde{u} - w| \leq 4 \cdot 2^{-p'}$. We finally round $\tilde{u}$ towards zero to obtain the desired approximation $\tilde{w} \in \tilde{\mathbb{C}}_\circ$ for w at the original precision p . This yields $\varepsilon(\tilde{w}) \leq 4 \cdot 2^{p-p'} + \sqrt{2} < 2$. $\square$

Lemma 2.13 (Real exponentials, negative case). *Let $k \geq 1$ and $j \geq 0$ be integers, and let $w := e^{-\pi j/k} \in \mathbb{C}_\circ$. Given j and k as input, we may compute an approximation $\tilde{w} \in \tilde{\mathbb{C}}_\circ$ such that $\varepsilon(\tilde{w}) < 2$ in time $O(\max(p, \log(j+1), \log k)^{1+\delta})$.*

Proof. We first check whether $j > kp$ in time $O(\max(\log p, \log(j+1), \log k)^{1+\delta})$. If so, then $e^{-\pi j/k} < e^{-\pi p} < 2^{-p}$, so we may simply take $\tilde{w} := 0$.

Otherwise, we may assume that $0 \leq j/k \leq p$. In this case, we first compute an integer $\tau \geq 0$ such that $\tau \leq \frac{\pi}{\log 2} j/k \leq \tau + 2$ in time $O(\max(\log p, \log k)^{1+\delta})$ (note that $\tau = O(p)$). Let $p' := p + 2$ and $z := \tau \log 2 - \pi j/k \in [-2 \log 2, 0]$. We next compute a p' -bit approximation $\tilde{z} \leq 0$ for z , i.e., so that $|\tilde{z} - z| \leq 2^{-p'}$, in time $O(\max(p, \log k)^{1+\delta})$. We then compute a p' -bit approximation $\tilde{u} \leq 1$ for $u := e^{\tilde{z}} \leq 1$, i.e., so that $|\tilde{u} - u| \leq 2^{-p'}$, in time $O(p^{1+\delta})$. Observe that $|e^{\tilde{z}} - e^z| \leq 2^{-p'}$, so $|\tilde{u} - e^z| \leq 2 \cdot 2^{-p'}$. We finally divide $\tilde{u}$ by 2^τ (a bit shift) and round the result towards zero to obtain the desired approximation $\tilde{w} \in \tilde{\mathbb{C}}_\circ$ for w at the original precision p . Since $|2^{-\tau} \tilde{u} - w| = |2^{-\tau} \tilde{u} - 2^{-\tau} e^z| \leq 2 \cdot 2^{-\tau-p'} \leq 2 \cdot 2^{-p'}$, we obtain $\varepsilon(\tilde{w}) \leq 2 \cdot 2^{p-p'} + 1 < 2$. $\square$

Lemma 2.14 (Real exponentials, positive case). *Let $k \geq 1$, $j \geq 0$ and $\sigma \geq 0$ be integers, and assume that $e^{\pi j/k} \leq 2^\sigma$ and $\sigma \leq 2p$. Let $w := 2^{-\sigma} e^{\pi j/k} \in \mathbb{C}_\circ$. Given j , k and σ as input, we may compute an approximation $\tilde{w} \in \tilde{\mathbb{C}}_\circ$ such that $\varepsilon(\tilde{w}) < 2$ in time $O(\max(p, \log k)^{1+\delta})$.*

Proof. The hypotheses automatically ensure that $j < kp$. We now proceed along similar lines to the proof of Lemma 2.13: we first compute an integer $\tau \geq 0$ near $\sigma - \frac{\pi}{\log 2} j/k$ (again with $\tau = O(p)$), and then at suitably increased precision we

approximate successively $z := (\tau - \sigma) \log 2 + \pi j/k$ and $e^z = 2^{\tau-\sigma} e^{\pi j/k}$, and finally divide by 2^τ and round towards zero to obtain an approximation for $2^{-\sigma} e^{\pi j/k}$ at the original precision p . We omit the details, which are similar to the proof of Lemma 2.13. $\square$

3. COMPLEX TRANSFORMS FOR POWER-OF-TWO SIZES

Let $p \geq 100$ be the working precision as defined in Section 2. The goal of this section is to construct an efficiently computable approximation for the d -dimensional complex transform $\mathcal{F}_{t_1, \dots, t_d}: \otimes_i \mathbb{C}^{t_i} \rightarrow \otimes_i \mathbb{C}^{t_i}$ (see Section 2.4) in the special case that the t_i are powers of two. The following theorem is proved at the end of the section.

Theorem 3.1 (Power-of-two complex transforms). *Let $d \geq 2$ and let $t_1, \dots, t_d$ be powers of two such that $t_d \geq \dots \geq t_1 \geq 2$. Let $T := t_1 \cdots t_d$ and assume that $T < 2^p$. Then we may construct a numerical approximation $\tilde{\mathcal{F}}_{t_1, \dots, t_d}: \otimes_i \tilde{\mathbb{C}}_o^{t_i} \rightarrow \otimes_i \tilde{\mathbb{C}}_o^{t_i}$ for $\mathcal{F}_{t_1, \dots, t_d}$ such that $\varepsilon(\tilde{\mathcal{F}}_{t_1, \dots, t_d}) < 8T \log_2 T$ and*

$$\mathcal{C}(\tilde{\mathcal{F}}_{t_1, \dots, t_d}) < \frac{4T}{t_d} \mathcal{M}(3t_d p) + O(Tp \log T + Tp^{1+\delta}).$$

Throughout this section we set

$$r := t_d, \quad \mathcal{R} := \mathbb{C}[y]/(y^r + 1).$$

The basic idea of the proof of Theorem 3.1 is to use Bluestein's method [2] to reduce the DFT to the problem of computing a $(d-1)$ -dimensional cyclic convolution of size $t_1 \times \dots \times t_{d-1}$ over $\mathcal{R}$, and then to perform that convolution by taking advantage of the synthetic roots of unity in $\mathcal{R}$. The $\mathcal{M}(\cdot)$ term in the complexity bound arises from the pointwise multiplications in $\mathcal{R}$. The $O(Tp \log T)$ term covers the cost of the synthetic FFTs over $\mathcal{R}$, and the $O(Tp^{1+\delta})$ term covers various auxiliary operations.

For the rest of this section, $\otimes_i \mathcal{R}^{t_i}$ always means $\otimes_{i=1}^{d-1} \mathcal{R}^{t_i}$ (with $d-1$ factors), and $\otimes_i \mathbb{C}^{t_i}$ always means $\otimes_{i=1}^d \mathbb{C}^{t_i}$ (with d factors). These are both vector spaces of dimension $T = t_1 \cdots t_d$ over $\mathbb{C}$.

3.1. Transforms and convolutions over $\mathcal{R}$. We begin with the one-dimensional case. Recall that we have defined a synthetic transform $\mathcal{G}_t: \mathcal{R}^t \rightarrow \mathcal{R}^t$ (see Section 2.4) for each positive divisor t of $2r$, i.e., for $t \in \{1, 2, 4, 8, \dots, 2r\}$.

Lemma 3.2 (FFT over $\mathcal{R}$). *For $t \in \{2, 4, 8, \dots, 2r\}$, we may construct a numerical approximation $\tilde{\mathcal{G}}_t: \tilde{\mathcal{R}}_o^t \rightarrow \tilde{\mathcal{R}}_o^t$ for $\mathcal{G}_t$ such that $\varepsilon(\tilde{\mathcal{G}}_t) \leq \log_2 t$ and $\mathcal{C}(\tilde{\mathcal{G}}_t) = O(trp \log t)$.*

Proof. First observe that $\mathcal{G}_1: \mathcal{R} \rightarrow \mathcal{R}$ is the identity map, and admits the trivial approximation $\tilde{\mathcal{G}}_1: \tilde{\mathcal{R}}_o \rightarrow \tilde{\mathcal{R}}_o$ given by $\tilde{\mathcal{G}}_1 u := u$. This satisfies $\varepsilon(\tilde{\mathcal{G}}_1) = 0$ and $\mathcal{C}(\tilde{\mathcal{G}}_1) = O(rp)$.

Now let $t \in \{2, 4, 8, \dots, 2r\}$, and assume that we have already constructed $\tilde{\mathcal{G}}_{t/2}: \tilde{\mathcal{R}}_o^{t/2} \rightarrow \tilde{\mathcal{R}}_o^{t/2}$ such that $\varepsilon(\tilde{\mathcal{G}}_{t/2}) \leq \log_2(t/2)$. Given as input $u \in \tilde{\mathcal{R}}_o^t$, we will use the well-known Cooley–Tukey algorithm [7] to approximate $\mathcal{G}_t u \in \mathcal{R}_o^t$.

For any $j \in \{0, \dots, t-1\}$, observe that

$$(\mathcal{G}_t u)_j = \frac{1}{t} \sum_{k=0}^{t-1} y^{-2rjk/t} u_k = \frac{1}{t} \sum_{k=0}^{\frac{t}{2}-1} y^{-2rjk/t} u_k + \frac{1}{t} \sum_{k=0}^{\frac{t}{2}-1} y^{-2rj(k+\frac{t}{2})/t} u_{k+\frac{t}{2}}$$

$$= \frac{1}{t/2} \sum_{k=0}^{\frac{t}{2}-1} y^{-2rjk/t} \frac{u_k + (-1)^j u_{k+\frac{t}{2}}}{2},$$

where we have used the fact that $y^r = -1$. For $\ell \in \{0, \dots, \frac{t}{2} - 1\}$ this implies that $(\mathcal{G}_t u)_{2\ell} = (\mathcal{G}_{t/2} v)_\ell$ and $(\mathcal{G}_t u)_{2\ell+1} = (\mathcal{G}_{t/2} w)_\ell$, where $v, w \in \mathcal{R}_o^{t/2}$ are given by

$$v_k := \frac{1}{2}(u_k + u_{k+\frac{t}{2}}), \quad w_k := \frac{1}{2}y^{-2rk/t}(u_k - u_{k+\frac{t}{2}}), \quad 0 \leq k < t/2.$$

We may therefore use the following algorithm.

(1) *Butterflies.* For $k \in \{0, \dots, \frac{t}{2} - 1\}$, we use Lemma 2.1 to compute approximations $\tilde{v}_k, \tilde{w}'_k \in \tilde{\mathcal{R}}_o$ for v_k and $w'_k := \frac{1}{2}(u_k - u_{k+\frac{t}{2}})$ such that $\varepsilon(\tilde{v}_k), \varepsilon(\tilde{w}'_k) < 1$. We then compute an approximation $\tilde{w}_k \in \tilde{\mathcal{R}}_o$ for $w_k = y^{-2rk/t}w'_k$; as $y^r = -1$, this amounts to cyclically permuting the coefficients of $\tilde{w}'_k$ (and adjusting signs), and clearly $\varepsilon(\tilde{w}_k) = \varepsilon(\tilde{w}'_k) < 1$. The cost of this step is $O(trp)$.

(2) *Recurse.* We compute $\tilde{\mathcal{G}}_{t/2}\tilde{v}$ and $\tilde{\mathcal{G}}_{t/2}\tilde{w}$ using the previously constructed map $\tilde{\mathcal{G}}_{t/2}$, and interleave the results (at a further cost of $O(trp)$) to obtain the output vector $\tilde{\mathcal{G}}_t u \in \tilde{\mathcal{R}}_o^t$ defined by $(\tilde{\mathcal{G}}_t u)_{2\ell} := (\tilde{\mathcal{G}}_{t/2}\tilde{v})_\ell$ and $(\tilde{\mathcal{G}}_t u)_{2\ell+1} := (\tilde{\mathcal{G}}_{t/2}\tilde{w})_\ell$ for $\ell \in \{0, \dots, \frac{t}{2} - 1\}$.

Recall from Example 2.6 that $\|\mathcal{G}_n\| \leq 1$ for all n . Applying Lemma 2.7 for $\mathcal{A} = \mathcal{G}_{t/2}$ and using the induction hypothesis, we obtain

$$\begin{aligned} 2^p \|(\tilde{\mathcal{G}}_t u)_{2\ell} - (\mathcal{G}_t u)_{2\ell}\| &= 2^p \|(\tilde{\mathcal{G}}_{t/2}\tilde{v})_\ell - (\mathcal{G}_{t/2}v)_\ell\| \\ &\leq \varepsilon(\tilde{\mathcal{G}}_{t/2}) + \varepsilon(\tilde{v}) \leq \log_2(t/2) + 1 = \log_2 t. \end{aligned}$$

A similar argument applies for $(\tilde{\mathcal{G}}_t u)_{2\ell+1}$. Therefore $\varepsilon(\tilde{\mathcal{G}}_t) \leq \log_2 t$.

As for the complexity, the above discussion shows that $\mathcal{C}(\tilde{\mathcal{G}}_t) < 2\mathcal{C}(\tilde{\mathcal{G}}_{t/2}) + O(trp)$. Together with the base case $\mathcal{C}(\tilde{\mathcal{G}}_1) = O(rp)$, this immediately yields the bound $\mathcal{C}(\tilde{\mathcal{G}}_t) = O(trp \log t)$ for $t \geq 2$. $\square$

Combining Lemmas 3.2 and 2.11, we obtain the following approximation for the multidimensional transform $\mathcal{G}_{t_1, \dots, t_{d-1}}: \otimes_i \mathcal{R}^{t_i} \rightarrow \otimes_i \mathcal{R}^{t_i}$ (defined in Section 2.4).

Proposition 3.3 (Multivariate FFT over $\mathcal{R}$). *Let $t_1, \dots, t_d$ and T be as in Theorem 3.1. We may construct a numerical approximation $\tilde{\mathcal{G}}_{t_1, \dots, t_{d-1}}: \otimes_i \tilde{\mathcal{R}}_o^{t_i} \rightarrow \otimes_i \tilde{\mathcal{R}}_o^{t_i}$ for $\mathcal{G}_{t_1, \dots, t_{d-1}}$ such that $\varepsilon(\tilde{\mathcal{G}}_{t_1, \dots, t_{d-1}}) < \log_2 T$ and $\mathcal{C}(\tilde{\mathcal{G}}_{t_1, \dots, t_{d-1}}) = O(Tp \log T)$.*

Proof. Since $t_i \geq 2$, Lemma 3.2 yields approximations $\tilde{\mathcal{G}}_{t_i}$ for $i = 1, \dots, d-1$. We apply Lemma 2.11 (with d replaced by $d-1$), taking $R := \mathcal{R}$, $m_i := t_i$, $n_i := t_i$ and $\mathcal{A}_i := \mathcal{G}_{t_i}$ for $i \in \{1, \dots, d-1\}$. The quantity M defined in Lemma 2.11 is given by $M = t_1 \cdots t_{d-1} = T/r$ (recall that $r = t_d$ throughout Section 3). We obtain

$$\varepsilon(\tilde{\mathcal{G}}_{t_1, \dots, t_{d-1}}) \leq \sum_{i=1}^{d-1} \varepsilon(\tilde{\mathcal{G}}_{t_i}) \leq \sum_{i=1}^{d-1} \log_2 t_i = \log_2(T/r) < \log_2 T$$

and

$$\begin{aligned} \mathcal{C}(\tilde{\mathcal{G}}_{t_1, \dots, t_{d-1}}) &\leq \frac{T}{r} \sum_{i=1}^{d-1} \frac{\mathcal{C}(\tilde{\mathcal{G}}_{t_i})}{t_i} + O\left(\frac{T}{r} rp \log \frac{T}{r}\right) \\ &\leq \frac{T}{r} \sum_{i=1}^{d-1} O(rp \log t_i) + O(Tp \log T) \end{aligned}$$

$$= O\left(\frac{T}{r}rp \log \frac{T}{r}\right) + O(Tp \log T) = O(Tp \log T). \quad \square$$

Next we will use the above result to approximate the normalised $(d-1)$ -dimensional convolution map $\mathcal{M}_{\mathcal{R}}: \otimes_i \mathcal{R}^{t_i} \times \otimes_i \mathcal{R}^{t_i} \rightarrow \otimes_i \mathcal{R}^{t_i}$ defined by

$$\mathcal{M}_{\mathcal{R}}(u, v) := \frac{1}{T}u * v, \quad u, v \in \otimes_i \mathcal{R}^{t_i},$$

where $*$ is the convolution operator defined in Section 2.4. Note that $\|\mathcal{M}_{\mathcal{R}}\| \leq 1$; indeed, each component of $u * v$ is a sum of $t_1 \cdots t_{d-1} = T/r$ terms of the form $u_{j_1, \dots, j_{d-1}} v_{k_1, \dots, k_{d-1}}$, and we saw just before Lemma 2.5 that $\|ab\| \leq r \|a\| \|b\|$ for all $a, b \in \mathcal{R}$.

Proposition 3.4 (Convolution over $\mathcal{R}$). *Let $t_1, \dots, t_d$ and T be as in Theorem 3.1. We may construct a numerical approximation $\tilde{\mathcal{M}}_{\mathcal{R}}: \otimes_i \tilde{\mathcal{R}}_{\circ}^{t_i} \times \otimes_i \tilde{\mathcal{R}}_{\circ}^{t_i} \rightarrow \otimes_i \tilde{\mathcal{R}}_{\circ}^{t_i}$ for $\mathcal{M}_{\mathcal{R}}$ such that $\varepsilon(\tilde{\mathcal{M}}_{\mathcal{R}}) < 3T \log_2 T + 2T + 3$ and*

$$\mathcal{C}(\tilde{\mathcal{M}}_{\mathcal{R}}) < \frac{4T}{r} \mathbf{M}(3rp) + O(Tp \log T + Tp^{1+\delta}).$$

Proof. We are given as input $u, v \in \otimes_i \tilde{\mathcal{R}}_{\circ}^{t_i}$. Let $w := \mathcal{M}_{\mathcal{R}}(u, v) = \frac{1}{T}u * v \in \otimes_i \mathcal{R}^{t_i}$ be the exact (normalised) convolution. According to (2.1) we have

$$rw = \frac{1}{t_1 \cdots t_{d-1}} u * v = (t_1 \cdots t_{d-1}) \mathcal{G}_{t_1, \dots, t_{d-1}}^* ((\mathcal{G}_{t_1, \dots, t_{d-1}} u) \cdot (\mathcal{G}_{t_1, \dots, t_{d-1}} v)).$$

Dividing both sides by $r = t_d$, we obtain $w = (T/r)w'$ where

$$w' := \mathcal{G}_{t_1, \dots, t_{d-1}}^* \left(\frac{1}{r} (\mathcal{G}_{t_1, \dots, t_{d-1}} u) \cdot (\mathcal{G}_{t_1, \dots, t_{d-1}} v) \right) \in \otimes_i \mathcal{R}_{\circ}^{t_i}.$$

We now use the following algorithm to approximate w .

(1) *Forward transforms.* We invoke Proposition 3.3 to compute approximations $\tilde{u}', \tilde{v}' \in \otimes_i \tilde{\mathcal{R}}_{\circ}^{t_i}$ for $u' := \mathcal{G}_{t_1, \dots, t_{d-1}} u \in \otimes_i \mathcal{R}_{\circ}^{t_i}$ and $v' := \mathcal{G}_{t_1, \dots, t_{d-1}} v \in \otimes_i \mathcal{R}_{\circ}^{t_i}$, with $\varepsilon(\tilde{u}'), \varepsilon(\tilde{v}') < \log_2 T$. The cost of this step (and step (3) below) is $O(Tp \log T)$.

(2) *Pointwise multiplications.* Let $\mathcal{A}: \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R}$ be the normalised multiplication map defined by $\mathcal{A}(a, b) := ab/r$; the bound $\|ab\| \leq r \|a\| \|b\|$ implies that $\|\mathcal{A}\| \leq 1$. Lemma 2.5 yields an approximation $\tilde{\mathcal{A}}: \tilde{\mathcal{R}}_{\circ} \times \tilde{\mathcal{R}}_{\circ} \rightarrow \tilde{\mathcal{R}}_{\circ}$ such that $\varepsilon(\tilde{\mathcal{A}}) < 2$ (note that $r = t_d \leq T/2 < 2^{p-1}$). Applying $\tilde{\mathcal{A}}$ to each component of $\tilde{u}'$ and $\tilde{v}'$, we obtain an approximation $\tilde{z} \in \otimes_i \tilde{\mathcal{R}}_{\circ}^{t_i}$ for $z := \frac{1}{r} u' \cdot v' \in \otimes_i \mathcal{R}_{\circ}^{t_i}$. This step requires time

$$\frac{T}{r} (4 \mathbf{M}(3rp) + O(rp)) = \frac{4T}{r} \mathbf{M}(3rp) + O(Tp),$$

and by Lemma 2.9 we have

$$\varepsilon(\tilde{z}) \leq \varepsilon(\tilde{\mathcal{A}}) + \varepsilon(\tilde{u}') + \varepsilon(\tilde{v}') < 2 + \log_2 T + \log_2 T = 2 \log_2 T + 2.$$

(3) *Inverse transform.* We use Proposition 3.3 again to compute an approximation $\tilde{w}' \in \otimes_i \tilde{\mathcal{R}}_{\circ}^{t_i}$ for $w' = \mathcal{G}_{t_1, \dots, t_{d-1}}^* z \in \otimes_i \mathcal{R}_{\circ}^{t_i}$. Recall from Example 2.6 that $\|\mathcal{G}_{t_1, \dots, t_{d-1}}^*\| \leq 1$. By Lemma 2.7, we obtain

$$\varepsilon(\tilde{w}') \leq \varepsilon(\tilde{\mathcal{G}}_{t_1, \dots, t_{d-1}}^*) + \varepsilon(\tilde{z}) < \log_2 T + (2 \log_2 T + 2) = 3 \log_2 T + 2.$$

(4) *Scaling.* Recall that $w = (T/r)w'$ and that $\|w\| \leq 1$. We may therefore apply Lemma 2.2 (with $c := T/r \leq 2^p$) to compute an approximation $\tilde{w} \in \otimes_i \tilde{\mathcal{R}}_{\circ}^{t_i}$ such that

$$\varepsilon(\tilde{w}) < 2(T/r)\varepsilon(\tilde{w}') + 3 \leq T(3 \log_2 T + 2) + 3$$

(here we have used the hypothesis that $r = t_d \geq 2$). The cost of this scaling step is $O(Tp^{1+\delta})$. Finally we take $\tilde{\mathcal{M}}_{\mathcal{R}}(u, v) := \tilde{w}$. $\square$

3.2. Transforms and convolutions over $\mathbb{C}$. We now transfer the results of the previous section from $\mathcal{R}$ to $\mathbb{C}$. Consider the normalised d -dimensional convolution map $\mathcal{M}_{\mathbb{C}}: \otimes_i \mathbb{C}^{t_i} \times \otimes_i \mathbb{C}^{t_i} \rightarrow \otimes_i \mathbb{C}^{t_i}$ defined by

$$\mathcal{M}_{\mathbb{C}}(u, v) := \frac{1}{T} u * v, \quad u, v \in \otimes_i \mathbb{C}^{t_i}.$$

As before we have $\|\mathcal{M}_{\mathbb{C}}\| \leq 1$.

Proposition 3.5 (Convolution over $\mathbb{C}$). *Let $t_1, \dots, t_d$ and T be as in Theorem 3.1. We may construct a numerical approximation $\tilde{\mathcal{M}}_{\mathbb{C}}: \otimes_i \tilde{\mathbb{C}}_{\circ}^{t_i} \times \otimes_i \tilde{\mathbb{C}}_{\circ}^{t_i} \rightarrow \otimes_i \tilde{\mathbb{C}}_{\circ}^{t_i}$ for $\mathcal{M}_{\mathbb{C}}$ such that $\varepsilon(\tilde{\mathcal{M}}_{\mathbb{C}}) < 3T \log_2 T + 2T + 15$ and*

$$\mathcal{C}(\tilde{\mathcal{M}}_{\mathbb{C}}) < \frac{4T}{r} \mathcal{M}(3rp) + O(Tp \log T + Tp^{1+\delta}).$$

Proof. Let $\zeta := e^{\pi i/r}$ and consider the $\mathbb{C}$ -linear map $\mathcal{S}: \mathbb{C}^r \rightarrow \mathcal{R}$ defined by

$$\mathcal{S}(w_0, \dots, w_{r-1}) := w_0 + \zeta w_1 y + \dots + \zeta^{r-1} w_{r-1} y^{r-1}.$$

Then $\mathcal{S}$ is a isomorphism of rings between $(\mathbb{C}^r, *)$ and $\mathcal{R} = \mathbb{C}[y]/(y^r + 1)$; in fact, recalling that $(\mathbb{C}^r, *) \cong \mathbb{C}[x]/(x^r - 1)$, we may regard $\mathcal{S}$ as the map sending x to ζy . Moreover, $\mathcal{S}$ induces an isomorphism of rings

$$\mathcal{T}: (\otimes_{i=1}^d \mathbb{C}^{t_i}, *) \rightarrow (\otimes_{i=1}^{d-1} \mathcal{R}^{t_i}, *).$$

Indeed, identifying these rings respectively as $\mathbb{C}[x_1, \dots, x_d]/(x_1^{t_1} - 1, \dots, x_d^{t_d} - 1)$ and $\mathbb{C}[x_1, \dots, x_{d-1}, y]/(x_1^{t_1} - 1, \dots, x_{d-1}^{t_{d-1}} - 1, y^r + 1)$, the isomorphism $\mathcal{T}$ sends $u(x_1, \dots, x_{d-1}, x_d)$ to $u(x_1, \dots, x_{d-1}, \zeta y)$. Writing $\mathcal{U} := \mathcal{T}^{-1}$ for the inverse isomorphism, we obtain

$$\mathcal{M}_{\mathbb{C}}(u, v) = \mathcal{U}(\mathcal{M}_{\mathcal{R}}(\mathcal{T}u, \mathcal{T}v)), \quad u, v \in \otimes_i \mathbb{C}^{t_i}.$$

Now we construct numerical approximations for the maps just introduced. We may construct an approximation $\tilde{\mathcal{S}}: \tilde{\mathbb{C}}_{\circ}^r \rightarrow \tilde{\mathcal{R}}_{\circ}$ by first using Lemma 2.12 to compute an approximation $\tilde{\zeta}_j \in \tilde{\mathbb{C}}_{\circ}$ for each $\zeta_j := \zeta^j = e^{\pi i j/r} \in \mathbb{C}_{\circ}$, and then using Corollary 2.10 to compute an approximation $\tilde{w}'_j \in \tilde{\mathbb{C}}_{\circ}$ for each product $w'_j := \zeta_j w_j \in \mathbb{C}_{\circ}$. We obtain $\varepsilon(\tilde{\zeta}_j) < 2$ and then $\varepsilon(\tilde{w}'_j) < \varepsilon(\tilde{\zeta}_j) + 2 < 4$, whence $\varepsilon(\tilde{\mathcal{S}}) < 4$. We also have $\mathcal{C}(\tilde{\mathcal{S}}) = O(r \max(p, \log r)^{1+\delta}) = O(rp^{1+\delta})$, since $\log r = O(p)$ (recall from the proof of Proposition 3.4 that $r < 2^{p-1}$). Then, applying $\tilde{\mathcal{S}}$ separately to the coefficient of each $x_1^{j_1} \dots x_{d-1}^{j_{d-1}}$, we obtain an approximation $\tilde{\mathcal{T}}: \otimes_{i=1}^d \tilde{\mathbb{C}}_{\circ}^{t_i} \rightarrow \otimes_{i=1}^{d-1} \tilde{\mathcal{R}}_{\circ}^{t_i}$ such that $\varepsilon(\tilde{\mathcal{T}}) < 4$ and $\mathcal{C}(\tilde{\mathcal{T}}) = O((T/r)rp^{1+\delta}) = O(Tp^{1+\delta})$. The inverse is handled similarly; we obtain an approximation $\tilde{\mathcal{U}}: \otimes_{i=1}^{d-1} \tilde{\mathcal{R}}_{\circ}^{t_i} \rightarrow \otimes_{i=1}^d \tilde{\mathbb{C}}_{\circ}^{t_i}$ such that $\varepsilon(\tilde{\mathcal{U}}) < 4$ and $\mathcal{C}(\tilde{\mathcal{U}}) = O(Tp^{1+\delta})$.

Finally, given as input $u, v \in \otimes_i \tilde{\mathbb{C}}_{\circ}^{t_i}$, we define $\tilde{\mathcal{M}}_{\mathbb{C}}(u, v) := \tilde{\mathcal{U}}(\tilde{\mathcal{M}}_{\mathcal{R}}(\tilde{\mathcal{T}}u, \tilde{\mathcal{T}}v))$. We already observed that $\|\mathcal{M}_{\mathcal{R}}\| \leq 1$, and clearly $\|\mathcal{T}\|, \|\mathcal{U}\| \leq 1$, so Lemma 2.7, Lemma 2.9 and Proposition 3.4 together imply that

$$\varepsilon(\tilde{\mathcal{M}}_{\mathbb{C}}) \leq \varepsilon(\tilde{\mathcal{U}}) + \varepsilon(\tilde{\mathcal{M}}_{\mathcal{R}}) + \varepsilon(\tilde{\mathcal{T}}) + \varepsilon(\tilde{\mathcal{T}}) < (3T \log_2 T + 2T + 3) + 4 + 4 + 4.$$

The estimate for $\mathcal{C}(\tilde{\mathcal{M}}_{\mathbb{C}})$ follows immediately from Proposition 3.4. $\square$

Finally we use Bluestein's trick [2] to prove the main result of this section.

Proof of Theorem 3.1. We are given as input $u \in \otimes_i \tilde{\mathbb{C}}_o^{t_i}$. We wish to approximate its transform $v := \mathcal{F}_{t_1, \dots, t_d} u \in \otimes_i \mathbb{C}_o^{t_i}$, which is given explicitly by

$$v_{j_1, \dots, j_d} = \frac{1}{T} \sum_{k_1=0}^{t_1-1} \dots \sum_{k_d=0}^{t_d-1} e^{-2\pi i(j_1 k_1/t_1 + \dots + j_d k_d/t_d)} u_{k_1, \dots, k_d}, \quad 0 \leq j_i < t_i.$$

For any $j_1, \dots, j_d \in \mathbb{Z}$ set

$$(3.1) \quad a_{j_1, \dots, j_d} := e^{\pi i(j_1^2/t_1 + \dots + j_d^2/t_d)} \in \mathbb{C}_o.$$

The identity $-2jk = (j-k)^2 - j^2 - k^2$ implies that

$$(3.2) \quad v_{j_1, \dots, j_d} = \bar{a}_{j_1, \dots, j_d} \frac{1}{T} \sum_{k_1=0}^{t_1-1} \dots \sum_{k_d=0}^{t_d-1} a_{j_1-k_1, \dots, j_d-k_d} (\bar{a}_{k_1, \dots, k_d} u_{k_1, \dots, k_d}),$$

where $\bar{\cdot}$ denotes complex conjugation. Moreover, we observe that $a_{j_1, \dots, j_d}$ is periodic in each j_i with period t_i , as $e^{\pi i(j_i+t_i)^2/t_i} = e^{\pi i j_i^2/t_i} (e^{\pi i})^{2j_i+t_i} = e^{\pi i j_i^2/t_i}$ (using the fact that t_i is even). Therefore, regarding (3.1) as defining a vector $a \in \otimes_i \mathbb{C}_o^{t_i}$, we may rewrite (3.2) in the form

$$v = \bar{a} \cdot \left(\frac{1}{T} a * (\bar{a} \cdot u) \right).$$

We now use the following algorithm.

(1) *Compute a .* Recalling that each t_i divides $r = t_d$, we may write

$$a_{j_1, \dots, j_d} = e^{2\pi i \eta_{j_1, \dots, j_d}/2r}, \quad \eta_{j_1, \dots, j_d} := \frac{r}{t_1} j_1^2 + \dots + \frac{r}{t_d} j_d^2 \pmod{2r}.$$

Iterating over the tuples $(j_1, \dots, j_d)$ in lexicographical order, we may compute $\eta_{j_1, \dots, j_d}$ in amortised time $O(\log r) = O(p)$ per tuple (for example by repeatedly using the identity $(j+1)^2 = j^2 + (2j+1)$, and the fact that each multiplication by r/t_i is a bit shift), and then use Lemma 2.12 to compute an approximation $\tilde{a}_{j_1, \dots, j_d} \in \tilde{\mathbb{C}}_o$ such that $\varepsilon(\tilde{a}_{j_1, \dots, j_d}) < 2$ in time $O(p^{1+\delta})$. We thus obtain $\tilde{a} \in \otimes_i \tilde{\mathbb{C}}_o^{t_i}$ with $\varepsilon(\tilde{a}) < 2$ in time $O(Tp^{1+\delta})$.

(2) *Pre-multiply.* We use Corollary 2.10 to compute an approximation $\tilde{b} \in \otimes_i \tilde{\mathbb{C}}_o^{t_i}$ for $b := \bar{a} \cdot u$ with $\varepsilon(\tilde{b}) < \varepsilon(\tilde{a}) + 2 < 4$ in time $O(Tp^{1+\delta})$.

(3) *Convolution.* We use Proposition 3.5 to compute an approximation $\tilde{c} \in \otimes_i \tilde{\mathbb{C}}_o^{t_i}$ for $c := \frac{1}{T} a * b$. This requires time $(4T/r)M(3rp) + O(Tp \log T + Tp^{1+\delta})$, and by Lemma 2.9 we have

$$\varepsilon(\tilde{c}) \leq \varepsilon(\tilde{\mathcal{M}}_{\mathbb{C}}) + \varepsilon(\tilde{a}) + \varepsilon(\tilde{b}) < 3T \log_2 T + 2T + 21.$$

(4) *Post-multiply.* We invoke Corollary 2.10 again to compute an approximation $\tilde{v} \in \otimes_i \tilde{\mathbb{C}}_o^{t_i}$ for $v = \bar{a} \cdot c$ such that

$$\varepsilon(\tilde{v}) \leq \varepsilon(\tilde{a}) + \varepsilon(\tilde{c}) + 2 < 3T \log_2 T + 2T + 25$$

in time $O(Tp^{1+\delta})$. We have $2T + 25 < 5T \log_2 T$ (because $T \geq t_1 t_2 \geq 4$), so $\varepsilon(\tilde{v}) < 8T \log_2 T$. Finally we take $\tilde{\mathcal{F}}_{t_1, \dots, t_d} u := \tilde{v}$. $\square$

Remark 3.6. In the algorithm developed in this section, it is essential that the multidimensional complex transform be performed “all at once”. If instead we decompose it into one-dimensional transforms in the usual way, and then use Bluestein’s method to convert each of these to a one-dimensional convolution, this would lead to an extraneous factor of $O(d)$ on the right hand side of (1.3).

Remark 3.7. An alternative method for reducing multidimensional complex transforms to synthetic transforms was described in [34]. Briefly, given as input $u \in \mathbb{C}[x_1, \dots, x_{d-1}, y]/(x_1^{t_1} - 1, \dots, x_{d-1}^{t_{d-1}} - 1, y^r + 1)$, assume that we wish to evaluate each x_i at the complex t_i -th roots of unity, and y at the “odd” complex $2r$ -th roots of unity (i.e., roots of $y^r + 1$). We first use $(d-1)$ -dimensional synthetic transforms to compute the polynomials $u_{i_1, \dots, i_{d-1}}(y) := u(y^{2ri_1/t_1}, \dots, y^{2ri_{d-1}/t_{d-1}}, y) \in \mathbb{C}[y]/(y^r + 1)$, for all $i_k \in \{0, \dots, t_k - 1\}$. It then suffices to compute the one-dimensional complex DFT of each $u_{i_1, \dots, i_{d-1}}(y)$, which could be done for example by Bluestein’s method. This alternative method has the same complexity (up to a constant factor) as the method presented in this section.

4. GAUSSIAN RESAMPLING

Let $p \geq 100$ be the working precision as defined in Section 2. The aim of this section is to show how to reduce the problem of approximating a multidimensional complex transform $\mathcal{F}_{s_1, \dots, s_d}: \otimes_i \mathbb{C}^{s_i} \rightarrow \otimes_i \mathbb{C}^{s_i}$ (defined in Section 2.4), for a given “source” size $s_1 \times \dots \times s_d$, to the problem of approximating another transform $\mathcal{F}_{t_1, \dots, t_d}: \otimes_i \mathbb{C}^{t_i} \rightarrow \otimes_i \mathbb{C}^{t_i}$ for a somewhat larger “target” size $t_1 \times \dots \times t_d$. (In Section 5 we will specialise to the case that the s_i are primes and the t_i are powers of two.) The following theorem is proved at the end of Section 4.3. It may be strengthened in various ways; see the discussion in Section 4.4.

Theorem 4.1 (Gaussian resampling). *Let $d \geq 1$, let $s_1, \dots, s_d$ and $t_1, \dots, t_d$ be integers such that $2 \leq s_i < t_i < 2^p$ and $\gcd(s_i, t_i) = 1$, and let $T := t_1 \dots t_d$. Let α be an integer in the interval $2 \leq \alpha < p^{1/2}$. For each i , let $\theta_i := t_i/s_i - 1$, and assume that $\theta_i \geq p/\alpha^4$.*

Then there exist linear maps $\mathcal{A}: \otimes_i \mathbb{C}^{s_i} \rightarrow \otimes_i \mathbb{C}^{t_i}$ and $\mathcal{B}: \otimes_i \mathbb{C}^{t_i} \rightarrow \otimes_i \mathbb{C}^{s_i}$, with $\|\mathcal{A}\|, \|\mathcal{B}\| \leq 1$, such that

$$\mathcal{F}_{s_1, \dots, s_d} = 2^\gamma \mathcal{B} \mathcal{F}_{t_1, \dots, t_d} \mathcal{A}, \quad \gamma := 2d\alpha^2.$$

Moreover, we may construct numerical approximations $\tilde{\mathcal{A}}: \otimes_i \tilde{\mathbb{C}}_o^{s_i} \rightarrow \otimes_i \tilde{\mathbb{C}}_o^{t_i}$ and $\tilde{\mathcal{B}}: \otimes_i \tilde{\mathbb{C}}_o^{t_i} \rightarrow \otimes_i \tilde{\mathbb{C}}_o^{s_i}$ such that $\varepsilon(\tilde{\mathcal{A}}), \varepsilon(\tilde{\mathcal{B}}) < dp^2$ and

$$\mathcal{C}(\tilde{\mathcal{A}}), \mathcal{C}(\tilde{\mathcal{B}}) = O(dTp^{3/2+\delta}\alpha + Tp \log T).$$

This theorem shows that to approximate a transform of size $s_1 \times \dots \times s_d$, one may first apply $\tilde{\mathcal{A}}$, then compute a transform of size $t_1 \times \dots \times t_d$, then apply $\tilde{\mathcal{B}}$, and finally multiply by 2^γ . The $dTp^{3/2+\delta}\alpha$ term in the complexity bound arises from the numerical computations at the heart of the “Gaussian resampling” method. The $Tp \log T$ term covers the cost of various data rearrangements in the Turing model (this term would not appear if we worked in the Boolean circuit model). In the application in Section 5, the parameters will be chosen so that the first term is negligible compared to the $O(Tp \log T)$ cost of evaluating $\tilde{\mathcal{F}}_{t_1, \dots, t_d}$.

Throughout this section we use the notation

$$[x] := \lfloor x + \tfrac{1}{2} \rfloor, \quad \langle x \rangle := x - [x], \quad x \in \mathbb{R}.$$

Thus $[x]$ is the nearest integer to x , rounding upwards in case of a tie, and $\langle x \rangle$ is the corresponding fractional part with $-\frac{1}{2} \leq \langle x \rangle < \frac{1}{2}$. For convenience of the reader, we provide in Table 2 a list of the linear maps appearing in this section and where they are defined.

$\mathcal{F}_n$	complex DFT of length n	(§2.4)
$\mathcal{F}_{n_1, \dots, n_d}$	complex multidimensional DFT of size $n_1 \times \dots \times n_d$	(§2.4)
$\mathcal{A}, \mathcal{B}$	the main maps we want to construct in Theorem 4.1	
$\mathcal{S}, \mathcal{T}$	resampling maps	(§4.1)
$\mathcal{P}_s, \mathcal{P}_t$	permutation maps	(§4.1)
$\mathcal{C}$	row-deleting map	(§4.2)
$\mathcal{T}'$	$\mathcal{CT}$ (a certain square submatrix of $\mathcal{T}$)	(§4.2)
$\mathcal{D}$	a diagonal map	(§4.2)
$\mathcal{N}$	normalised version of $\mathcal{T}$	(§4.2)
$\mathcal{E}$	$\mathcal{N} - \mathcal{I}$ (a map with small norm)	(§4.2)
$\mathcal{J}$	inverse of $\mathcal{N}$	(§4.3)
$\mathcal{S}', \mathcal{J}', \mathcal{D}'$	normalised versions of $\mathcal{S}, \mathcal{J}, \mathcal{D}$	(§4.3)

TABLE 2. Glossary of linear maps appearing in Section 4

4.1. The resampling identity. Throughout Sections 4.1 and 4.2, let s and $t > s$ be positive integers such that $\gcd(s, t) = 1$, and let $\alpha \in (0, \infty)$. Recall from Section 2.4 that u_j always means $u_{j \bmod s}$ if $u \in \mathbb{C}^s$, whereas u_k always means $u_{k \bmod t}$ if $u \in \mathbb{C}^t$.

Define “resampling maps” $\mathcal{S}: \mathbb{C}^s \rightarrow \mathbb{C}^t$ and $\mathcal{T}: \mathbb{C}^s \rightarrow \mathbb{C}^t$ by

$$(\mathcal{S}u)_k := \alpha^{-1} \sum_{j \in \mathbb{Z}} e^{-\pi \alpha^{-2} s^2 \left(\frac{k}{t} - \frac{j}{s}\right)^2} u_j, \quad u \in \mathbb{C}^s, \quad 0 \leq k < t,$$

$$(\mathcal{T}u)_k := \sum_{j \in \mathbb{Z}} e^{-\pi \alpha^2 t^2 \left(\frac{k}{t} - \frac{j}{s}\right)^2} u_j, \quad u \in \mathbb{C}^s, \quad 0 \leq k < t.$$

These sums certainly converge due to the rapid decay of the function e^{-x^2} . Each entry $(\mathcal{S}u)_k$ and $(\mathcal{T}u)_k$ is a weighted linear combination of $u_0, \dots, u_{s-1}$, with the largest weightings given to those u_j for which j/s is closest to k/t modulo 1. Figure 2 shows examples of the matrices of $\mathcal{S}$ and $\mathcal{T}$. They have relatively large entries near the “diagonal” of slope t/s , and the entries decay rapidly away from the diagonal according to a Gaussian law. The parameter α controls the rate of decay.

We also define permutation maps $\mathcal{P}_s: \mathbb{C}^s \rightarrow \mathbb{C}^s$ and $\mathcal{P}_t: \mathbb{C}^t \rightarrow \mathbb{C}^t$ by

$$(\mathcal{P}_s u)_j := u_{tj}, \quad u \in \mathbb{C}^s, \quad 0 \leq j < s,$$

$$(\mathcal{P}_t u)_k := u_{-sk}, \quad u \in \mathbb{C}^t, \quad 0 \leq k < t.$$

Then we have the following fundamental identity, which uses $\mathcal{S}$ and $\mathcal{T}$ to transform $\mathcal{F}_s$ into $\mathcal{F}_t$.

Theorem 4.2 (Resampling identity). *We have $\mathcal{T}\mathcal{P}_s\mathcal{F}_s = \mathcal{P}_t\mathcal{F}_t\mathcal{S}$. In other words, the following diagram commutes:*

$$\begin{array}{ccccc} \mathbb{C}^s & \xrightarrow{\mathcal{F}_s} & \mathbb{C}^s & \xrightarrow{\mathcal{P}_s} & \mathbb{C}^s \\ \mathcal{S} \downarrow & & & & \downarrow \mathcal{T} \\ \mathbb{C}^t & \xrightarrow{\mathcal{F}_t} & \mathbb{C}^t & \xrightarrow{\mathcal{P}_t} & \mathbb{C}^t \end{array}$$

0.5000	0.2280	0.0216	4.3e-4	1.7e-6	3.0e-9	1.7e-6	4.3e-4	0.0216	0.2280
0.3142	0.4795	0.1522	0.0100	1.4e-4	3.9e-7	9.0e-9	7.1e-6	0.0012	0.0428
0.0779	0.3982	0.4230	0.0934	0.0043	4.1e-5	8.1e-8	4.7e-8	2.7e-5	0.0032
0.0076	0.1305	0.4642	0.3432	0.0527	0.0017	1.1e-5	1.6e-8	2.3e-7	9.3e-5
2.9e-4	0.0169	0.2011	0.4977	0.2561	0.0274	6.1e-4	2.8e-6	3.5e-9	1.1e-6
4.5e-6	8.6e-4	0.0344	0.2849	0.4908	0.1757	0.0131	2.0e-4	6.5e-7	5.3e-9
2.7e-8	1.7e-5	0.0023	0.0644	0.3714	0.4452	0.1109	0.0057	6.2e-5	1.4e-7
2.7e-8	1.4e-7	6.2e-5	0.0057	0.1109	0.4452	0.3714	0.0644	0.0023	1.7e-5
4.5e-6	5.3e-9	6.5e-7	2.0e-4	0.0131	0.1757	0.4908	0.2849	0.0344	8.6e-4
2.9e-4	1.1e-6	3.5e-9	2.8e-6	6.1e-4	0.0274	0.2561	0.4977	0.2011	0.0169
0.0076	9.3e-5	2.3e-7	1.6e-8	1.1e-5	0.0017	0.0527	0.3432	0.4642	0.1305
0.0779	0.0032	2.7e-5	4.7e-8	8.1e-8	4.1e-5	0.0043	0.0934	0.4230	0.3982
0.3142	0.0428	0.0012	7.1e-6	9.0e-9	3.9e-7	1.4e-4	0.0100	0.1522	0.4795

1.0000	6.0e-10	1.3e-37	9.8e-84	2.7e-148	5.3e-231	2.7e-148	9.8e-84	1.3e-37	6.0e-10
3.5e-6	0.3227	1.1e-14	1.3e-46	5.4e-97	8.1e-166	1.6e-210	9.2e-132	1.9e-71	1.3e-29
1.5e-22	0.0021	0.0108	2.0e-20	1.3e-56	3.1e-111	2.6e-184	1.1e-190	3.3e-116	3.7e-60
7.6e-50	1.7e-16	0.1339	3.8e-5	3.9e-27	1.4e-67	1.8e-126	8.4e-204	7.1e-172	1.2e-101
4.8e-88	1.6e-40	2.0e-11	0.8819	1.4e-8	7.8e-35	1.6e-79	1.1e-142	2.9e-224	5.0e-154
3.7e-137	1.9e-75	3.7e-32	2.5e-7	0.6049	5.3e-13	1.6e-43	1.8e-92	7.2e-160	2.5e-217
3.4e-197	2.8e-121	8.2e-64	8.6e-25	3.2e-4	0.0432	2.1e-18	3.6e-53	2.2e-106	4.8e-178
3.4e-197	4.8e-178	2.2e-106	3.6e-53	2.1e-18	0.0432	3.2e-4	8.6e-25	8.2e-64	2.8e-121
3.7e-137	2.5e-217	7.2e-160	1.8e-92	1.6e-43	5.3e-13	0.6049	2.5e-7	3.7e-32	1.9e-75
4.8e-88	5.0e-154	2.9e-224	1.1e-142	1.6e-79	7.8e-35	1.4e-8	0.8819	2.0e-11	1.6e-40
7.6e-50	1.2e-101	7.1e-172	8.4e-204	1.8e-126	1.4e-67	3.9e-27	3.8e-5	0.1339	1.7e-16
1.5e-22	3.7e-60	3.3e-116	1.1e-190	2.6e-184	3.1e-111	1.3e-56	2.0e-20	0.0108	0.0021
3.5e-6	1.3e-29	1.9e-71	9.2e-132	1.6e-210	8.1e-166	5.4e-97	1.3e-46	1.1e-14	0.3227

FIGURE 2. Matrices of $\mathcal{S}$ and $\mathcal{T}$ for $s = 10$, $t = 13$, $\alpha = 2$. Maximal entries in each column are shown in bold. All entries are rounded to the number of significant figures shown.

Proof. Given $u \in \mathbb{C}^s$, define a smooth, 1-periodic function $f_u: \mathbb{R} \rightarrow \mathbb{C}$ by

$$f_u(x) := \sum_{m \in \mathbb{Z}} u_m g(x - \frac{m}{s}), \quad g(x) := e^{-\pi \alpha^{-2} s^2 x^2}.$$

It has an absolutely and uniformly convergent Fourier expansion

$$f_u(x) = \sum_{r \in \mathbb{Z}} \hat{f}_u(r) e^{2\pi i r x},$$

where the Fourier coefficients are given by

$$\begin{aligned} \hat{f}_u(r) &= \int_0^1 e^{-2\pi i r x} f_u(x) dx \\ &= \int_0^1 \sum_{m \in \mathbb{Z}} u_m e^{-2\pi i r x} g(x - \frac{m}{s}) dx \\ &= \int_0^1 \sum_{j=0}^{s-1} \sum_{q \in \mathbb{Z}} u_j e^{-2\pi i r x} g(x - q - \frac{j}{s}) dx \\ &= \sum_{j=0}^{s-1} u_j \int_{-\infty}^{\infty} e^{-2\pi i r x} g(x - \frac{j}{s}) dx \\ &= \sum_{j=0}^{s-1} u_j e^{-2\pi i r j / s} \int_{-\infty}^{\infty} e^{-2\pi i r x} g(x) dx. \end{aligned}$$

Using the well-known fact that the Fourier transform of $g(x)$ on $\mathbb{R}$ is given by

$$\int_{-\infty}^{\infty} e^{-2\pi i y x} g(x) dx = \alpha s^{-1} e^{-\pi \alpha^2 s^{-2} y^2}, \quad y \in \mathbb{R},$$

we obtain

$$\hat{f}_u(r) = \alpha e^{-\pi\alpha^2 s^{-2} r^2} (\mathcal{F}_s u)_r, \quad r \in \mathbb{Z}.$$

By definition $(\mathcal{S}u)_\ell = \alpha^{-1} f_u(\ell/t)$ for any $\ell \in \mathbb{Z}$, so for any $k \in \{0, \dots, t-1\}$ we have

$$\begin{aligned} (\mathcal{P}_t \mathcal{F}_t \mathcal{S}u)_k &= (\mathcal{F}_t \mathcal{S}u)_{-sk} \\ &= \alpha^{-1} t^{-1} \sum_{\ell=0}^{t-1} e^{2\pi i s k \ell / t} f_u(\ell/t) \\ &= \alpha^{-1} t^{-1} \sum_{\ell=0}^{t-1} e^{2\pi i s k \ell / t} \sum_{r \in \mathbb{Z}} \hat{f}_u(r) e^{2\pi i r \ell / t} \\ &= \alpha^{-1} \sum_{r = -sk \bmod t} \hat{f}_u(r) \\ &= \sum_{r = -sk \bmod t} e^{-\pi\alpha^2 s^{-2} r^2} (\mathcal{F}_s u)_r \\ &= \sum_{j \in \mathbb{Z}} e^{-\pi\alpha^2 s^{-2} (tj - sk)^2} (\mathcal{F}_s u)_{tj - sk} \\ &= \sum_{j \in \mathbb{Z}} e^{-\pi\alpha^2 t^2 \left(\frac{j}{s} - \frac{k}{t}\right)^2} (\mathcal{P}_s \mathcal{F}_s u)_j \\ &= (\mathcal{T} \mathcal{P}_s \mathcal{F}_s u)_k. \end{aligned} \quad \square$$

Remark 4.3. Another interpretation of the above proof is that the measure $f_u(x) dx$ is the convolution of the measures $\sum_{j=0}^{s-1} u_j \delta_{j/s}$ and $\sum_{j \in \mathbb{Z}} g(x - j) dx$ on $\mathbb{R}/\mathbb{Z}$, where δ_x means a unit mass concentrated at x . The key point is that the Fourier transform maps convolution of measures to pointwise multiplication of their Fourier coefficients.

Remark 4.4. If the hypothesis $\gcd(s, t) = 1$ is not satisfied, then the results of Sections 4.1–4.2 still hold as stated. However, in this situation Theorem 4.2 is of no use, as the map $\mathcal{P}_s$ fails to be invertible. For further discussion, see Section 4.4.1.

We conclude this section with a straightforward bound for $\|\mathcal{S}\|$.

Lemma 4.5. *We have $\|\mathcal{S}\| < 1 + \alpha^{-1}$.*

Proof. For any $u \in \mathbb{C}_o^s$ and $k \in \{0, \dots, t-1\}$, we have

$$|(\mathcal{S}u)_k| \leq \alpha^{-1} \sum_{j \in \mathbb{Z}} e^{-\pi\alpha^{-2} s^2 \left(\frac{k}{t} - \frac{j}{s}\right)^2} = \alpha^{-1} \sum_{j \in \mathbb{Z}} e^{-\pi\alpha^{-2} \left(j - \frac{sk}{t}\right)^2} = \alpha^{-1} \sum_{j \in \mathbb{Z}} G(\eta + j),$$

where $\eta := \langle -\frac{sk}{t} \rangle \in [-\frac{1}{2}, \frac{1}{2}]$ and $G(x) := e^{-\pi\alpha^{-2} x^2}$.

First suppose that $\eta \in [-\frac{1}{2}, 0]$. Then $G(x)$ is increasing on $(-\infty, \eta)$ and decreasing on $(\eta + 1, \infty)$, so

$$\int_{-\infty}^{\eta} G(x) dx > \sum_{j=-\infty}^{-1} G(\eta + j), \quad \int_{\eta+1}^{\infty} G(x) dx > \sum_{j=2}^{\infty} G(\eta + j).$$

For the remaining interval $(\eta, \eta + 1)$, we observe that $G(x) \geq G(\eta + 1)$ for $x \in (0, \eta + 1)$ and $G(x) \geq G(\eta)$ for $x \in (\eta, 0)$; but we have additionally $G(\eta) \geq G(\eta + 1)$

because $|\eta| \leq \frac{1}{2} \leq |\eta + 1|$, so in fact $G(x) \geq G(\eta + 1)$ on the whole interval $(\eta, \eta + 1)$. This implies that $\int_{\eta}^{\eta+1} G(x) dx \geq G(\eta + 1)$, and adding the three integrals yields

$$\sum_{j \in \mathbb{Z}} G(\eta + j) = G(\eta) + \sum_{j \neq 0} G(\eta + j) < 1 + \int_{-\infty}^{\infty} G(x) dx = 1 + \alpha.$$

A symmetrical argument yields the same bound for the case $\eta \in [0, \frac{1}{2}]$. We conclude that $|(\mathcal{S}u)_k| < \alpha^{-1}(1 + \alpha) = 1 + \alpha^{-1}$, and hence $\|\mathcal{S}\| < 1 + \alpha^{-1}$. $\square$

4.2. Solving the system. We wish to use Theorem 4.2 to express $\mathcal{F}_s$ in terms of $\mathcal{F}_t$. To do this, we must show how to solve a system of the form $\mathcal{T}x = y$. This system is overdetermined, as $t > s$. For fixed α , it turns out that the system is numerically unstable if t/s is too close to 1, or in other words, if the quantity $\theta := t/s - 1$ is too close to zero. On the other hand, we will show that imposing the condition $\theta \geq 1/\alpha^2$ is enough to ensure that the system becomes numerically tractable, and in this case we may even construct an explicit left inverse for $\mathcal{T}$.

We begin by reducing from a rectangular to a square system. Consider the function $\ell \mapsto [t\ell/s]$, which maps $\{0, \dots, s-1\}$ (injectively) into $\{0, \dots, t-1\}$. We use this to define a map $\mathcal{C}: \mathbb{C}^t \rightarrow \mathbb{C}^s$ by the formula

$$(\mathcal{C}u)_{\ell} := u_{[t\ell/s]}, \quad u \in \mathbb{C}^t, \quad 0 \leq \ell < s.$$

We then set

$$\mathcal{T}' := \mathcal{C}\mathcal{T}: \mathbb{C}^s \rightarrow \mathbb{C}^s.$$

Note that the matrix of $\mathcal{T}'$ is obtained by deleting $t - s$ rows from the matrix of $\mathcal{T}$. If we can show that $\mathcal{T}'$ is invertible, then a left inverse for $\mathcal{T}$ is given by $(\mathcal{T}')^{-1}\mathcal{C}$.

The entries of $\mathcal{T}'u$ are given explicitly by

$$\begin{aligned} (\mathcal{T}'u)_{\ell} &= (\mathcal{T}u)_{[t\ell/s]} = \sum_{j \in \mathbb{Z}} e^{-\pi\alpha^2 t^2 \left(\frac{1}{t} \left[\frac{t\ell}{s}\right] - \frac{j}{s}\right)^2} u_j = \sum_{j \in \mathbb{Z}} e^{-\pi\alpha^2 \left(\frac{tj}{s} - \left[\frac{t\ell}{s}\right]\right)^2} u_j \\ &= \sum_{h \in \mathbb{Z}} e^{-\pi\alpha^2 \left(\frac{th}{s} + \beta_{\ell}\right)^2} u_{\ell+h}, \end{aligned}$$

where

$$\beta_{\ell} := \frac{t\ell}{s} - \left[\frac{t\ell}{s}\right] = \left\langle \frac{t\ell}{s} \right\rangle, \quad \ell \in \mathbb{Z}.$$

Observe that β_{ℓ} is periodic in ℓ , i.e., $\beta_{\ell} = \beta_{\ell'}$ if $\ell \equiv \ell' \pmod{s}$, and that $|\beta_{\ell}| \leq \frac{1}{2}$ for all ℓ .

We normalise $\mathcal{T}'$ as follows. Let $\mathcal{D}: \mathbb{C}^s \rightarrow \mathbb{C}^s$ be the diagonal map defined by $(\mathcal{D}u)_{\ell} := d_{\ell}u_{\ell}$, where $d_{\ell} := e^{\pi\alpha^2 \beta_{\ell}^2}$ for $\ell \in \mathbb{Z}$. Since $\beta_{\ell}^2 \leq \frac{1}{4}$ we have $1 \leq d_{\ell} \leq e^{\pi\alpha^2/4}$, and in particular

$$(4.1) \quad \|\mathcal{D}\| \leq e^{\pi\alpha^2/4}.$$

Define

$$\mathcal{N} := \mathcal{T}'\mathcal{D}: \mathbb{C}^s \rightarrow \mathbb{C}^s.$$

In other words, the matrix of $\mathcal{N}$ is obtained by multiplying the ℓ -th column of the matrix of $\mathcal{T}'$ by d_{ℓ} . Explicitly,

$$(\mathcal{N}u)_{\ell} = \sum_{h \in \mathbb{Z}} e^{-\pi\alpha^2 \left(\frac{th}{s} + \beta_{\ell}\right)^2} d_{\ell+h} u_{\ell+h} = \sum_{h \in \mathbb{Z}} e^{-\pi\alpha^2 \left(\left(\frac{th}{s} + \beta_{\ell}\right)^2 - \beta_{\ell+h}^2\right)} u_{\ell+h}.$$

9.6e-923	1.9e-9	9.6e-37	1.1e-83	4.4e-148	1.2e-229	4.4e-148	1.1e-83	9.6e-37	1.9e-9
3.5e-6	1.8e-880	8.0e-14	1.4e-46	8.9e-97	1.9e-164	2.7e-210	1.0e-131	1.4e-70	4.2e-29
7.6e-50	5.2e-16	2.8e-866	4.3e-5	6.4e-27	3.2e-66	3.0e-126	9.6e-204	5.3e-171	3.8e-101
4.8e-88	5.1e-40	1.5e-10	7.4e-909	2.3e-8	1.8e-33	2.6e-79	1.3e-142	2.2e-223	1.5e-153
3.7e-137	6.0e-75	2.7e-31	2.8e-7	1.1e-894	1.2e-11	2.7e-43	2.1e-92	5.4e-159	7.6e-217
3.4e-197	1.5e-177	1.6e-105	4.1e-53	3.4e-18	4.3e-852	5.3e-4	9.7e-25	6.1e-63	8.6e-121
3.7e-137	7.6e-217	5.4e-159	2.1e-92	2.7e-43	1.2e-11	1.1e-894	2.8e-7	2.7e-31	6.0e-75
4.8e-88	1.5e-153	2.2e-223	1.3e-142	2.6e-79	1.8e-33	2.3e-8	7.4e-909	1.5e-10	5.1e-40
7.6e-50	3.8e-101	5.3e-171	9.6e-204	3.0e-126	3.2e-66	6.4e-27	4.3e-5	2.8e-866	5.2e-16
3.5e-6	4.2e-29	1.4e-70	1.0e-131	2.7e-210	1.9e-164	8.9e-97	1.4e-46	8.0e-14	1.8e-880

FIGURE 3. Matrix of $\mathcal{E}$ for $s = 10$, $t = 13$, $\alpha = 2$.

In this last expression, the $h = 0$ term is simply u_ℓ . Therefore, setting $\mathcal{E} := \mathcal{N} - \mathcal{I}$, where $\mathcal{I}: \mathbb{C}^s \rightarrow \mathbb{C}^s$ is the identity map, we have

$$(\mathcal{E}u)_\ell = \sum_{h \in \mathbb{Z} \setminus \{0\}} e^{-\pi\alpha^2 \left(\left(\frac{th}{s} + \beta_\ell \right)^2 - \beta_{\ell+h}^2 \right)} u_{\ell+h}, \quad u \in \mathbb{C}^s, \quad 0 \leq \ell < s.$$

An example of the matrix of $\mathcal{E}$ is shown in Figure 3.

The following estimate is crucial for establishing left-invertibility of $\mathcal{T}$ and for obtaining a fast algorithm for solving the system $\mathcal{T}x = y$.

Lemma 4.6. *Assume that $\alpha^2\theta \geq 1$. Then*

$$\|\mathcal{E}\| < 2.01 \cdot e^{-\pi\alpha^2\theta/2} < 2^{-\alpha^2\theta}.$$

Proof. For any $u \in \mathbb{C}_o^s$, the above formula for $(\mathcal{E}u)_\ell$ implies that

$$|(\mathcal{E}u)_\ell| \leq \sum_{h \in \mathbb{Z} \setminus \{0\}} e^{-\pi\alpha^2 \left(\left(\frac{th}{s} + \beta_\ell \right)^2 - \beta_{\ell+h}^2 \right)}, \quad 0 \leq \ell < s.$$

Since $|\beta_\ell| \leq \frac{1}{2} < \frac{t}{2s}$, we have $|\frac{th}{s} + \beta_\ell| \geq |\frac{th}{s}| - |\beta_\ell| > \frac{t}{s}(|h| - \frac{1}{2})$. For $h \neq 0$ we have $|h| - \frac{1}{2} \geq \frac{1}{2} > 0$, so

$$\begin{aligned} \left(\frac{th}{s} + \beta_\ell \right)^2 - \beta_{\ell+h}^2 &> (t/s)^2(|h| - \frac{1}{2})^2 - \frac{1}{4} \\ &= (1 + \theta)^2(|h| - \frac{1}{2})^2 - \frac{1}{4} \\ &\geq (1 + 2\theta)(|h| - \frac{1}{2})^2 - \frac{1}{4} \\ &= (|h| - \frac{1}{2})^2 - \frac{1}{4} + 2\theta(|h| - \frac{1}{2})^2 \\ &\geq 2\theta(|h| - \frac{1}{2})^2. \end{aligned}$$

Therefore

$$|(\mathcal{E}u)_\ell| < \sum_{h \in \mathbb{Z} \setminus \{0\}} e^{-2\pi\alpha^2\theta(|h| - \frac{1}{2})^2} = 2(w^{1/4} + w^{9/4} + w^{25/4} + \dots)$$

where $w := e^{-2\pi\alpha^2\theta}$. Since $\alpha^2\theta \geq 1$ we have $w \leq e^{-2\pi} < 0.002$, so

$$|(\mathcal{E}u)_\ell| < 2w^{1/4}(1 + w^2 + w^6 + \dots) < 2.01 \cdot w^{1/4} = 2.01 \cdot e^{-\pi\alpha^2\theta/2} < 2^{-\alpha^2\theta},$$

where we have used the fact that $2.01 \cdot e^{-\pi x/2} < 2^{-x}$ for all $x \geq 1$. $\square$

Under the hypothesis of Lemma 4.6, we see that $\|\mathcal{E}\| < \frac{1}{2}$, so $\mathcal{N} = \mathcal{I} + \mathcal{E}$ is invertible, with inverse given by $\mathcal{N}^{-1} = \mathcal{I} - \mathcal{E} + \mathcal{E}^2 - \dots$. Moreover, $\mathcal{D}\mathcal{N}^{-1}\mathcal{C}$ is the promised left inverse for $\mathcal{T}$, as

$$(4.2) \quad (\mathcal{D}\mathcal{N}^{-1}\mathcal{C})\mathcal{T} = \mathcal{D}\mathcal{N}^{-1}\mathcal{T}' = \mathcal{D}(\mathcal{T}'\mathcal{D})^{-1}\mathcal{T}' = \mathcal{I}.$$

4.3. Proof of the theorem. We may now prove the following special case of Theorem 4.1.

Proposition 4.7 (Gaussian resampling in one dimension). *Let s and t be integers such that $2 \leq s < t < 2^p$ and $\gcd(s, t) = 1$. Let α be an integer in the interval $2 \leq \alpha < p^{1/2}$. Let $\theta := t/s - 1 > 0$, and assume that $\theta \geq p/\alpha^4$. Then:*

- (i) *There exist linear maps $\mathcal{A}: \mathbb{C}^s \rightarrow \mathbb{C}^t$ and $\mathcal{B}: \mathbb{C}^t \rightarrow \mathbb{C}^s$ with $\|\mathcal{A}\|, \|\mathcal{B}\| \leq 1$ such that $\mathcal{F}_s = 2^{2\alpha^2} \mathcal{B} \mathcal{F}_t \mathcal{A}$.*
- (ii) *We may construct numerical approximations $\tilde{\mathcal{A}}: \tilde{\mathbb{C}}_0^s \rightarrow \tilde{\mathbb{C}}_0^t$ and $\tilde{\mathcal{B}}: \tilde{\mathbb{C}}_0^t \rightarrow \tilde{\mathbb{C}}_0^s$ such that $\varepsilon(\tilde{\mathcal{A}}), \varepsilon(\tilde{\mathcal{B}}) < p^2$ and $\mathcal{C}(\tilde{\mathcal{A}}), \mathcal{C}(\tilde{\mathcal{B}}) = O(tp^{3/2+\delta}\alpha + tp \log t)$.*

Proof of (i). We apply the results of Sections 4.1–4.2 with the given s , t and α . Lemma 4.5 implies that $\|\mathcal{S}\| < \frac{3}{2}$. The hypotheses $\alpha < p^{1/2}$ and $\theta \geq p/\alpha^4$ imply that $\alpha^2\theta \geq 1$, so Lemma 4.6 yields $\|\mathcal{E}\| < 2.01 \cdot e^{-\pi/2} < 0.42$. In particular, $\mathcal{N} = \mathcal{I} + \mathcal{E}$ is invertible and $\|\mathcal{N}^{-1}\| < 1 + 0.42 + (0.42)^2 + \dots < \frac{7}{4}$.

Let $\mathcal{J} := \mathcal{N}^{-1}$, and define normalised maps

$$\mathcal{S}' := \mathcal{S}/2, \quad \mathcal{J}' := \mathcal{J}/2, \quad \mathcal{D}' := \mathcal{D}/2^{2\alpha^2-2}.$$

Then $\|\mathcal{S}'\| < \frac{3}{4} < 1$ and $\|\mathcal{J}'\| < \frac{7}{8} < 1$. By (4.1) we have $\|\mathcal{D}\| \leq e^{\pi\alpha^2/4} < 2^{1.14\alpha^2} < 2^{2\alpha^2-2}$, as $1.14x < 2x - 2$ for all $x \geq 4$; hence also $\|\mathcal{D}'\| < 1$.

Now define

$$\mathcal{A} := \mathcal{S}', \quad \mathcal{B} := \mathcal{P}_s^{-1} \mathcal{D}' \mathcal{J}' \mathcal{C} \mathcal{P}_t,$$

where $\mathcal{P}_s$ and $\mathcal{P}_t$ are as in Theorem 4.2, and $\mathcal{C}$ is as in Section 4.2. Note that $\mathcal{P}_s$ is invertible thanks to the hypothesis $\gcd(s, t) = 1$. It is clear that $\|\mathcal{P}_t\| = \|\mathcal{P}_s^{-1}\| = \|\mathcal{C}\| = 1$, so $\|\mathcal{A}\| < 1$ and $\|\mathcal{B}\| < 1$. Moreover, by (4.2) and Theorem 4.2 we have

$$\begin{aligned} 2^{2\alpha^2} \mathcal{B} \mathcal{F}_t \mathcal{A} &= \mathcal{P}_s^{-1} (2^{2\alpha^2-2} \mathcal{D}') (2 \mathcal{J}') \mathcal{C} \mathcal{P}_t \mathcal{F}_t (2 \mathcal{S}') \\ &= \mathcal{P}_s^{-1} (\mathcal{D} \mathcal{N}^{-1} \mathcal{C}) (\mathcal{P}_t \mathcal{F}_t \mathcal{S}) \\ &= \mathcal{P}_s^{-1} (\mathcal{D} \mathcal{N}^{-1} \mathcal{C}) (\mathcal{T} \mathcal{P}_s \mathcal{F}_s) = \mathcal{P}_s^{-1} \mathcal{P}_s \mathcal{F}_s = \mathcal{F}_s. \end{aligned} \quad \square$$

We break up the proof of (ii) into several lemmas. We begin with a straightforward algorithm for approximating $\mathcal{D}'$ (Lemma 4.8). Next we give algorithms for approximating $\mathcal{S}' = \mathcal{S}/2$ and $\mathcal{E}$ (Lemmas 4.9 and 4.11); these amount to merely evaluating sufficiently many terms of the defining series, which converge quickly thanks to the rapid decay of the Gaussian weights. We then give an algorithm for approximating $\mathcal{J}' = \mathcal{N}^{-1}/2$, using the series $\mathcal{N}^{-1} = \mathcal{I} - \mathcal{E} + \mathcal{E}^2 - \dots$ (Lemma 4.12); here the fast convergence is guaranteed by the bound on $\|\mathcal{E}\|$ given in Lemma 4.6.

Lemma 4.8. *Assume the hypotheses of Proposition 4.7. We may construct a numerical approximation $\tilde{\mathcal{D}}': \tilde{\mathbb{C}}_0^s \rightarrow \tilde{\mathbb{C}}_0^t$ for $\mathcal{D}'$ such that $\varepsilon(\tilde{\mathcal{D}}') < 4$ and $\mathcal{C}(\tilde{\mathcal{D}}') = O(tp^{1+\delta})$.*

Proof. We are given as input $u \in \tilde{\mathbb{C}}_0^s$. For each $\ell \in \{0, \dots, s-1\}$, by definition $(\mathcal{D}'u)_\ell = d'_\ell u_\ell$ where

$$d'_\ell := d_\ell / 2^{2\alpha^2-2} = e^{\pi\alpha^2\beta_\ell^2} / 2^{2\alpha^2-2} < 1$$

(the last inequality follows from the estimate $\|\mathcal{D}'\| < 1$ in the proof of part (i) of Proposition 4.7). We may rewrite the rational part of the exponent of d'_ℓ as $\alpha^2\beta_\ell^2 = \alpha^2 \langle \frac{t\ell}{s} \rangle^2 = \alpha^2 k_\ell / s^2$ for some non-negative integer $k_\ell \leq s^2/4$. As α , s , t and ℓ are all integers with $O(p)$ bits (here we have used the hypotheses $s, t < 2^p$

and $\alpha < p^{1/2}$), we may compute $\alpha^2 k_\ell$ and s^2 in $O(p^{1+\delta})$ bit operations. Feeding this as input to Lemma 2.14 (with $\sigma := 2\alpha^2 - 2 < 2p$), we obtain an approximation $\tilde{d}'_\ell \in \tilde{\mathbb{C}}_\circ$ such that $\varepsilon(\tilde{d}'_\ell) < 2$ in time $O(p^{1+\delta})$. We then use Corollary 2.10 to compute an approximation $\tilde{z}_\ell \in \tilde{\mathbb{C}}_\circ$ for $z_\ell := d'_\ell u_\ell$ such that $\varepsilon(\tilde{z}_\ell) < \varepsilon(\tilde{d}'_\ell) + 2 < 4$ in time $O(p^{1+\delta})$. Finally we set $(\tilde{D}'u)_\ell := \tilde{z}_\ell$. The total cost over all ℓ is $O(sp^{1+\delta}) = O(tp^{1+\delta})$. $\square$

Lemma 4.9. *Assume the hypotheses of Proposition 4.7. We may construct a numerical approximation $\tilde{S}': \tilde{\mathbb{C}}_\circ^s \rightarrow \tilde{\mathbb{C}}_\circ^t$ for S' such that $\varepsilon(\tilde{S}') < 16p$ and $\mathcal{C}(\tilde{S}') = O(tp^{3/2+\delta}\alpha)$.*

Proof. We are given as input $u \in \tilde{\mathbb{C}}_\circ^s$. For each $k = 0, \dots, t-1$ in turn, we approximate $(S'u)_k$ as follows. By definition

$$(S'u)_k = (\tfrac{1}{2}Su)_k = \sum_{j \in \mathbb{Z}} \frac{1}{2} \alpha^{-1} e^{-\pi \alpha^{-2} (j - \frac{sk}{t})^2} u_j.$$

Let $m := \lceil p^{1/2} \rceil \alpha$ and consider the truncated sum

$$(4.3) \quad T_k := \sum_{\left| j - \frac{sk}{t} \right| < m} \frac{1}{2} \alpha^{-1} e^{-\pi \alpha^{-2} (j - \frac{sk}{t})^2} u_j.$$

Since $\|u\| \leq 1$ and $\alpha \geq 2$ we have

$$|(S'u)_k - T_k| \leq \sum_{\left| j - \frac{sk}{t} \right| \geq m} \frac{1}{2} \alpha^{-1} e^{-\pi \alpha^{-2} (j - \frac{sk}{t})^2} |u_j| \leq \frac{1}{4} \sum_{\left| j - \frac{sk}{t} \right| \geq m} e^{-\pi \alpha^{-2} (j - \frac{sk}{t})^2}.$$

Let $w := e^{-\pi \alpha^{-2}} < 1$; then

$$\begin{aligned} |(S'u)_k - T_k| &\leq \frac{1}{2} (w^{m^2} + w^{(m+1)^2} + w^{(m+2)^2} + \dots) \\ &= \frac{1}{2} w^{m^2} (1 + w^{2m+1} + w^{4m+4} + \dots). \end{aligned}$$

Since $\alpha < p^{1/2}$ we have

$$w^m = e^{-\pi \lceil p^{1/2} \rceil / \alpha} \leq e^{-\pi p^{1/2} / \alpha} < e^{-\pi} < 0.05,$$

so certainly $1 + w^{2m+1} + w^{4m+4} + \dots < 2$. We conclude that

$$|(S'u)_k - T_k| < w^{m^2} \leq e^{-\pi \lceil p^{1/2} \rceil^2} \leq e^{-\pi p} < 2^{-p}.$$

Now we explain how to compute a suitable fixed-point approximation for T_k . There are at most $2m$ terms in the sum (4.3). Let $\beta := \frac{1}{2}\alpha^{-1}$, and for each j appearing in (4.3), let $x_j := e^{-\pi \alpha^{-2} (j - sk/t)^2}$, $y_j := \beta x_j$, $z_j := y_j u_j$, so that $T_k = \sum_j z_j$. We first compute $\tilde{\beta} := \rho(\beta) = 2^{-p} \rho_0(2^{p-1}/\alpha) \in \tilde{\mathbb{C}}_\circ$ in time $O(p^{1+\delta})$; clearly $\varepsilon(\tilde{\beta}) = 2^p |\rho(\beta) - \beta| < 1$ (as β is real). Then for each j we perform the following steps. As s, t, j, k and α are all integers with $O(p)$ bits, the same holds for the numerator and the denominator of the rational number $\alpha^{-2} (j - sk/t)^2$, so we may use Lemma 2.13 to compute an approximation $\tilde{x}_j \in \tilde{\mathbb{C}}_\circ$ with $\varepsilon(\tilde{x}_j) < 2$ in time $O(p^{1+\delta})$. We then use Corollary 2.10 to compute an approximation $\tilde{y}_j \in \tilde{\mathbb{C}}_\circ$ such that $\varepsilon(\tilde{y}_j) < \varepsilon(\tilde{\beta}) + \varepsilon(\tilde{x}_j) + 2 < 5$, and again to obtain $\tilde{z}_j \in \tilde{\mathbb{C}}_\circ$ such that $\varepsilon(\tilde{z}_j) < \varepsilon(\tilde{y}_j) + 2 < 7$, in time $O(p^{1+\delta})$. Finally, we form the sum $\tilde{T}_k := \sum_j \tilde{z}_j$; that is, writing $\tilde{z}_j = 2^{-p} a_j$ for integers a_j , we compute $\sum_j a_j$ and set $\tilde{T}_k := 2^{-p} \sum_j a_j$.

Defining $(\tilde{\mathcal{S}}'u)_k := \tilde{T}_k$, we must check that $|\tilde{T}_k| \leq 1$ (so that $\tilde{T}_k \in \tilde{\mathbb{C}}_o$) and that $2^p |\tilde{T}_k - (\mathcal{S}'u)_k| < 16p$. For the latter, observe that

$$\begin{aligned} 2^p |\tilde{T}_k - (\mathcal{S}'u)_k| &\leq 2^p |\tilde{T}_k - T_k| + 2^p |T_k - (\mathcal{S}'u)_k| \\ &< (\sum_j 2^p |\tilde{z}_j - z_j|) + 1 < (2m) \cdot 7 + 1 = 14m + 1. \end{aligned}$$

As $m < \lceil p^{1/2} \rceil p^{1/2} \leq p + p^{1/2}$ and $p \geq 100$, we find that

$$2^p |\tilde{T}_k - (\mathcal{S}'u)_k| \leq 14p + 14p^{1/2} + 1 < 16p,$$

as desired. Recalling from the proof of Proposition 4.7(i) that $\|\mathcal{S}'\| < \frac{3}{4}$, we also have

$$|\tilde{T}_k| \leq |\tilde{T}_k - (\mathcal{S}'u)_k| + |(\mathcal{S}'u)_k| < 16p \cdot 2^{-p} + \|\mathcal{S}'\| \|u\| \leq 10^{-26} + \frac{3}{4} \cdot 1 < 1.$$

The cost of the above procedure for each k is $O(mp^{1+\delta}) = O(p^{3/2+\delta}\alpha)$, so the total over all k is $O(tp^{3/2+\delta}\alpha)$. $\square$

Remark 4.10. The algorithm in the proof of Lemma 4.9 amounts to multiplying a vector by a matrix of the type shown in Figure 2, including only those entries that are numerically significant, which form a strip of width roughly $2m$ around the diagonal. In the Turing model we must account for the cost of moving the tape head to access the input data needed to process each row, i.e., for row k we must access those $u_{j \bmod s}$ such that $|j - \frac{sk}{t}| < m$. For most rows this is straightforward, as the relevant u_j 's lie in a contiguous interval, and the $(k+1)$ -th interval is obtained by shifting the k -th interval $O(1)$ cells to the right. However, for the rows near the top and bottom of the matrix, namely for $k < (t/s)(m-1)$ and $k > (t/s)(s-m)$, the relevant u_j actually lie in two intervals separated by a gap of about $s-2m$ cells. For example, when $k=0$, the relevant values are $u_0, \dots, u_{m-1}$ and $u_{s-m+1}, \dots, u_{s-1}$. As there are $O(mt/s)$ exceptional rows, the extra cost of jumping over these gaps is $O(mtp) = O(tp^{3/2}\alpha)$, which still fits within the target time bound. Similar remarks apply to the proof of Lemma 4.11 below.

Lemma 4.11. *Assume the hypotheses of Proposition 4.7. We may construct a numerical approximation $\tilde{\mathcal{E}}: \tilde{\mathbb{C}}_o^s \rightarrow \tilde{\mathbb{C}}_o^s$ for $\mathcal{E}$ such that $\varepsilon(\tilde{\mathcal{E}}) < \frac{1}{3}p$ and $\mathcal{C}(\tilde{\mathcal{E}}) = O(tp^{3/2+\delta}\alpha^{-1})$.*

Proof. The argument is similar to the proof of Lemma 4.9. Given as input $u \in \tilde{\mathbb{C}}_o^s$, for each $\ell = 0, \dots, s-1$ we approximate $(\mathcal{E}u)_\ell$ as follows. By definition

$$(\mathcal{E}u)_\ell = \sum_{h \in \mathbb{Z} \setminus \{0\}} e^{-\pi\alpha^2 \left(\left(\frac{th}{s} + \beta_\ell \right)^2 - \beta_{\ell+h}^2 \right)} u_{\ell+h}.$$

As in the proof of Lemma 4.6, for $h \neq 0$ we have

$$\begin{aligned} \left(\frac{th}{s} + \beta_\ell \right)^2 - \beta_{\ell+h}^2 &> (t/s)^2 (|h| - \frac{1}{2})^2 - \frac{1}{4} \\ &> (|h| - \frac{1}{2})^2 - \frac{1}{4} = |h|(|h| - 1) \geq (|h| - 1)^2. \end{aligned}$$

Let $m := \lceil (p/4\alpha^2)^{1/2} \rceil = \lceil p^{1/2}/2\alpha \rceil \geq 1$, and consider the truncated sum

$$(4.4) \quad T_\ell := \sum_{\substack{h \in \mathbb{Z} \setminus \{0\} \\ |h| \leq m}} e^{-\pi\alpha^2 \left(\left(\frac{th}{s} + \beta_\ell \right)^2 - \beta_{\ell+h}^2 \right)} u_{\ell+h}.$$

As $\|u\| \leq 1$ we have

$$|(\mathcal{E}u)_\ell - T_\ell| \leq \sum_{|h| > m} e^{-\pi\alpha^2 \left(\left(\frac{th}{s} + \beta_\ell \right)^2 - \beta_{\ell+h}^2 \right)} |u_{\ell+h}| < \sum_{|h| > m} e^{-\pi\alpha^2 (|h|-1)^2}.$$

Let $w := e^{-\pi\alpha^2} \leq e^{-4\pi} < 10^{-5}$; then $w^m < 10^{-5}$ and

$$\begin{aligned} |(\mathcal{E}u)_\ell - T_\ell| &\leq 2(w^{m^2} + w^{(m+1)^2} + w^{(m+2)^2} + \dots) \\ &= 2w^{m^2}(1 + w^{2m+1} + w^{4m+4} + \dots) \\ &< 3w^{m^2} \leq 3e^{-\pi\alpha^2(p/4\alpha^2)} = 3e^{-\pi p/4} < 3 \cdot 2^{-p}. \end{aligned}$$

Now we explain how to approximate T_ℓ . The sum (4.4) has exactly $2m$ terms. For each h appearing in (4.4), let $x_h := e^{-\pi\alpha^2((th/s+\beta_\ell)^2-\beta_{\ell+h}^2)}$ and $z_h := x_h u_{\ell+h}$, so that $T_\ell = \sum_h z_h$. As in the proof of Lemma 4.9, we may use Lemma 2.13 to compute an approximation $\tilde{x}_h \in \tilde{\mathbb{C}}_\circ$ such that $\varepsilon(\tilde{x}_h) < 2$ in time $O(p^{1+\delta})$. Using Corollary 2.10, we then compute $\tilde{z}_h \in \tilde{\mathbb{C}}_\circ$ such that $\varepsilon(\tilde{z}_h) < \varepsilon(\tilde{x}_h) + 2 < 4$. Finally we set $\tilde{T}_\ell := \sum_h \tilde{z}_h$. We have

$$\begin{aligned} 2^p |\tilde{T}_\ell - (\mathcal{E}u)_\ell| &\leq 2^p |\tilde{T}_\ell - T_\ell| + 2^p |T_\ell - (\mathcal{E}u)_\ell| \\ &< (\sum_h 2^p |\tilde{z}_h - z_h|) + 3 < (2m) \cdot 4 + 3 = 8m + 3. \end{aligned}$$

As $m \leq (p^{1/2}/2\alpha) + 1 \leq \frac{1}{4}p^{1/2} + 1$ and $p \geq 100$, we find that

$$2^p |\tilde{T}_\ell - (\mathcal{E}u)_\ell| < 2p^{1/2} + 11 < \frac{1}{3}p.$$

Recalling that $\|\mathcal{E}\| < 0.42$ (see the proof of Proposition 4.7(i)), it follows that

$$|\tilde{T}_\ell| \leq |\tilde{T}_\ell - (\mathcal{E}u)_\ell| + |(\mathcal{E}u)_\ell| < \frac{1}{3}p \cdot 2^{-p} + \|\mathcal{E}\| \|u\| < 10^{-28} + 0.42 \cdot 1 < 1,$$

so we may define $(\tilde{\mathcal{E}}u)_\ell := \tilde{T}_\ell \in \tilde{\mathbb{C}}_\circ$. The cost of the above procedure for each ℓ is $O(mp^{1+\delta})$. The hypothesis $\alpha < p^{1/2}$ implies that $m \leq \frac{1}{2}p^{1/2}\alpha^{-1} + 1 = O(p^{1/2}\alpha^{-1})$, so the total cost over all ℓ is $O(tp^{3/2+\delta}\alpha^{-1})$. $\square$

Lemma 4.12. *Assume the hypotheses of Proposition 4.7. We may construct a numerical approximation $\tilde{\mathcal{J}}': \tilde{\mathbb{C}}_\circ^s \rightarrow \tilde{\mathbb{C}}_\circ^s$ for $\mathcal{J}'$ such that $\varepsilon(\tilde{\mathcal{J}}') < \frac{3}{4}p^2$ and $\mathcal{C}(\tilde{\mathcal{J}}') = O(tp^{3/2+\delta}\alpha)$.*

Proof. We are given as input $u \in \tilde{\mathbb{C}}_\circ^s$. Let $v := u/2 \in \mathbb{C}_\circ^s$, and define $v^{(j)} := \mathcal{E}^j v \in \mathbb{C}_\circ^s$ for $j \geq 0$ (recall that $\|\mathcal{E}\| < 0.42$). We wish to approximate

$$\mathcal{J}'u = (\mathcal{N}^{-1}/2)u = \mathcal{N}^{-1}v = v - \mathcal{E}v + \mathcal{E}^2v - \dots = v^{(0)} - v^{(1)} + v^{(2)} - \dots.$$

Let $n := \lceil p/\alpha^2\theta \rceil = \lceil ps/\alpha^2(t-s) \rceil \geq 1$. We compute a sequence of approximations $\tilde{v}^{(0)}, \dots, \tilde{v}^{(n-1)} \in \tilde{\mathbb{C}}_\circ^s$ as follows. First set $\tilde{v}^{(0)} := \rho(v^{(0)}) = 2^{-p}\rho_0(2^{p-1}u) \in \tilde{\mathbb{C}}_\circ^s$, so that $\varepsilon(\tilde{v}^{(0)}) < 2$. Then compute in sequence $\tilde{v}^{(j)} := \tilde{\mathcal{E}}\tilde{v}^{(j-1)} \in \tilde{\mathbb{C}}_\circ^s$ for $j = 1, \dots, n-1$, using Lemma 4.11. We claim that $\varepsilon(\tilde{v}^{(j)}) < \frac{2}{3}p$ for each j . This is clear for $j = 0$. For $j \geq 1$ it follows by induction, as

$$\begin{aligned} \varepsilon(\tilde{v}^{(j)}) &= 2^p \|\tilde{v}^{(j)} - v^{(j)}\| = 2^p \|\tilde{\mathcal{E}}\tilde{v}^{(j-1)} - \mathcal{E}v^{(j-1)}\| \\ &\leq 2^p \|\tilde{\mathcal{E}}\tilde{v}^{(j-1)} - \mathcal{E}\tilde{v}^{(j-1)}\| + 2^p \|\mathcal{E}\tilde{v}^{(j-1)} - \mathcal{E}v^{(j-1)}\| \\ &\leq \varepsilon(\tilde{\mathcal{E}}) + \|\mathcal{E}\| \varepsilon(\tilde{v}^{(j-1)}) < \frac{1}{3}p + \frac{1}{2} \cdot \frac{2}{3}p = \frac{2}{3}p. \end{aligned}$$

Finally, define $\tilde{\mathcal{J}}'u := \tilde{v}^{(0)} - \tilde{v}^{(1)} + \dots \pm \tilde{v}^{(n-1)}$. Then, as $\|v^{(j)}\| \leq \|\mathcal{E}^j\| \|v\| \leq \frac{1}{2} \|\mathcal{E}\|^j$ for all j , we have

$$\begin{aligned} 2^p \|\tilde{\mathcal{J}}'u - \mathcal{J}'u\| &\leq \sum_{j=0}^{n-1} 2^p \|\tilde{v}^{(j)} - v^{(j)}\| + 2^p \sum_{j=n}^{\infty} \|v^{(j)}\| < \frac{2}{3}np + \frac{2^p \|\mathcal{E}\|^n}{2(1 - \|\mathcal{E}\|)} \\ &< \frac{2}{3}np + 2^p \|\mathcal{E}\|^n. \end{aligned}$$

We already saw in the proof of Proposition 4.7(i) that $\alpha^2\theta \geq 1$; this implies that $n \leq p$, and also (via Lemma 4.6) that $\|\mathcal{E}\|^n < 2^{-\alpha^2\theta n} \leq 2^{-p}$. Thus

$$2^p \|\tilde{\mathcal{J}}'u - \mathcal{J}'u\| < \frac{2}{3}p^2 + 1 < \frac{3}{4}p^2$$

(since $p \geq 100$). This also shows that $\tilde{\mathcal{J}}'u \in \tilde{\mathbb{C}}_o^s$, as

$$\|\tilde{\mathcal{J}}'u\| \leq \|\mathcal{J}'u\| + \|\tilde{\mathcal{J}}'u - \mathcal{J}'u\| < \|\mathcal{J}'\| \|u\| + \frac{3}{4}p^2 \cdot 2^{-p} < \frac{7}{8} \cdot 1 + 10^{-26} < 1,$$

where the estimate $\|\mathcal{J}'\| < \frac{7}{8}$ again comes from the proof of Proposition 4.7(i).

In the above algorithm, the bulk of the work consists of $n - 1$ invocations of $\tilde{\mathcal{E}}$. The hypothesis $\theta \geq p/\alpha^4$ implies that

$$n \leq \frac{p}{\alpha^2\theta} + 1 \leq \alpha^2 + 1,$$

so the total cost is $O(tpn^{3/2+\delta}\alpha^{-1}) = O(tp^{3/2+\delta}\alpha)$. $\square$

Now we may complete the proof of Proposition 4.7.

Proof of Proposition 4.7 (ii). For $\tilde{\mathcal{A}}$ we simply take $\tilde{\mathcal{A}} := \tilde{\mathcal{S}}'$ where $\tilde{\mathcal{S}}'$ is as described in Lemma 4.9; then $\varepsilon(\tilde{\mathcal{A}}) < 16p < p^2$ (as $p \geq 100$), and $\mathcal{C}(\tilde{\mathcal{A}}) = O(tp^{3/2+\delta}\alpha)$.

For $\tilde{\mathcal{B}}$ we take $\tilde{\mathcal{B}} := \tilde{\mathcal{P}}_s^{-1} \tilde{\mathcal{D}}' \tilde{\mathcal{J}}' \tilde{\mathcal{C}} \tilde{\mathcal{P}}_t$, where $\tilde{\mathcal{D}}'$ and $\tilde{\mathcal{J}}'$ are as described in Lemmas 4.8 and 4.12, and where $\tilde{\mathcal{C}}: \tilde{\mathbb{C}}_o^t \rightarrow \tilde{\mathbb{C}}_o^s$, $\tilde{\mathcal{P}}_s^{-1}: \tilde{\mathbb{C}}_o^s \rightarrow \tilde{\mathbb{C}}_o^s$ and $\tilde{\mathcal{P}}_t: \tilde{\mathbb{C}}_o^t \rightarrow \tilde{\mathbb{C}}_o^t$ are the maps performing the obvious data rearrangements corresponding to $\mathcal{C}$, $\mathcal{P}_s^{-1}$ and $\mathcal{P}_t$, namely

$$\begin{aligned} (\tilde{\mathcal{C}}u)_\ell &:= u_{[t\ell/s]}, & u \in \tilde{\mathbb{C}}_o^t, \quad 0 \leq \ell < s, \\ (\tilde{\mathcal{P}}_s^{-1}u)_j &:= u_{(t^{-1} \bmod s)j}, & u \in \tilde{\mathbb{C}}_o^s, \quad 0 \leq j < s, \\ (\tilde{\mathcal{P}}_tu)_k &:= u_{-sk}, & u \in \tilde{\mathbb{C}}_o^t, \quad 0 \leq k < t. \end{aligned}$$

These do not perform any arithmetic in $\tilde{\mathbb{C}}_o$ so $\varepsilon(\tilde{\mathcal{C}}) = \varepsilon(\tilde{\mathcal{P}}_s^{-1}) = \varepsilon(\tilde{\mathcal{P}}_t) = 0$. By Corollary 2.8 we obtain $\varepsilon(\tilde{\mathcal{B}}) \leq \varepsilon(\tilde{\mathcal{D}}') + \varepsilon(\tilde{\mathcal{J}}') < 4 + \frac{3}{4}p^2 < p^2$.

As for the complexity, first observe that $\tilde{\mathcal{C}}$ simply copies its input in order, skipping $t - s$ unwanted entries, so $\mathcal{C}(\tilde{\mathcal{C}}) = O(tp)$. To compute $\tilde{\mathcal{P}}_s^{-1}u$ for $u \in \tilde{\mathbb{C}}_o^s$, we use a “label-and-sort” algorithm: we first construct the list of ordered pairs $(tj \bmod s, u_j)$ for $j = 0, \dots, s - 1$ in time $O(tp)$ (each label occupies $O(p)$ bits as $s < 2^p$), then sort the list by the first entry using merge sort in time $O(tp \log t)$ [31], and finally extract the second entries to obtain the desired output in time $O(tp)$. Thus $\mathcal{C}(\tilde{\mathcal{P}}_s^{-1}) = O(tp \log t)$, and similarly $\mathcal{C}(\tilde{\mathcal{P}}_t) = O(tp \log t)$. Altogether we have

$$\mathcal{C}(\tilde{\mathcal{B}}) = \mathcal{C}(\tilde{\mathcal{D}}') + \mathcal{C}(\tilde{\mathcal{J}}') + O(tp \log t) = O(tp^{1+\delta} + tp^{3/2+\delta}\alpha + tp \log t). \quad \square$$

Finally we show how to deduce the general case from the one-dimensional case.

Proof of Theorem 4.1. Let $s_1, \dots, s_d$ and $t_1, \dots, t_d$ be as in the statement of the theorem. Applying Proposition 4.7 for each i , we obtain maps $\mathcal{A}_i: \mathbb{C}^{s_i} \rightarrow \mathbb{C}^{t_i}$ and $\mathcal{B}_i: \mathbb{C}^{t_i} \rightarrow \mathbb{C}^{s_i}$ with $\|\mathcal{A}_i\|, \|\mathcal{B}_i\| \leq 1$ such that $\mathcal{F}_{s_i} = 2^{2\alpha^2} \mathcal{B}_i \mathcal{F}_{t_i} \mathcal{A}_i$, and approximations $\tilde{\mathcal{A}}_i: \tilde{\mathbb{C}}_o^{s_i} \rightarrow \tilde{\mathbb{C}}_o^{t_i}$ and $\tilde{\mathcal{B}}_i: \tilde{\mathbb{C}}_o^{t_i} \rightarrow \tilde{\mathbb{C}}_o^{s_i}$ such that $\varepsilon(\tilde{\mathcal{A}}_i), \varepsilon(\tilde{\mathcal{B}}_i) < p^2$ and $\mathcal{C}(\tilde{\mathcal{A}}_i), \mathcal{C}(\tilde{\mathcal{B}}_i) = O(t_i p^{3/2+\delta} \alpha + t_i p \log t_i)$.

Now observe that

$$\mathcal{F}_{s_1, \dots, s_d} = \otimes_i \mathcal{F}_{s_i} = 2^{2d\alpha^2} (\otimes_i \mathcal{B}_i) (\otimes_i \mathcal{F}_{t_i}) (\otimes_i \mathcal{A}_i) = 2^{2d\alpha^2} \mathcal{B} \mathcal{F}_{t_1, \dots, t_d} \mathcal{A},$$

where $\mathcal{A} := \otimes_i \mathcal{A}_i: \otimes_i \mathbb{C}^{s_i} \rightarrow \otimes_i \mathbb{C}^{t_i}$ and $\mathcal{B} := \otimes_i \mathcal{B}_i: \otimes_i \mathbb{C}^{t_i} \rightarrow \otimes_i \mathbb{C}^{s_i}$. Applying Lemma 2.11 (with $R := \mathbb{C}$, $r := 1$, $m_i := s_i$, $n_i := t_i$), we may construct an approximation $\tilde{\mathcal{A}}: \otimes_i \tilde{\mathbb{C}}_o^{s_i} \rightarrow \otimes_i \tilde{\mathbb{C}}_o^{t_i}$ such that $\varepsilon(\tilde{\mathcal{A}}) \leq \sum_i \varepsilon(\tilde{\mathcal{A}}_i) < dp^2$. Moreover, let M be as in Lemma 2.11; then $M = \prod_i \max(s_i, t_i) = t_1 \cdots t_d = T$, so

$$\begin{aligned} \mathcal{C}(\tilde{\mathcal{A}}) &< T \sum_i \frac{\mathcal{C}(\tilde{\mathcal{A}}_i)}{t_i} + O(Tp \log T) \\ &< T \sum_i O(p^{3/2+\delta} \alpha + p \log t_i) + O(Tp \log T) \\ &= O(dTp^{3/2+\delta} \alpha + Tp \log T). \end{aligned}$$

We may similarly construct an approximation $\tilde{\mathcal{B}}$ satisfying exactly the same error and cost bounds, and this completes the proof. $\square$

4.4. Further remarks. Our presentation of the Gaussian resampling technique has been optimised in favour of giving the simplest possible proof of the main $M(n) = O(n \log n)$ bound. In this section we outline several ways in which these results may be improved and generalised, with an eye towards practical applications.

4.4.1. Minor technical issues. In our presentation we insisted that α be an integer and that $\alpha \geq 2$. Neither of these restrictions are essential; they were made for technical reasons to simplify certain proofs.

Similarly, the assumption $\gcd(s, t) = 1$ is not necessary. We briefly outline what modifications must be made to handle the case $g := \gcd(s, t) > 1$. For $0 \leq h < g$, define maps $\mathcal{P}_{s,h}: \mathbb{C}^s \rightarrow \mathbb{C}^{s/g}$, $\mathcal{P}_{t,h}: \mathbb{C}^t \rightarrow \mathbb{C}^{t/g}$ and $\mathcal{T}_h: \mathbb{C}^{s/g} \rightarrow \mathbb{C}^{t/g}$ by

$$\begin{aligned} (\mathcal{P}_{s,h} u)_j &:= u_{tj+h}, & u \in \mathbb{C}^s, & \quad 0 \leq j < s/g, \\ (\mathcal{P}_{t,h} u)_k &:= u_{-sk+h}, & u \in \mathbb{C}^t, & \quad 0 \leq k < t/g, \\ (\mathcal{T}_h u)_k &:= \sum_{j \in \mathbb{Z}} e^{-\pi \alpha^2 t^2 \left(\frac{k}{t} - \frac{j}{s} - \frac{h}{st} \right)^2} u_j, & u \in \mathbb{C}^{s/g}, & \quad 0 \leq k < t/g. \end{aligned}$$

Then one may prove (analogously to Theorem 4.2) that $\mathcal{T}_h \mathcal{P}_{s,h} \mathcal{F}_s = \mathcal{P}_{t,h} \mathcal{F}_t \mathcal{S}$ for each h . In other words, for $u \in \mathbb{C}^s$, the matrix $\mathcal{T}_h$ gives a system of linear equations that relate the coefficients $(\mathcal{F}_t \mathcal{S} u)_j$ and $(\mathcal{F}_s u)_j$ for those j congruent to h modulo g . One may use this to prove an analogue of Theorem 4.1, by first constructing a left inverse for each $\mathcal{T}_h$ along the lines of Section 4.2.

4.4.2. Faster system solving. The iterative method used in Lemma 4.12 to approximate $\mathcal{J} = \mathcal{N}^{-1}$ (i.e., to solve the system $\mathcal{T}x = y$) has complexity $O(tp^{5/2+\delta}/\alpha^3 \theta)$. To ensure that this step does not dominate the $O(tp^{3/2+\delta} \alpha)$ complexity of approximating $\mathcal{S}$ (Lemma 4.9), we were compelled to introduce the hypothesis $\theta \geq p/\alpha^4$. On the other hand, to make the target DFT of length t as cheap as possible, it

is desirable for θ to be as close to zero as possible. Together, these considerations imply that we cannot take α smaller than about $p^{1/4}$. (Indeed, in Section 5, for fixed d , we do take $\alpha = \Theta(p^{1/4})$ for this very reason.) For the choice $\alpha = \Theta(p^{1/4})$, the overall complexity in Proposition 4.7 is $O(tp^{7/4+\delta})$.

A better complexity bound may be obtained by precomputing an LU decomposition for $\mathcal{N}$, and then solving the system directly. The cost of the precomputation is $O(tp^{2+\delta}/\alpha^2)$ (assuming classical matrix arithmetic, while exploiting the circular banded structure), and then the cost of each application of $\mathcal{J}$ becomes $O(tp^{3/2+\delta}/\alpha)$. This allows us to relax the condition $\theta \geq p/\alpha^4$ to merely $\theta \geq 1/\alpha^2$. Taking $\alpha = \Theta(1)$, the overall complexity in Proposition 4.7 (discounting the precomputation) falls to $O(tp^{3/2+\delta})$. We did not use this method in our presentation because the error analysis is considerably more intricate than for the iterative method.

After making this modification, it would be interesting to investigate whether this method is competitive for practical computations of complex DFTs of length s when s is a large prime. One would choose a smooth transform length t somewhat larger than s , say $1.25s < t < 1.5s$, and use the algorithm to reduce the desired DFT of length s to a DFT of length t ; the latter could be handled via existing software libraries implementing the Cooley–Tukey algorithm. For large enough s , perhaps around 2^{20} or 2^{30} , we expect that the invocations of $\tilde{\mathcal{S}}$ and $\tilde{\mathcal{J}}$ would be quite cheap compared to the FFT of length t . Indeed, $\tilde{\mathcal{S}}$ can be computed in a single pass over the input vector, and $\tilde{\mathcal{J}}$ in two passes (one for each of the L and U matrices), so they have excellent locality. It is conceivable that a highly optimised implementation could outperform existing software libraries, which handle transforms of prime length by techniques such as Rader’s algorithm [38]. Such techniques introduce a large constant factor overhead that does not arise in the method just sketched.

4.4.3. Comparison with the Dutt–Rokhlin method. There is an enormous literature on “non-uniform FFTs” (sometimes called “non-equispaced FFTs”), going back to the seminal paper of Dutt and Rokhlin [11]. They consider transforms of the type

$$(4.5) \quad v_j := \sum_{k=0}^{t-1} e^{2\pi i \omega_k y_j} u_k, \quad 0 \leq j < t.$$

The ordinary “uniform” DFT may be regarded as the special case where $\omega_k := k$ and $y_j := j/t$, but the Dutt–Rokhlin algorithms may be applied in cases where the frequencies ω_k are not necessarily integers, and/or the sample points y_j are not necessarily integer multiples of $1/t$. In these cases the algorithms reduce the problem to an ordinary FFT of length t (or in some variants, a small multiple of t). The complexity, counting floating-point operations, is $O(t \log t + tp)$, where p is the desired precision in bits.

If we now take instead $\omega_k := k$ and $y_j := j/s$, where s is the “source” transform length, we see that (4.5) is exactly a DFT of length s (apart from some inconsequential zero-padding), so the Dutt–Rokhlin algorithms may be used to compute a DFT of length s by means of an FFT of length t . Inspection of their algorithms in this case reveals them to be *essentially equivalent to our method in the special case that $\alpha = \Theta(p^{1/2})$* .

For example, consider [11, Algorithm 2], which corresponds roughly to a “transposed” version of our algorithm. Step 3 of that algorithm is analogous to approximating $\mathcal{S}$ (see Lemma 4.9). For the choice $\alpha = \Theta(p^{1/2})$, the complexity for this step is $O(tp^{2+\delta})$ bit operations, corresponding to the $O(tp)$ term in their complexity bound. Step 2 corresponds to our $\mathcal{F}_t$, and yields the $O(t \log t)$ term. The most interesting point of comparison is Step 1, which corresponds roughly to solving the system $\mathcal{T}x = y$. The choice $\alpha = \Theta(p^{1/2})$ implies that this system is essentially *diagonal*, i.e., the off-diagonal entries of $\mathcal{T}$ decay so rapidly that for numerical purposes they may be discarded. Solving the system is therefore trivial: their Step 1 consists of simply dividing each coefficient by the corresponding diagonal entry of $\mathcal{T}$ (in the literature these are often called “scale factors”). This step contributes only $O(t)$ floating-point operations.

The reason that Dutt and Rokhlin are (in effect) unable to take α smaller than about $p^{1/2}$ is essentially due to the approximation error committed when they truncate the Gaussian, for example in [11, Theorem 2.7]. Our Theorem 4.1 may be viewed as an “exact” replacement for that theorem. Rather than truncate the Gaussian, we take into account the effect of the Gaussian tail, which manifests as the off-diagonal entries of our $\mathcal{T}$ matrix. For α considerably smaller than $p^{1/2}$, these entries are numerically significant and cannot be ignored.

In our algorithm, assuming that we use the LU decomposition method mentioned in Section 4.4.2, as α decreases from $\Theta(p^{1/2})$ to $\Theta(1)$ we see that the complexity of approximating $\mathcal{S}$ decreases from $O(tp)$ to $O(tp^{1/2})$ floating-point operations, and the complexity of approximating $\mathcal{J}$ (i.e., solving the $\mathcal{T}$ system) increases from $O(t)$ to $O(tp^{1/2})$ floating-point operations. When $\alpha = \Theta(1)$ they are balanced, and the overall complexity drops to $O(t \log t + tp^{1/2})$; the last term improves on the Dutt–Rokhlin bound by a factor of $p^{1/2}$. Note that the Dutt–Rokhlin bound is not strong enough for our application to integer multiplication; using their bound, the error term in Proposition 5.2 would grow to $O(n(\log n)^{1+\delta})$ which is unacceptably large.

Of course, our discussion has only considered the case corresponding to the DFT of length s , i.e., the choice $y_j := j/s$. An interesting question is whether the bound $O(t \log t + tp^{1/2})$ can be proved for the general non-equispaced case, and if so, whether this method outperforms the Dutt–Rokhlin algorithm in practice.

5. THE MAIN ALGORITHM

In this section we present the main integer multiplication algorithm. We actually give a family of algorithms, parameterised by a dimension parameter $d \geq 2$. Let

$$n_0 := 2^{d^{12}} \geq 2^{4096},$$

and suppose that we wish to multiply integers with n bits. For $n < n_0$, we may use any convenient base-case multiplication algorithm, such as the classical $O(n^2)$ algorithm. For $n \geq n_0$ we will describe a recursive algorithm that reduces the problem to a collection of multiplication problems of size roughly $n^{1/d}$. We will show that this algorithm achieves $M(n) = O(n \log n)$, provided that $d \geq 1729$.

5.1. Parameter selection. Henceforth we assume that $n \geq n_0$. We first discuss the computation of several parameters depending on n that will be needed later.

Let

$$(5.1) \quad b := \lceil \log_2 n \rceil \geq d^{12} \geq 4096$$

be the “chunk size”, and let the working precision be

$$(5.2) \quad p := 6b = 6\lceil \log_2 n \rceil \geq 6d^{12} \geq 24576 > 100.$$

Define

$$(5.3) \quad \alpha := \lceil (12d^2b)^{1/4} \rceil.$$

Clearly $\alpha \geq 2$, and as $d \leq b^{1/12}$ and $b \geq 4096$, we also have

$$(5.4) \quad \alpha \leq \lceil 12^{1/4} b^{7/24} \rceil \leq 1.87 \cdot b^{7/24} + 1 < 2b^{7/24} < p^{1/2}.$$

As in Theorem 4.1, set

$$(5.5) \quad \gamma := 2d\alpha^2 < 2b^{1/12} \cdot 4b^{7/12} = 8b^{2/3}.$$

Let T be the unique power of two lying in the interval

$$(5.6) \quad 4n/b \leq T < 8n/b,$$

and let r be the unique power of two in the interval

$$(5.7) \quad T^{1/d} \leq r < 2T^{1/d}.$$

We certainly have $b \leq 4n^{1/2}$, so

$$(5.8) \quad r \geq T^{1/d} \geq (4n/b)^{1/d} \geq (n^{1/2})^{1/d} \geq n^{1/d^2} \geq n_0^{1/d^2} \geq 2^{d^{10}}.$$

We now construct a factorisation $T = t_1 \cdots t_d$ satisfying the hypotheses of Theorem 3.1. Let $d' := \log_2(r^d/T)$. As $T \leq r^d < 2^dT$ we have $1 \leq r^d/T < 2^d$ and hence $0 \leq d' < d$. Define

$$t_1, \dots, t_{d'} := \frac{r}{2}, \quad t_{d'+1}, \dots, t_d := r.$$

Then $t_d \geq \dots \geq t_1 \geq 2$ and

$$t_1 \cdots t_d = (r/2)^{d'} r^{d-d'} = r^d / 2^{d'} = T.$$

Also

$$(5.9) \quad T < 8n/b < n \leq 2^b < 2^p,$$

so the hypotheses of Theorem 3.1 are indeed satisfied. The parameters $b, p, \alpha, \gamma, T, r$ and $t_1, \dots, t_d$ may all be computed in time $(\log n)^{O(1)}$.

Our next task is to choose distinct primes $s_1, \dots, s_d$ that are slightly smaller than the corresponding $t_1, \dots, t_d$. In a moment we will use Theorem 4.1 to reduce a transform of size $s_1 \times \dots \times s_d$ to a transform of size $t_1 \times \dots \times t_d$; to avoid excessive data expansion in this reduction, we must ensure that the ratio $t_1 \cdots t_d / s_1 \cdots s_d$ is not too large. On the other hand, to satisfy the requirements of the theorem, we must also ensure that the individual ratios t_i/s_i are not too close to 1. We will achieve this by means of the following result.

Lemma 5.1. *Let $\eta \in (0, \frac{1}{4})$ and let $x \geq e^{2/\eta}$. Then there are at least $\frac{1}{2}\eta x / \log x$ primes q in the interval $(1 - 2\eta)x < q \leq (1 - \eta)x$.*

Proof. Let $\vartheta(y) := \sum_{q \leq y} \log q$ (sum taken over primes) denote the usual Chebyshev function. According to [39, Thm. 4], for all $y \geq 563$ we have

$$y - \frac{y}{2 \log y} < \vartheta(y) < y + \frac{y}{2 \log y}.$$

As the function $y/\log y$ is increasing for $y \geq 563$, we see that

$$y - \frac{x}{2\log x} < \vartheta(y) < y + \frac{x}{2\log x}, \quad 563 \leq y \leq x.$$

Applying this result for $y_0 := (1 - 2\eta)x > x/2 > e^8/2 > 563$, and then again for $y_1 := (1 - \eta)x > 3x/4 > 563$, we obtain

$$\vartheta(y_1) - \vartheta(y_0) > y_1 - \frac{x}{2\log x} - y_0 - \frac{x}{2\log x} = \eta x - \frac{x}{\log x} \geq \eta x - \frac{x}{2/\eta} = \frac{\eta x}{2}$$

and hence

$$\sum_{y_0 < q \leq y_1} 1 \geq \frac{1}{\log x} \sum_{y_0 < q \leq y_1} \log q = \frac{\vartheta(y_1) - \vartheta(y_0)}{\log x} > \frac{\eta x}{2\log x}. \quad \square$$

Let us apply Lemma 5.1 with

$$(5.10) \quad \eta := \frac{1}{4d} \leq \frac{1}{8}$$

and $x := r/2$, noting that (5.8) implies that $r/2 \geq 2^{d^{10}-1} \geq e^{8d} = e^{2/\eta}$. We find that there are at least

$$\frac{1}{8d} \cdot \frac{r/2}{\log(r/2)} \geq \frac{1}{16d} \cdot \frac{r}{\log r} \geq \frac{1}{16d} \cdot \frac{2^{d^{10}}}{\log(2^{d^{10}})} = \frac{2^{d^{10}}}{(16 \log 2)d^{11}} \geq d \geq d'$$

primes q in the interval

$$(1 - 2\eta)\frac{r}{2} < q \leq (1 - \eta)\frac{r}{2}.$$

Using Eratosthenes' sieve, we may find d' such primes $s_1, \dots, s_{d'}$ in time $r^{1+o(1)} = o(n)$. Applying Lemma 5.1 again, with the same η but now with $x := r \geq e^{2/\eta}$, we find that there are at least $d \geq d - d'$ primes q in the interval

$$(1 - 2\eta)r < q \leq (1 - \eta)r.$$

Again, we may find $d - d'$ such primes $s_{d'+1}, \dots, s_d$ in time $o(n)$. These two collections of primes are disjoint, as

$$(1 - \eta)\frac{r}{2} < \frac{r}{2} < \frac{3r}{4} \leq (1 - 2\eta)r.$$

In summary, we have found d distinct primes $s_1, \dots, s_d$ such that

$$(5.11) \quad (1 - 2\eta)t_i < s_i \leq (1 - \eta)t_i, \quad i \in \{1, \dots, d\}.$$

Setting $S := s_1 \cdots s_d < T$, we see that

$$(5.12) \quad \frac{S}{T} > (1 - 2\eta)^d = \left(1 - \frac{1}{2d}\right)^d > \frac{1}{2},$$

as $(1 - x)^d > 1 - dx$ for all $x \in (0, 1)$.

5.2. DFTs and convolutions for coprime sizes. The following result combines Theorems 3.1 and 4.1 to obtain an approximation for the complex transform $\mathcal{F}_{s_1, \dots, s_d} : \otimes_i \mathbb{C}^{s_i} \rightarrow \otimes_i \mathbb{C}^{s_i}$ (defined in Section 2.4). Recall that the working precision was chosen to be $p := 6b = 6\lceil \log_2 n \rceil$.

Proposition 5.2. *We may construct a numerical approximation $\tilde{\mathcal{F}}_{s_1, \dots, s_d} : \otimes_i \tilde{\mathbb{C}}_o^{s_i} \rightarrow \otimes_i \tilde{\mathbb{C}}_o^{s_i}$ for $\mathcal{F}_{s_1, \dots, s_d}$ such that $\varepsilon(\tilde{\mathcal{F}}_{s_1, \dots, s_d}) < 2^{\gamma+5} T \log_2 T$ and*

$$\mathbf{C}(\tilde{\mathcal{F}}_{s_1, \dots, s_d}) < \frac{4T}{r} \mathbf{M}(3rp) + O(n \log n).$$

Proof. Let us verify the hypotheses of Theorem 4.1. We have already shown that $2 \leq s_i < t_i < 2^p$ for all i (see (5.9) and (5.11)) and that $2 \leq \alpha < p^{1/2}$ (see (5.4)). We certainly have $\gcd(s_i, t_i) = 1$, as s_i is an odd prime and t_i is a power of two. Let $\theta_i := t_i/s_i - 1$; then by (5.10) and (5.11) we have

$$\theta_i \geq \frac{1}{1-\eta} - 1 = \frac{\eta}{1-\eta} > \eta = \frac{1}{4d}.$$

From (5.3) and (5.2) we see that

$$\alpha^4 \theta_i \geq \frac{12d^2 b}{4d} = 3db = \frac{d}{2} \cdot p \geq p,$$

so $\theta_i \geq p/\alpha^4$ as required.

Theorem 4.1 thus produces maps $\mathcal{A} : \otimes_i \mathbb{C}^{s_i} \rightarrow \otimes_i \mathbb{C}^{t_i}$ and $\mathcal{B} : \otimes_i \mathbb{C}^{t_i} \rightarrow \otimes_i \mathbb{C}^{s_i}$ such that $\mathcal{F}_{s_1, \dots, s_d} = 2^\gamma \mathcal{B} \mathcal{F}_{t_1, \dots, t_d} \mathcal{A}$ (where γ is given by (5.5)), and approximations $\tilde{\mathcal{A}} : \otimes_i \tilde{\mathbb{C}}_o^{s_i} \rightarrow \otimes_i \tilde{\mathbb{C}}_o^{t_i}$ and $\tilde{\mathcal{B}} : \otimes_i \tilde{\mathbb{C}}_o^{t_i} \rightarrow \otimes_i \tilde{\mathbb{C}}_o^{s_i}$. Applying Theorem 3.1 (whose hypotheses were verified in Section 5.1), we obtain furthermore an approximation $\tilde{\mathcal{F}}_{t_1, \dots, t_d}$ for $\mathcal{F}_{t_1, \dots, t_d}$. Now consider the scaled transform

$$\mathcal{F}'_{s_1, \dots, s_d} := 2^{-\gamma} \mathcal{F}_{s_1, \dots, s_d} = \mathcal{B} \mathcal{F}_{t_1, \dots, t_d} \mathcal{A},$$

and the approximation $\tilde{\mathcal{F}}'_{s_1, \dots, s_d} := \tilde{\mathcal{B}} \tilde{\mathcal{F}}_{t_1, \dots, t_d} \tilde{\mathcal{A}}$. The maps $\mathcal{A}$, $\mathcal{B}$ and $\mathcal{F}_{t_1, \dots, t_d}$ all have norm at most 1 (by Theorem 4.1 and Example 2.6), so Corollary 2.8 implies that

$$\varepsilon(\tilde{\mathcal{F}}'_{s_1, \dots, s_d}) \leq \varepsilon(\tilde{\mathcal{B}}) + \varepsilon(\tilde{\mathcal{F}}_{t_1, \dots, t_d}) + \varepsilon(\tilde{\mathcal{A}}) < 2dp^2 + 8T \log_2 T.$$

As $\|\mathcal{F}_{s_1, \dots, s_d}\| \leq 1$, we obtain the desired approximation $\tilde{\mathcal{F}}_{s_1, \dots, s_d}$ by applying Lemma 2.2 with $c := 2^\gamma$ to the output of $\tilde{\mathcal{F}}'_{s_1, \dots, s_d}$ (the condition $c \leq 2^p$ holds as $\gamma < 8b^{2/3} < p$; see (5.5)). We therefore obtain

$$\begin{aligned} \varepsilon(\tilde{\mathcal{F}}_{s_1, \dots, s_d}) &< 2^{\gamma+1} \varepsilon(\tilde{\mathcal{F}}'_{s_1, \dots, s_d}) + 3 \\ &< 2^{\gamma+1} (2dp^2 + 8T \log_2 T) + 3 \\ &< 2^{\gamma+1} (3dp^2 + 8T \log_2 T). \end{aligned}$$

Moreover, by (5.1), (5.2) and (5.6) we have

$$3dp^2 \leq 108b^{1/12}b^2 = 108\lceil \log_2 n \rceil^{25/12} < n^{1/2} < T < 8T \log_2 T,$$

so we conclude that $\varepsilon(\tilde{\mathcal{F}}_{s_1, \dots, s_d}) < 2^{\gamma+5} T \log_2 T$.

The cost of the scaling step is $O(Sp^{1+\delta}) = O(Tp^{1+\delta})$, so by Theorem 3.1 and Theorem 4.1 the overall complexity is

$$\begin{aligned} \mathbf{C}(\tilde{\mathcal{F}}_{s_1, \dots, s_d}) &= \mathbf{C}(\tilde{\mathcal{A}}) + \mathbf{C}(\tilde{\mathcal{F}}_{t_1, \dots, t_d}) + \mathbf{C}(\tilde{\mathcal{B}}) + O(Tp^{1+\delta}) \\ &= \frac{4T}{r} \mathbf{M}(3rp) + O(dTp^{3/2+\delta}\alpha + Tp \log T + Tp^{1+\delta}). \end{aligned}$$

Recalling (5.4) and the assumption $\delta < \frac{1}{8}$ from Section 2.1, we see that

$$dp^{3/2+\delta}\alpha = O(p^{1/12}p^{3/2}p^{1/8}p^{7/24}) = O(p^2).$$

By definition $p = O(\log n)$, and (5.6) yields $T = O(n/\log n)$. The bound for $\mathbb{C}(\tilde{\mathcal{F}}_{s_1, \dots, s_d})$ thus simplifies to $(4T/r)\mathbb{M}(3rp) + O(n \log n)$. $\square$

Next we construct an approximation for the scaled convolution map $\mathcal{M}: \otimes_i \mathbb{C}^{s_i} \times \otimes_i \mathbb{C}^{s_i} \rightarrow \otimes_i \mathbb{C}^{s_i}$ given by $\mathcal{M}(u, v) := \frac{1}{S}u * v$, where $*$ is the convolution operator defined in Section 2.4. Note that $\|\mathcal{M}\| \leq 1$.

Proposition 5.3. *We may construct a numerical approximation $\tilde{\mathcal{M}}: \otimes_i \tilde{\mathbb{C}}_{\circ}^{s_i} \times \otimes_i \tilde{\mathbb{C}}_{\circ}^{s_i} \rightarrow \otimes_i \tilde{\mathbb{C}}_{\circ}^{s_i}$ for $\mathcal{M}$ such that $\varepsilon(\tilde{\mathcal{M}}) < 2^{\gamma+8}T^2 \log_2 T$ and*

$$(5.13) \quad \mathbb{C}(\tilde{\mathcal{M}}) < \frac{12T}{r} \mathbb{M}(3rp) + O(n \log n).$$

Proof. We are given as input $u, v \in \otimes_i \tilde{\mathbb{C}}_{\circ}^{s_i}$. Let $w := \mathcal{M}(u, v) = \frac{1}{S}u * v \in \otimes_i \mathbb{C}^{s_i}$. According to (2.1) we have $w = Sw'$ where $w' := \mathcal{F}_{s_1, \dots, s_d}^*(\mathcal{F}_{s_1, \dots, s_d}u \cdot \mathcal{F}_{s_1, \dots, s_d}v)$. We use the following algorithm (essentially the same as in the proof of Proposition 3.4, but working over $\mathbb{C}$ instead of $\mathcal{R}$). We first compute an approximation $\tilde{w}' \in \otimes_i \tilde{\mathbb{C}}_{\circ}^{s_i}$ by using Proposition 5.2 to handle the forward and inverse transforms, and Corollary 2.10 to handle the pointwise multiplications. Applying Lemma 2.7 in the usual way, we obtain

$$\begin{aligned} \varepsilon(\tilde{w}') &\leq \varepsilon(\tilde{\mathcal{F}}_{s_1, \dots, s_d}) + \varepsilon(\tilde{\mathcal{F}}_{s_1, \dots, s_d}) + \varepsilon(\tilde{\mathcal{F}}_{s_1, \dots, s_d}^*) + 2 \\ &< 3 \cdot 2^{\gamma+5}T \log_2 T + 2 < \frac{7}{2} \cdot 2^{\gamma+5}T \log_2 T. \end{aligned}$$

Then we apply Lemma 2.2 (with $c := S \leq T \leq 2^p$, thanks to (5.9)) to obtain an approximation $\tilde{w} \in \otimes_i \tilde{\mathbb{C}}_{\circ}^{s_i}$ such that

$$\varepsilon(\tilde{w}) < 2S\varepsilon(\tilde{w}') + 3 < 7S \cdot 2^{\gamma+5}T \log_2 T + 3 < 2^{\gamma+8}T^2 \log_2 T.$$

The cost of the pointwise multiplications and scalings is $O(Sp^{1+\delta}) = O(n \log n)$, and the constant 12 accounts for the three invocations of Proposition 5.2. $\square$

5.3. Integer multiplication. We are now in a position to describe the recursive step of the main integer multiplication algorithm.

Proposition 5.4. *For $n \geq n_0$ we have*

$$(5.14) \quad \mathbb{M}(n) < \frac{12T}{r} \mathbb{M}(3rp) + O(n \log n).$$

Proof. We are given as input integers $0 \leq f, g < 2^n$. The algorithm consists of the following series of reductions.

(1) *Reduce to one-dimensional convolution over $\mathbb{Z}$.* Let $N := \lceil n/b \rceil$ where $b := \lceil \log_2 n \rceil$ as in (5.1). We split f and g into N chunks of b bits, i.e., we write $f = F(2^b)$ and $g = G(2^b)$ where

$$F(x) = \sum_{j=0}^{N-1} F_j x^j \in \mathbb{Z}[x], \quad G(x) = \sum_{j=0}^{N-1} G_j x^j \in \mathbb{Z}[x], \quad 0 \leq F_j, G_j < 2^b.$$

We have $fg = (FG)(2^b)$, so it suffices to compute the polynomial product $H(x) := F(x)G(x)$ and then evaluate at $x = 2^b$. The coefficients of $H(x)$ lie in the interval $0 \leq H_j < 2^{2b}N < 2^{3b}$; in particular, they have at most $3b = O(\log n)$ bits, so the

evaluation may be achieved via a straightforward overlap-add algorithm in time $O(N \log n) = O(n)$. By (5.6) and (5.12) we have

$$\deg H \leq 2N - 2 \leq \frac{2n}{b} \leq \frac{T}{2} < S,$$

so it suffices to compute $F(x)G(x) \pmod{x^S - 1}$.

(2) *Reduce to d -dimensional convolution over $\mathbb{Z}$.* We now use the Agarwal–Cooley method [1] to reduce to a multidimensional convolution. Consider the ring

$$\mathcal{A} := \mathbb{Z}[x_1, \dots, x_d] / (x_1^{s_1} - 1, \dots, x_d^{s_d} - 1).$$

As the integers $s_1, \dots, s_d$ are pairwise relatively prime, there is an isomorphism of rings $\mathbb{Z}[x] / (x^S - 1) \cong \mathcal{A}$ induced by the Chinese remainder theorem, namely by sending x to $x_1 \cdots x_d$. Let $F', G', H' \in \mathcal{A}$ be the images of F, G and H , so that $H' = F'G'$. In the Turing model, F', G' and H' are represented as d -dimensional arrays of integers of $3b = O(\log n)$ bits, using a similar layout to the tensor products in Section 2.3. The isomorphism amounts to a data rearrangement, and may be computed by attaching labels and sorting, as in the proof of Proposition 4.7(ii). The label $(i_1 \bmod s_1, \dots, i_d \bmod s_d)$ occupies $O(\sum_i \log s_i) = O(\log n)$ bits, and may be incremented to $(i_1 + 1 \bmod s_1, \dots, i_d + 1 \bmod s_d)$ in time $O(\log n)$. Therefore the isomorphism may be computed in either direction in time

$$O(S \log n \log S) = O(T \log^2 n) = O(n \log n).$$

(For an alternative algorithm that does not rely on sorting, see [23, Sec. 2.3].) We have thus reduced to the problem of computing $H' = F'G'$ in $\mathcal{A}$.

(3) *Reduce to approximate d -dimensional convolution over $\mathbb{C}$.* Regarding $\mathcal{A}$ as a subring of

$$\mathbb{C}[x_1, \dots, x_d] / (x_1^{s_1} - 1, \dots, x_d^{s_d} - 1) \cong (\otimes_i \mathbb{C}^{s_i}, *),$$

let $F'', G'', H'' \in \otimes_i \mathbb{C}^{s_i}$ be the elements corresponding to F', G' and H' , so that $H'' = F'' * G''$. Let $u := 2^{-b}F'', v := 2^{-b}G''$ and $w := \mathcal{M}(u, v) = \frac{1}{S}u * v$. Then $\|u\|, \|v\|, \|w\| \leq 1$, and $H'' = 2^{2b}Sw$. Recalling our choice of working precision $p := 6b = 6\lceil \log_2 n \rceil$, we may use Proposition 5.3 to compute an approximation $\tilde{w} := \tilde{\mathcal{M}}(u, v) \in \otimes_i \tilde{\mathbb{C}}_0^{s_i}$ such that $\varepsilon(\tilde{w}) < 2^{\gamma+8}T^2 \log_2 T$ in time $(12T/r)M(3rp) + O(n \log n)$.

Now observe that

$$\|H'' - 2^{2b}S\tilde{w}\| = 2^{2b}S\|w - \tilde{w}\| \leq 2^{2b}T \cdot 2^{-p}\varepsilon(\tilde{w}) < 2^{2b+\gamma+8-p}T^3 \log_2 T.$$

Since $T < n \leq 2^b$ and $T \log_2 T \leq T \log_2 n \leq Tb < 8n \leq 2^{b+3}$ (by (5.6)), this yields

$$\|H'' - 2^{2b}S\tilde{w}\| < 2^{2b+\gamma+8-p} \cdot 2^{2b} \cdot 2^{b+3} = 2^{5b+\gamma+11-p} = 2^{-b+\gamma+11}.$$

But (5.5) yields $\gamma < 8b^{2/3} < b - 13$ (as $b \geq 4096$), so

$$\|H'' - 2^{2b}S\tilde{w}\| < \frac{1}{4}.$$

In particular, we may recover H'' in time $O(Sp^{1+\delta}) = O(n \log n)$ by multiplying each coefficient of $\tilde{w}$ by $2^{2b}S$ and then rounding to the nearest integer. $\square$

Corollary 5.5. *Define $T(n) := M(n)/(n \log n)$ for $n \geq 2$. For $n \geq n_0$ we have*

$$T(n) < \frac{1728}{d - \frac{1}{2}} T(3rp) + O(1).$$

Proof. Dividing (5.14) by $n \log n$ yields

$$\mathsf{T}(n) < \frac{36Tp}{n} \cdot \frac{\log(3rp)}{\log n} \cdot \mathsf{T}(3rp) + O(1).$$

By (5.2) and (5.6) we have $36Tp/n = 216Tb/n < 1728$. Moreover, (5.7) implies that $r < 2T^{1/d} < 2n^{1/d}$, so

$$\frac{\log(3rp)}{\log n} = \frac{\log(r/2) + \log(36b)}{\log n} < \frac{1}{d} + \frac{\log(36b)}{\log n} = \frac{1}{d} + \frac{\log_2(36b)}{\log_2 n} \leq \frac{1}{d} + \frac{\log_2(36b)}{b-1}.$$

Since $b \geq 4096$ and $d \leq b^{1/12}$ (see (5.1)) we have

$$\frac{\log_2(36b)}{b-1} < \frac{1}{2b^{1/6}} \leq \frac{1}{2d^2},$$

and the result follows from the observation that

$$\frac{1}{d} + \frac{1}{2d^2} < \frac{1}{d} \left(1 - \frac{1}{2d}\right)^{-1} = \frac{1}{d - \frac{1}{2}}. \quad \square$$

Finally we may prove the main result of the paper.

Proof of Theorem 1.1. According to Corollary 5.5, there is an absolute constant $A > 0$ such that

$$\mathsf{T}(n) < \frac{1728}{d - \frac{1}{2}} \cdot \mathsf{T}(3rp) + A$$

for all $n \geq n_0$. We now take $d := 1729$. Then for all $n \geq n_0 = 2^{1729^{12}}$ we have

$$\mathsf{T}(n) < 0.9998 \cdot \mathsf{T}(3rp) + A.$$

Define

$$B := \max_{2 \leq n < n_0} \mathsf{T}(n), \quad C := \max(B, 5000A).$$

(Recall that for $n < n_0$, we use any convenient base-case multiplication algorithm to define $\mathsf{M}(n)$, and hence $\mathsf{T}(n)$.)

We prove by induction that $\mathsf{T}(n) \leq C$ for all $n \geq 2$. The choice of B ensures that the statement holds for $n < n_0$. Now assume that $n \geq n_0$. By (5.7), (5.1) and (5.6) we have

$$3rp < 6T^{1/d}p < 36n^{1/d}b = 36n^{1/1729} \lceil \log_2 n \rceil < n.$$

By induction,

$$\mathsf{T}(n) < 0.9998C + A \leq 0.9998C + 0.0002C = C.$$

Hence $\mathsf{T}(n) = O(1)$ and $\mathsf{M}(n) = O(n \log n)$. $\square$

5.4. Optimising the dimension threshold. It is possible to improve the factor $K = 1728$ appearing in Corollary 5.5, at the expense of introducing various technical complications into the algorithm. In this section we outline a number of such modifications that together reduce the constant to $K = 8 + \epsilon$, so that the modified algorithm achieves $\mathsf{M}(n) = O(n \log n)$ for any $d \geq 9$ (rather than $d \geq 1729$). The techniques described here are similar to those used in [25] to optimise the value of K in the Fürer-type bound $\mathsf{M}(n) = O(n \log n K^{\log^* n})$.

- (1) We may increase the chunk size from $b = \Theta(\log n)$ to $b = \Theta(\log n \log \log n)$, and then take the working precision to be $p = 2b + O(\log n) = (2 + o(1))b$ rather than $6b$. This improves K by a factor of 3. Note that the $O(\log n)$ term must be chosen large enough to ensure correct rounding at the end of the proof of Proposition 5.4. (We cannot take b as large as $(\log n)^2$, as is done in [25], because then the Gaussian resampling becomes too expensive.)
- (2) The choice of b in the previous item allows us to improve the term $3rp$ in Lemma 2.5 to $(2 + o(1))rp$, by packing the coefficients together more tightly in the Kronecker substitution step (the hypothesis $r < 2^{p-1}$ must also be tightened somewhat). This improves K by a factor of $3/2$.
- (3) In Bluestein's algorithm (see proof of Proposition 3.1), the multiplicand a is invariant, i.e., does not depend on the input vector u . To take advantage of this, we change the basic problem from multiplication of two arbitrary integers to multiplication of an arbitrary integer by a fixed integer. Consequently we save one forward transform in the proof of Proposition 5.3, reducing the factor 12 in (5.13) to 8. This improves K by a factor of $3/2$.
- (4) We may choose the primes s_i closer to t_i , so that instead of $T < 2S$ (see (5.12)) we have $T < (1 + o(1))S$. This improves K by a factor of 2. Some care is needed to avoid excessive precision loss in the Gaussian resampling step, due to the larger values of α and γ .
- (5) By allowing T to contain small odd prime factors, or alternatively by introducing flexibility into the choice of b , we may improve the choice of T to $(4 + o(1))n/b$ (see (5.6)). This improves K by a factor of 2.
- (6) We may change the basic problem from multiplication in $\mathbb{Z}$ to multiplication in $\mathbb{Z}[i]$. In step (1) of the proof of Proposition 5.4, the chunks F_j and G_j are taken in $\mathbb{Z}[i]$ instead of $\mathbb{Z}$, and in step (1) of the proof of Lemma 2.5, the evaluations lie in $\mathbb{Z}[i]$ instead of $\mathbb{Z}$. This improves K by a factor of 4, essentially by eliminating the factor 4 appearing in Lemma 2.5.
- (7) We may change the basic problem from multiplication in $\mathbb{Z}[i]$ to multiplication in $\mathbb{Z}[i]/(2^n + 1)\mathbb{Z}[i]$. Note that the Kronecker substitution in Lemma 2.5 maps the multiplication modulo $y^r + 1$ naturally onto this problem. This improves K by a factor of 2, because it avoids the degree growth in step (1) in the proof of Proposition 5.4. It also introduces a technical difficulty into that step: to reduce a multiplication modulo $2^n + 1$ to a polynomial multiplication modulo $x^S - 1$ (or $x^S + 1$), we must split an n -bit integer into S chunks, even though n will not in general be divisible by S . This may be addressed by means of the Crandall–Fagin algorithm [9].

REFERENCES

1. R. Agarwal and J. Cooley, *New algorithms for digital convolution*, IEEE Transactions on Acoustics, Speech, and Signal Processing **25** (1977), no. 5, 392–410.
2. L. I. Bluestein, *A linear filtering approach to the computation of discrete Fourier transform*, IEEE Transactions on Audio and Electroacoustics **18** (1970), no. 4, 451–455.
3. J. M. Borwein and P. B. Borwein, *Pi and the AGM*, Canadian Mathematical Society Series of Monographs and Advanced Texts, John Wiley & Sons, Inc., New York, 1987, A study in analytic number theory and computational complexity, A Wiley-Interscience Publication. MR 877728
4. A. Bostan, P. Gaudry, and É. Schost, *Linear recurrences with polynomial coefficients and application to integer factorization and Cartier-Manin operator*, SIAM J. Comput. **36** (2007), no. 6, 1777–1806. MR 2299425 (2008a:11156)

5. R. P. Brent and P. Zimmermann, *Modern Computer Arithmetic*, Cambridge Monographs on Applied and Computational Mathematics, vol. 18, Cambridge University Press, Cambridge, 2011. MR 2760886
6. S. A. Cook and S. O. Aanderaa, *On the minimum computation time of functions*, Trans. Amer. Math. Soc. **142** (1969), 291–314. MR 0249212
7. J. W. Cooley and J. W. Tukey, *An algorithm for the machine calculation of complex Fourier series*, Math. Comp. **19** (1965), 297–301. MR 0178586
8. S. Covanov and E. Thomé, *Fast integer multiplication using generalized Fermat primes*, Math. Comp. **88** (2019), no. 317, 1449–1477. MR 3904152
9. R. Crandall and B. Fagin, *Discrete weighted transforms and large-integer arithmetic*, Math. Comp. **62** (1994), no. 205, 305–324. MR 1185244
10. A. De, P. Kurur, C. Saha, and R. Saptharishi, *Fast integer multiplication using modular arithmetic*, SIAM Journal on Computing **42** (2013), no. 2, 685–699.
11. A. Dutt and V. Rokhlin, *Fast Fourier transforms for nonequispaced data*, SIAM J. Sci. Comput. **14** (1993), no. 6, 1368–1393. MR 1241591
12. M. Fürer, *Faster integer multiplication*, STOC’07—Proceedings of the 39th Annual ACM Symposium on Theory of Computing, ACM, New York, 2007, pp. 57–66. MR 2402428 (2009e:68124)
13. ———, *Faster integer multiplication*, SIAM J. Comput. **39** (2009), no. 3, 979–1005. MR 2538847 (2011b:68296)
14. J. von zur Gathen and J. Gerhard, *Modern Computer Algebra*, 3rd ed., Cambridge University Press, Cambridge, 2013. MR 3087522
15. P. Gaudry, A. Kruppa, and P. Zimmermann, *A GMP-based implementation of Schönhage-Strassen’s large integer multiplication algorithm*, ISSAC 2007, ACM, New York, 2007, pp. 167–174. MR 2396199
16. I. J. Good, *The interaction algorithm and practical Fourier analysis*, J. Roy. Statist. Soc. Ser. B **20** (1958), 361–372. MR 0102888 (21 #1674)
17. T. Granlund, *The GNU Multiple Precision Arithmetic Library (Version 6.1.2)*, <http://gmplib.org/>.
18. D. Harvey, *Faster truncated integer multiplication*, <https://arxiv.org/abs/1703.00640>, 2017.
19. D. Harvey and J. van der Hoeven, *Faster integer and polynomial multiplication using cyclotomic coefficient rings*, <https://arxiv.org/abs/1712.03693>, 2017.
20. ———, *On the complexity of integer matrix multiplication*, J. Symbolic Comput. **89** (2018), 1–8. MR 3804803
21. ———, *Faster integer multiplication using plain vanilla FFT primes*, Math. Comp. **88** (2019), no. 315, 501–514. MR 3854069
22. ———, *Faster integer multiplication using short lattice vectors*, Proceedings of the Thirteenth Algorithmic Number Theory Symposium, Open Book Series 2 (R. Scheidler and J. Sorenson, eds.), Mathematical Sciences Publishers, Berkeley, 2019, pp. 293–310.
23. ———, *Faster polynomial multiplication over finite fields using cyclotomic coefficient rings*, J. Complexity **54** (2019), 101404, 18. MR 3983215
24. ———, *Polynomial multiplication over finite fields in time $O(n \log n)$* , HAL preprint <http://hal.archives-ouvertes.fr/hal-02070816>, 2019.
25. D. Harvey, J. van der Hoeven, and G. Lecerf, *Even faster integer multiplication*, J. Complexity **36** (2016), 1–30. MR 3530637
26. ———, *Faster polynomial multiplication over finite fields*, J. ACM **63** (2017), no. 6, 52:1–52:23.
27. D. R. Heath-Brown, *Zero-free regions for Dirichlet L -functions, and the least prime in an arithmetic progression*, Proc. London Math. Soc. (3) **64** (1992), no. 2, 265–338. MR 1143227 (93a:11075)
28. M. T. Heideman, D. H. Johnson, and C. S. Burrus, *Gauss and the history of the fast Fourier transform*, Arch. Hist. Exact Sci. **34** (1985), no. 3, 265–277. MR 815154 (87f:01018)
29. J. van der Hoeven, *Faster relaxed multiplication*, ISSAC 2014—Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation, ACM, New York, 2014, pp. 405–412. MR 3239953
30. ———, *Faster Chinese remaindering*, HAL preprint <http://hal.archives-ouvertes.fr/hal-01403810>, 2016.

31. D. E. Knuth, *The Art of Computer Programming. Vol. 3*, Addison-Wesley, Reading, MA, 1998, Sorting and searching, Second edition [of MR0445948]. MR 3077154
32. H. J. Nussbaumer, *Fast polynomial transform algorithms for digital convolution*, IEEE Trans. Acoust. Speech Signal Process. **28** (1980), no. 2, 205–215. MR MR563380 (80m:94004)
33. H. J. Nussbaumer and P. Quandalle, *Computation of convolutions and discrete Fourier transforms by polynomial transforms*, IBM J. Res. Develop. **22** (1978), no. 2, 134–144. MR 0471260
34. ———, *Fast computation of discrete Fourier transforms using polynomial transforms*, IEEE Trans. Acoust. Speech Signal Process. **27** (1979), no. 2, 169–181. MR 523618
35. C. H. Papadimitriou, *Computational Complexity*, Addison-Wesley Publishing Company, Reading, MA, 1994. MR 1251285 (95f:68082)
36. M. S. Paterson, M. J. Fischer, and A. R. Meyer, *An improved overlap argument for on-line multiplication*, (1974), 97–111. SIAM-AMS Proc., Vol. VII. MR 0423875
37. J. M. Pollard, *The fast Fourier transform in a finite field*, Math. Comp. **25** (1971), 365–374. MR 0301966
38. C. M. Rader, *Discrete Fourier transforms when the number of data samples is prime*, Proc. IEEE **56** (1968), no. 6, 1107–1108.
39. J. B. Rosser and L. Schoenfeld, *Approximate formulas for some functions of prime numbers*, Illinois J. Math. **6** (1962), 64–94. MR 0137689
40. A. Schönhage and V. Strassen, *Schnelle Multiplikation grosser Zahlen*, Computing (Arch. Elektron. Rechnen) **7** (1971), 281–292. MR 0292344 (45 #1431)
41. M. Sipser, *Introduction to the Theory of Computation*, 2nd ed., Thomson Course Technology, 2006.
42. L. H. Thomas, *Using computers to solve problems in physics*, Applications of digital computers **458** (1963), 42–57.
43. T. Xylouris, *On the least prime in an arithmetic progression and estimates for the zeros of Dirichlet L -functions*, Acta Arith. **150** (2011), no. 1, 65–91. MR 2825574 (2012m:11129)

SCHOOL OF MATHEMATICS AND STATISTICS, UNIVERSITY OF NEW SOUTH WALES, SYDNEY NSW 2052, AUSTRALIA

Email address: `d.harvey@unsw.edu.au`

CNRS, LABORATOIRE D'INFORMATIQUE, ÉCOLE POLYTECHNIQUE, 91128 PALAISEAU, FRANCE

Email address: `vdhoeven@lix.polytechnique.fr`